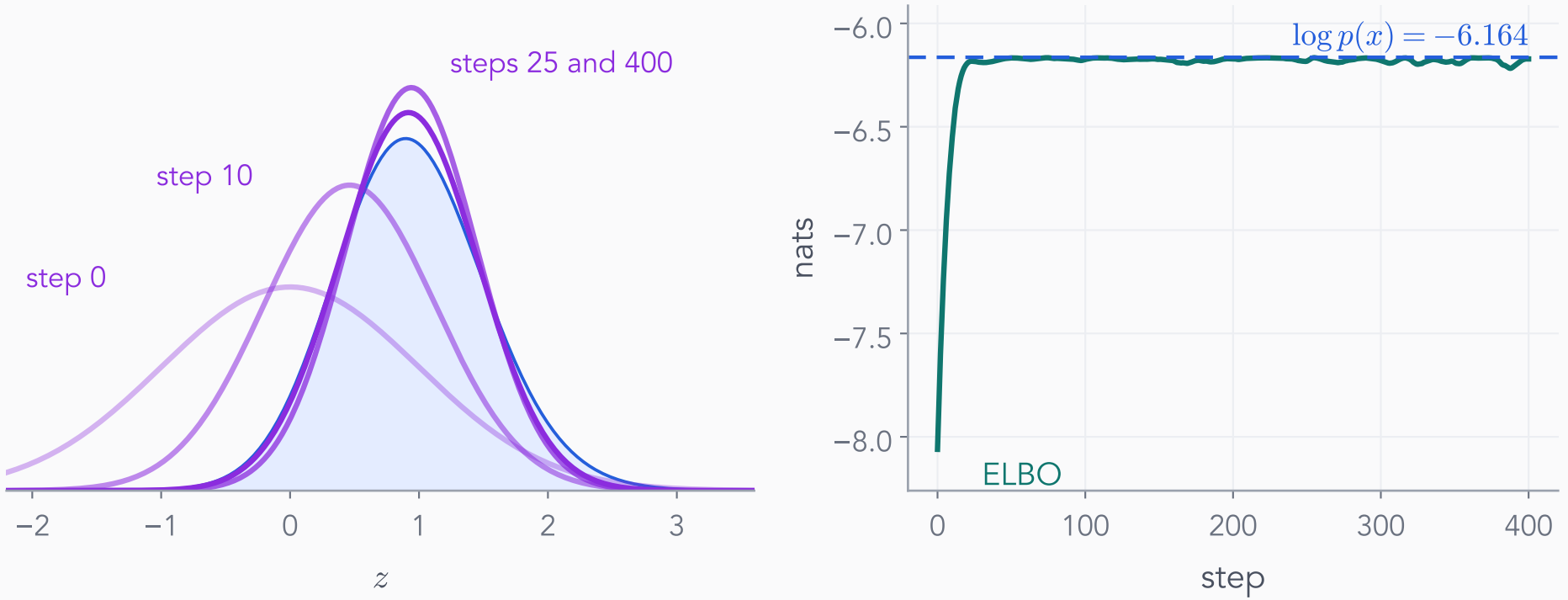
KL divergence and variational inference

Built slowly on a coin, two bumps and a few shapes in a plane



A simple bell curve q (purple) slides onto a posterior that has no formula (blue), while a number called the ELBO climbs to $\log p(x)$. By the end of today, every word of that sentence will mean something precise.

Nipun Batra · IIT Gandhinagar

THE PLAN

One problem, two tools, one method, and where it leads

1. **Bayes, as a reminder**

   prior, likelihood, posterior, evidence, on ten coin tosses

2. **Why the posterior is hard**

   the evidence integral, and three ways around it

THE PLAN

One problem, two tools, one method, and where it leads

1. **Bayes, as a reminder**
   prior, likelihood, posterior, evidence, on ten coin tosses
2. **Why the posterior is hard**
   the evidence integral, and three ways around it
3. **Averages from samples**
   what $\mathbb{E}$ means, and Monte Carlo
4. **KL divergence**
   a number for "how far apart" two distributions are

THE PLAN

One problem, two tools, one method, and where it leads

1. **Bayes, as a reminder**
   prior, likelihood, posterior, evidence, on ten coin tosses
2. **Why the posterior is hard**
   the evidence integral, and three ways around it
3. **Averages from samples**
   what $\mathbb{E}$ means, and Monte Carlo
4. **KL divergence**
   a number for "how far apart" two distributions are
5. **Variational inference**
   pick the closest simple q; the ELBO, derived one move per slide
6. **Climbing the ELBO**
   estimates from samples, gradients through a random draw, live

THE PLAN

One problem, two tools, one method, and where it leads

1. **Bayes, as a reminder**
   prior, likelihood, posterior, evidence, on ten coin tosses
2. **Why the posterior is hard**
   the evidence integral, and three ways around it
3. **Averages from samples**
   what $\mathbb{E}$ means, and Monte Carlo
4. **KL divergence**
   a number for "how far apart" two distributions are
5. **Variational inference**
   pick the closest simple q; the ELBO, derived one move per slide
6. **Climbing the ELBO**
   estimates from samples, gradients through a random draw, live
7. **Beyond one number**
   shapes a simple q cannot fit
8. **Many data points**
   learn to predict q: the VAE's encoder

> **Every piece of the VAE's loss appears here first, on toys small enough to check by hand.**

PART 1 OF 8

A reminder: prior, likelihood, posterior

Variational inference approximates exactly one thing: a **posterior**. So before any new maths, rebuild Bayes' rule on the simplest data there is: ten tosses of a coin.

1. **Bayes: prior, likelihood, posterior**
2. Why the posterior is hard
3. Averages from samples
4. KL divergence
5. Variational inference and the ELBO
6. Climbing the ELBO: gradients through samples
7. Beyond one number
8. Many data points: towards the VAE

THE DATA

Ten tosses: 8 heads, 2 tails

H H T H H H H T H H

What is this coin's chance of heads? Call it θ.

THE DATA

Ten tosses: 8 heads, 2 tails

H H T H H H H T H H

What is this coin's chance of heads? Call it θ.

The obvious answer: $8/10 = 0.8$. That is the maximum-likelihood estimate.

THE DATA

Ten tosses: 8 heads, 2 tails

H H T H H H H T H H

What is this coin's chance of heads? Call it θ.

The obvious answer: $8/10 = 0.8$. That is the maximum-likelihood estimate.

But ten tosses do not rule out $\theta = 0.7$, or even 0.5. We want a whole distribution over θ: how strongly we believe each value.

Bayes' rule turns "what I believed before" and "what I saw" into "what I believe now".

START SMALL: PRETEND THERE ARE ONLY THREE POSSIBLE COINS

Bayes' rule as a table you can do by hand

coin θ	prior $p(\theta)$
0.2	1/3
0.5	1/3
0.8	1/3

START SMALL: PRETEND THERE ARE ONLY THREE POSSIBLE COINS

Bayes' rule as a table you can do by hand

coin θ	prior $p(\theta)$	likelihood $p(x \mid \theta) = \theta^8(1-\theta)^2$
0.2	1/3	0.016×10^{-4}
0.5	1/3	9.77×10^{-4}
0.8	1/3	67.11×10^{-4}

START SMALL: PRETEND THERE ARE ONLY THREE POSSIBLE COINS

Bayes' rule as a table you can do by hand

coin θ	prior $p(\theta)$	likelihood $p(x \mid \theta) = \theta^8(1-\theta)^2$	joint: prior × likelihood
0.2	1/3	0.016×10^{-4}	0.005×10^{-4}
0.5	1/3	9.77×10^{-4}	3.26×10^{-4}
0.8	1/3	67.11×10^{-4}	22.37×10^{-4}

START SMALL: PRETEND THERE ARE ONLY THREE POSSIBLE COINS

Bayes' rule as a table you can do by hand

coin θ	prior $p(\theta)$	likelihood $p(x \mid \theta) = \theta^8(1-\theta)^2$	joint: prior × likelihood
0.2	1/3	0.016×10^{-4}	0.005×10^{-4}
0.5	1/3	9.77×10^{-4}	3.26×10^{-4}
0.8	1/3	67.11×10^{-4}	22.37×10^{-4}
sum	**1**		25.63×10^{-4}

> Add up the joint column: 25.63×10^{-4}. Keep this number; it has a name.

START SMALL: PRETEND THERE ARE ONLY THREE POSSIBLE COINS

Bayes' rule as a table you can do by hand

coin θ	prior $p(\theta)$	likelihood $p(x \mid \theta) = \theta^8(1-\theta)^2$	joint: prior × likelihood	posterior $p(\theta \mid x)$
0.2	1/3	0.016×10^{-4}	0.005×10^{-4}	0.0002
0.5	1/3	9.77×10^{-4}	3.26×10^{-4}	0.127
0.8	1/3	67.11×10^{-4}	22.37×10^{-4}	0.873
sum	**1**		25.63×10^{-4}	**1**

> **Posterior = joint ÷ sum. The coin with $\theta = 0.8$ now has 87% of our belief. The sum we divided by is $p(x)$, the evidence.**

THE SAME TABLE, IN SYMBOLS

Bayes' rule, and the troublemaker at the bottom

$$\underbrace{p(\theta \mid x)}_{\text{posterior}} = \frac{\overbrace{p(\theta)}^{\text{prior}}\ \overbrace{p(x \mid \theta)}^{\text{likelihood}}}{\underbrace{p(x)}_{\text{evidence}}}$$

THE SAME TABLE, IN SYMBOLS

Bayes' rule, and the troublemaker at the bottom

$$\underbrace{p(\theta \mid x)}_{\text{posterior}} = \frac{\overbrace{p(\theta)}^{\text{prior}}\ \overbrace{p(x \mid \theta)}^{\text{likelihood}}}{\underbrace{p(x)}_{\text{evidence}}}$$

$$p(x) = \sum_{\theta \in \{0.2,\, 0.5,\, 0.8\}} p(\theta)\, p(x \mid \theta)$$

three coins: a sum of three terms

THE SAME TABLE, IN SYMBOLS

Bayes' rule, and the troublemaker at the bottom

$$\underbrace{p(\theta \mid x)}_{\text{posterior}} = \frac{\overbrace{p(\theta)}^{\text{prior}}\ \overbrace{p(x \mid \theta)}^{\text{likelihood}}}{\underbrace{p(x)}_{\text{evidence}}}$$

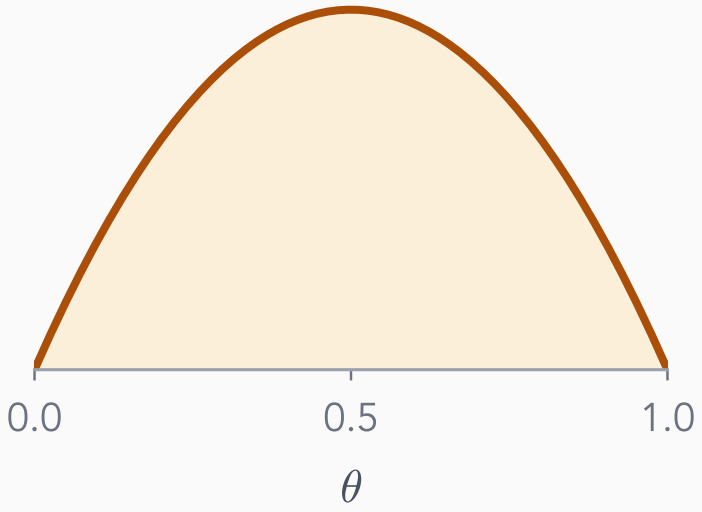
$$p(x) = \sum_{\theta \in \{0.2,\, 0.5,\, 0.8\}} p(\theta)\, p(x \mid \theta) \qquad \text{three coins: a sum of three terms}$$

$$p(x) = \int_0^1 p(\theta)\, p(x \mid \theta)\, d\theta \qquad \text{every } \theta \text{ between 0 and 1: an integral}$$

> **The evidence is the joint, added up over every possible value of the hidden quantity. Watch it: it is the whole difficulty.**

EVERY VALUE OF θ AT ONCE

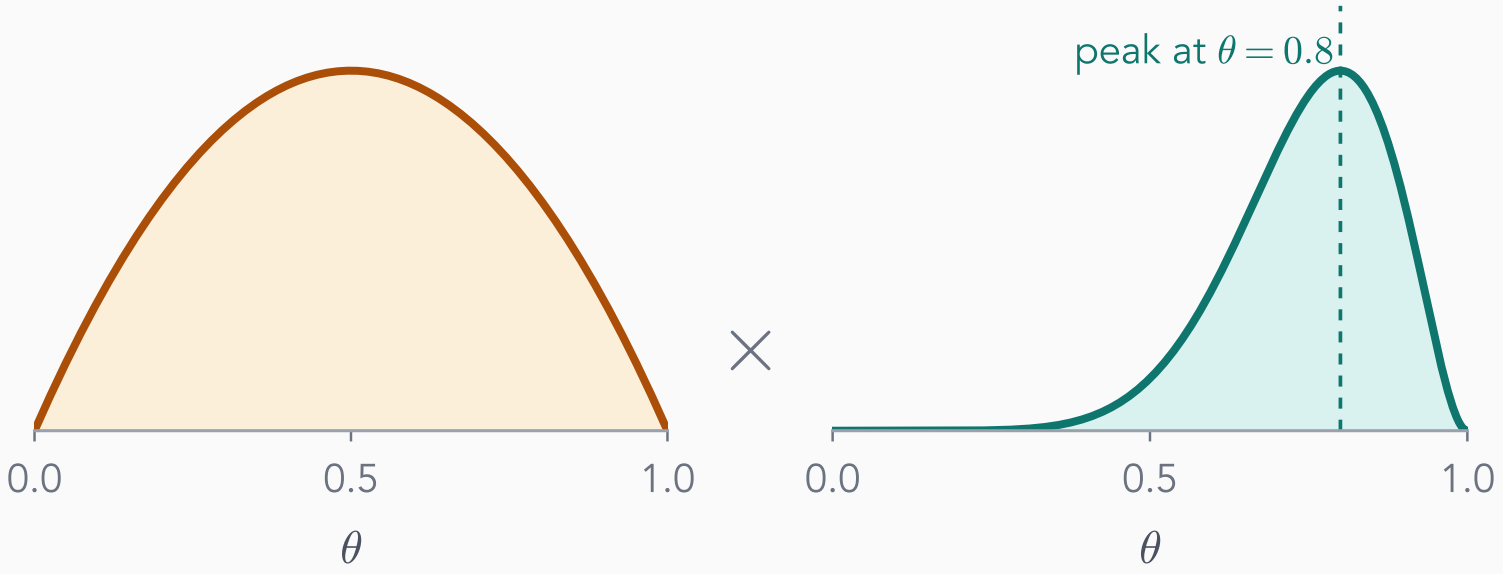
Prior × likelihood, rescaled: the posterior



prior: $\text{Beta}(2, 2) = 6\theta(1 - \theta)$
a gentle preference for fair coins

EVERY VALUE OF θ AT ONCE

Prior × likelihood, rescaled: the posterior

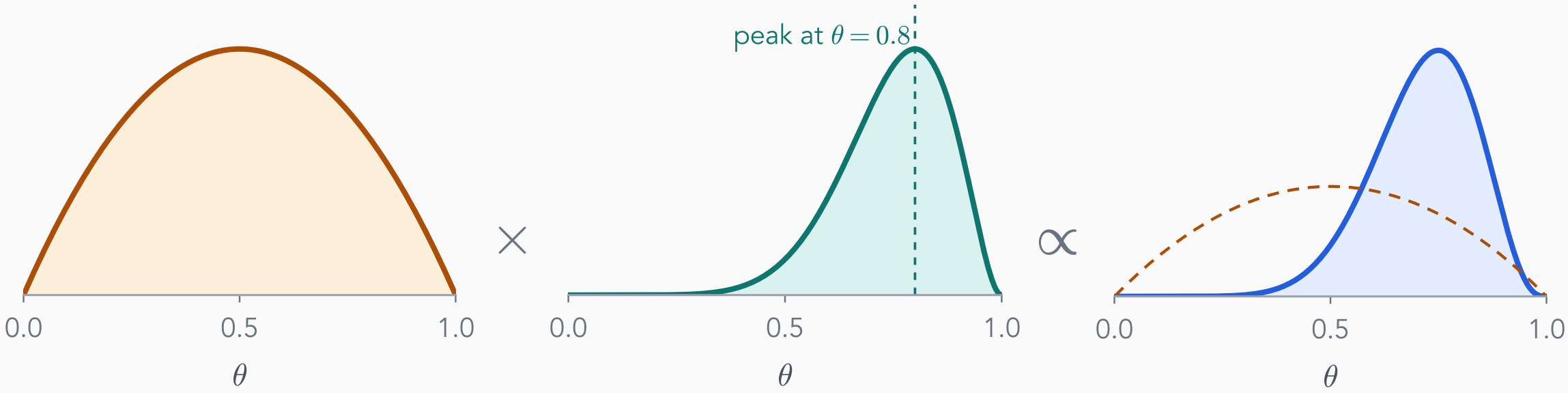


prior: $\mathrm{Beta}(2, 2) = 6\theta(1 - \theta)$
a gentle preference for fair coins

likelihood: $\theta^8(1 - \theta)^2$
what the ten tosses say

EVERY VALUE OF θ AT ONCE

Prior × likelihood, rescaled: the posterior



prior: $\mathrm{Beta}(2, 2) = 6\theta(1-\theta)$
a gentle preference for fair coins

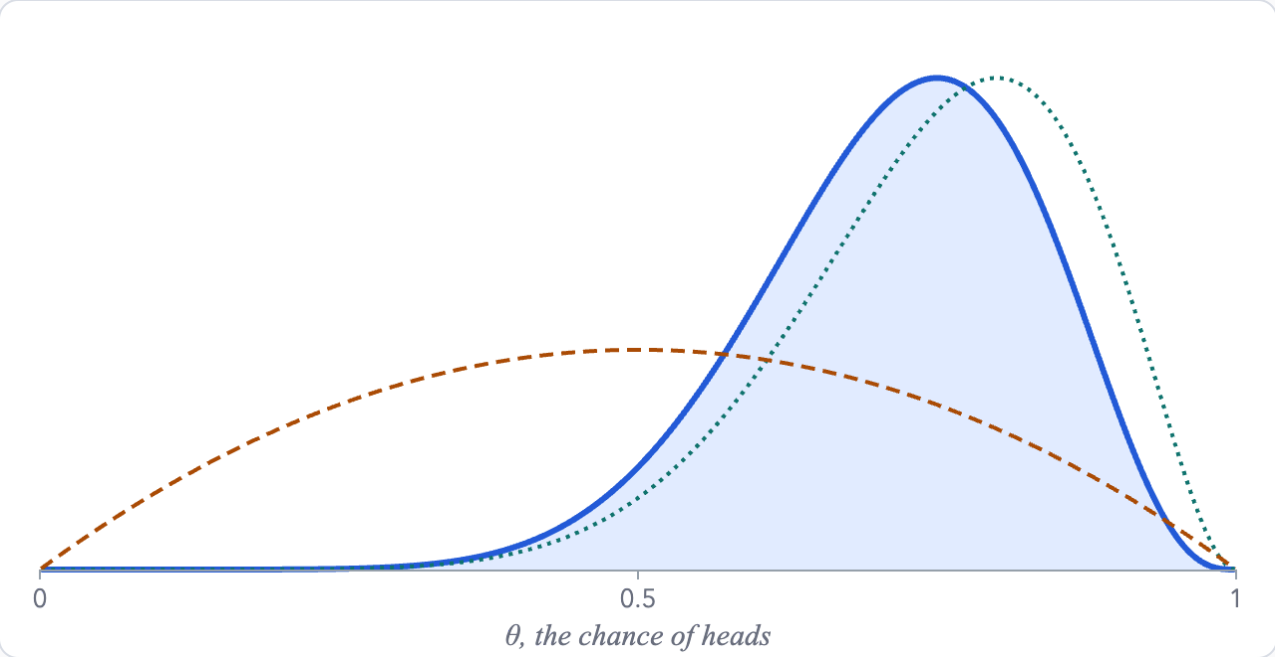
likelihood: $\theta^8(1-\theta)^2$
what the ten tosses say

posterior: $\mathrm{Beta}(10, 4)$
(prior dashed, for comparison)

Multiply the two curves point by point, rescale to area 1. Posterior mean 0.714: between the prior's 0.5 and the data's 0.8.

TRY IT

Toss by toss, the posterior sharpens



HHTHHHHTHH

8 heads, **2** tails

posterior: $\text{Beta}(2 + \text{heads},\ 2 + \text{tails})$

$= \text{Beta}(\mathbf{10, 4})$

posterior mean: **0.714**

prior likelihood (shape) posterior

Each head multiplies in one more θ, each tail one more $(1 - \theta)$. More tosses: a narrower posterior.

FOR THIS PRIOR, WE GOT LUCKY

The evidence has a formula

$$p(x) = \int_0^1 \underbrace{6\theta(1-\theta)}_{\text{prior}} \underbrace{\theta^8(1-\theta)^2}_{\text{likelihood}}\, d\theta = 6\int_0^1 \theta^9(1-\theta)^3\, d\theta = 6\,B(10,4)$$

FOR THIS PRIOR, WE GOT LUCKY

The evidence has a formula

$$p(x) = \int_0^1 \underbrace{6\theta(1-\theta)}_{\text{prior}} \underbrace{\theta^8(1-\theta)^2}_{\text{likelihood}}\, d\theta = 6\int_0^1 \theta^9(1-\theta)^3\, d\theta = 6\,B(10,4)$$

$B(a,b)$, the Beta function, is a tabulated integral: $B(10,4) = \dfrac{9!\;3!}{13!}$. So $p(x) = 0.00210$ and $\log p(x) = -6.167$.

FOR THIS PRIOR, WE GOT LUCKY

The evidence has a formula

$$p(x) = \int_0^1 \underbrace{6\theta(1-\theta)}_{\text{prior}} \underbrace{\theta^8(1-\theta)^2}_{\text{likelihood}}\, d\theta = 6\int_0^1 \theta^9(1-\theta)^3\, d\theta = 6\,B(10,4)$$

$B(a,b)$, the Beta function, is a tabulated integral: $B(10,4) = \dfrac{9!\;3!}{13!}$. So $p(x) = 0.00210$ and $\log p(x) = -6.167$.

The integrand is a power of θ times a power of $(1-\theta)$: the same form as the prior. That is **conjugacy**. It is rare, and it is the only reason this integral could be done.

Change the prior slightly and the luck runs out. That is Part 2.

THE SAME FIVE WORDS, NEXT LECTURE

The coin is a tiny version of the VAE's problem

word	the coin	the VAE
hidden quantity	θ, the chance of heads	
data x	ten tosses	
prior	$\mathrm{Beta}(2, 2)$	
likelihood	$\theta^8(1-\theta)^2$	
posterior	$\mathrm{Beta}(10, 4)$	
evidence	$6\,B(10, 4) = 0.00210$	

THE SAME FIVE WORDS, NEXT LECTURE

The coin is a tiny version of the VAE's problem

word	the coin	the VAE
hidden quantity	θ, the chance of heads	a code z of a few numbers
data x	ten tosses	one image: 784 pixels
prior	$\mathrm{Beta}(2, 2)$	$p(z) = \mathcal{N}(0, I)$
likelihood	$\theta^8(1-\theta)^2$	$p(x \mid z)$: the **decoder**, a neural network
posterior	$\mathrm{Beta}(10, 4)$	
evidence	$6\,B(10, 4) = 0.00210$	

THE SAME FIVE WORDS, NEXT LECTURE

The coin is a tiny version of the VAE's problem

word	the coin	the VAE
hidden quantity	θ, the chance of heads	a code z of a few numbers
data x	ten tosses	one image: 784 pixels
prior	$\mathrm{Beta}(2, 2)$	$p(z) = \mathcal{N}(0, I)$
likelihood	$\theta^8(1-\theta)^2$	$p(x \mid z)$: the **decoder**, a neural network
posterior	$\mathrm{Beta}(10, 4)$	"which codes could have made this image?": no formula
evidence	$6\,B(10, 4) = 0.00210$	$\int p(z)\,p(x \mid z)\,dz$: no formula

> **The VAE needs a posterior for every training image, and it never has a formula. This lecture builds the tool for that.**

PART 2 OF 8

When the evidence has no formula

Keep the same ten tosses. Change only the prior, to one that looks like a tiny **decoder**. Bayes' rule still holds, but the integral at its bottom can no longer be done. That one change is the whole reason variational inference exists.

✓ Bayes: prior, likelihood, posterior

2 **Why the posterior is hard**

3 Averages from samples

4 KL divergence

5 Variational inference and the ELBO

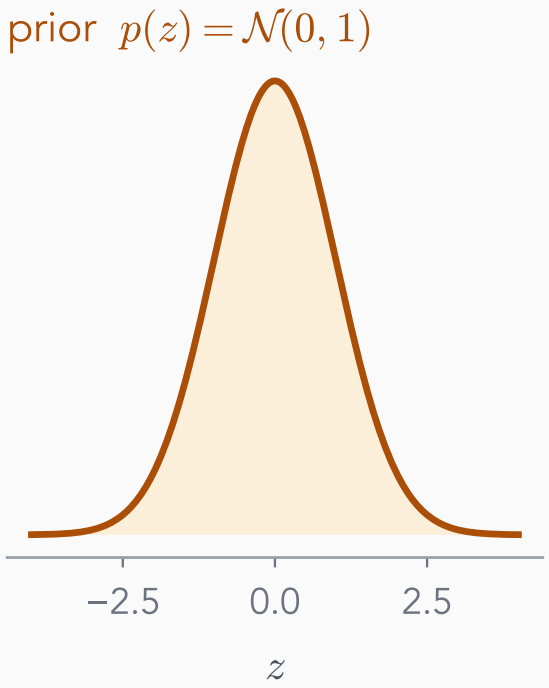
6 Climbing the ELBO: gradients through samples

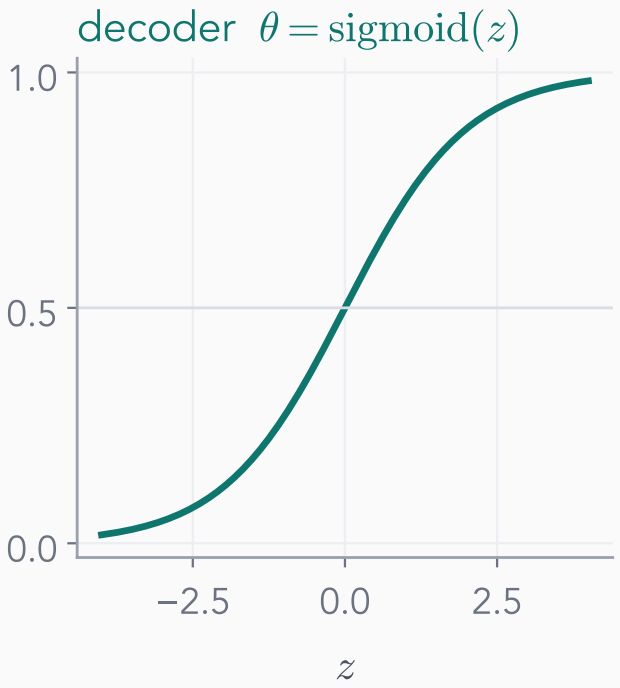
7 Beyond one number

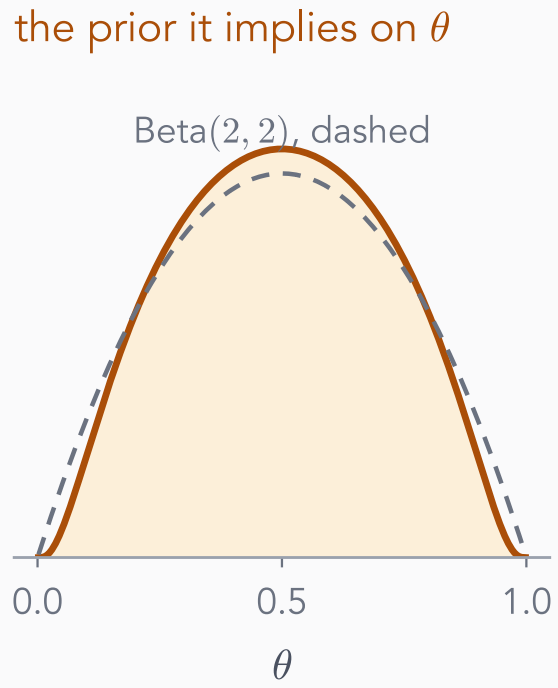
8 Many data points: towards the VAE

SAME COIN, A DIFFERENT STORY FOR θ

A one-number decoder





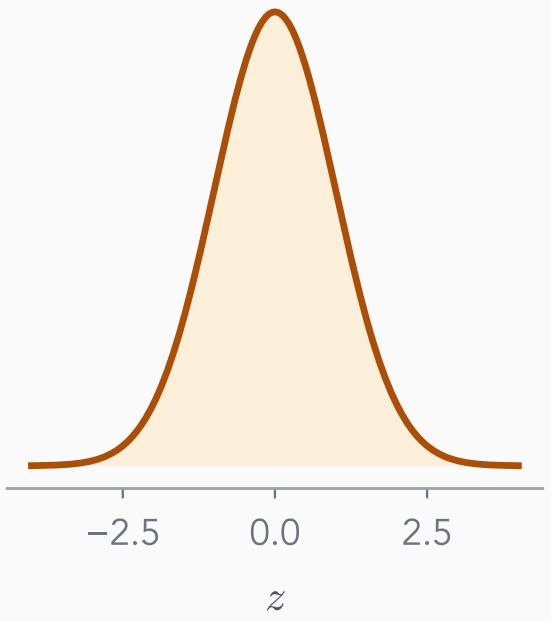


Draw a hidden number $z \sim \mathcal{N}(0, 1)$, squash it: $\theta = \text{sigmoid}(z)$, toss ten times with that θ.

SAME COIN, A DIFFERENT STORY FOR θ

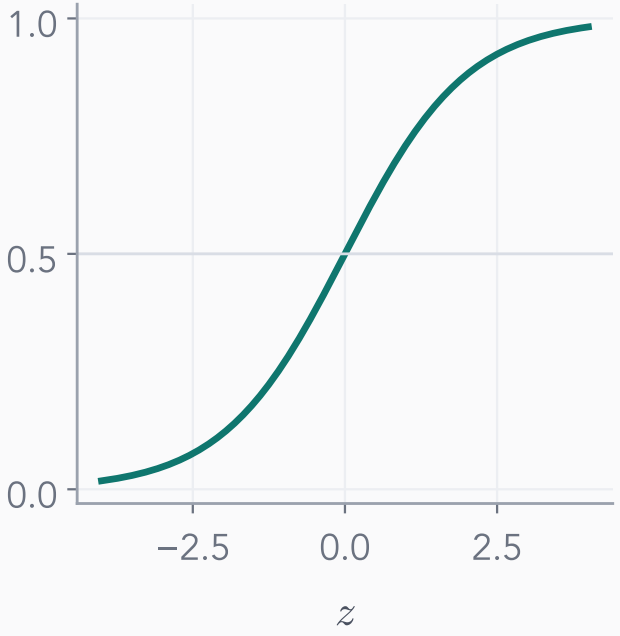
A one-number decoder

prior $p(z) = \mathcal{N}(0, 1)$



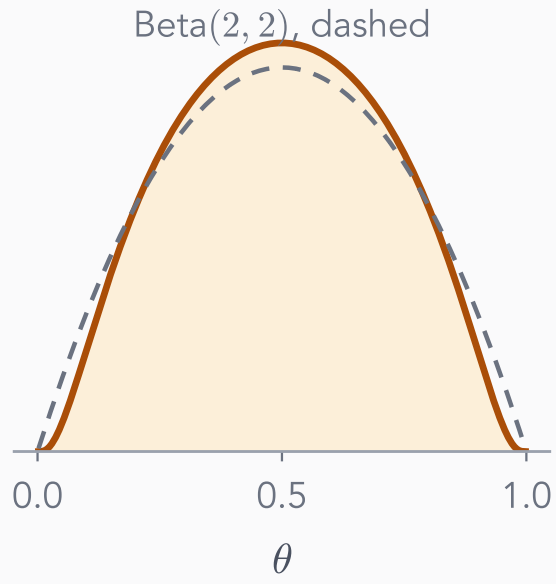
Draw a hidden number $z \sim \mathcal{N}(0, 1)$, squash it: $\theta = \mathrm{sigmoid}(z)$, toss ten times with that θ.

decoder $\theta = \mathrm{sigmoid}(z)$



The VAE's story in miniature: a prior on a code, then a network that turns the code into probabilities.

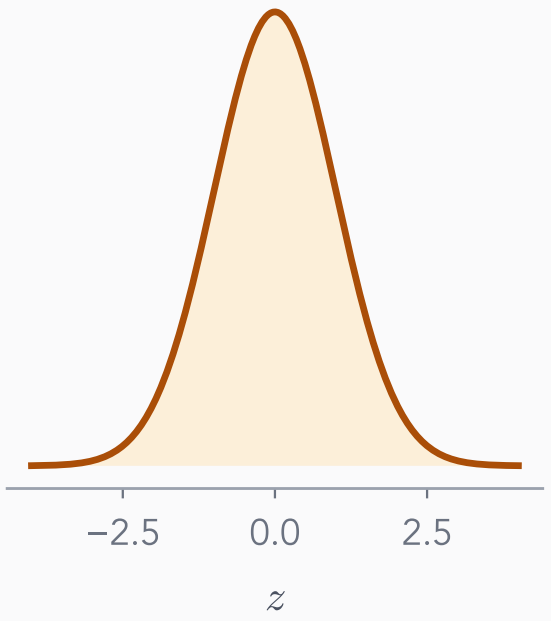
the prior it implies on θ



SAME COIN, A DIFFERENT STORY FOR θ

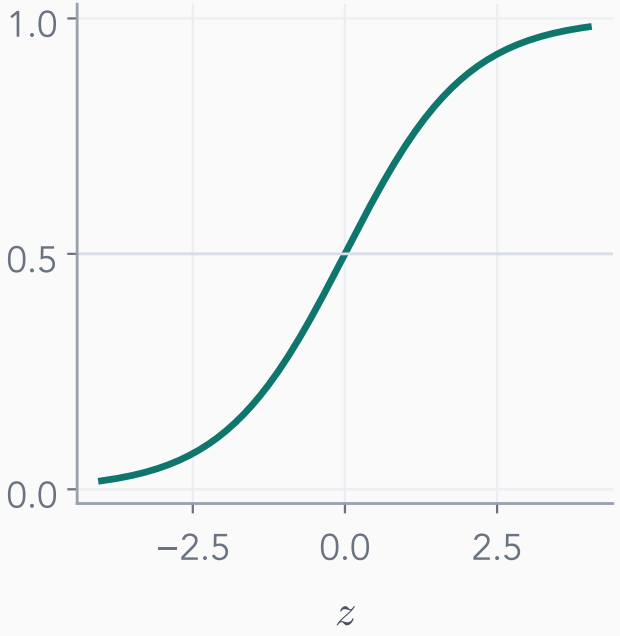
A one-number decoder

prior $p(z) = \mathcal{N}(0, 1)$



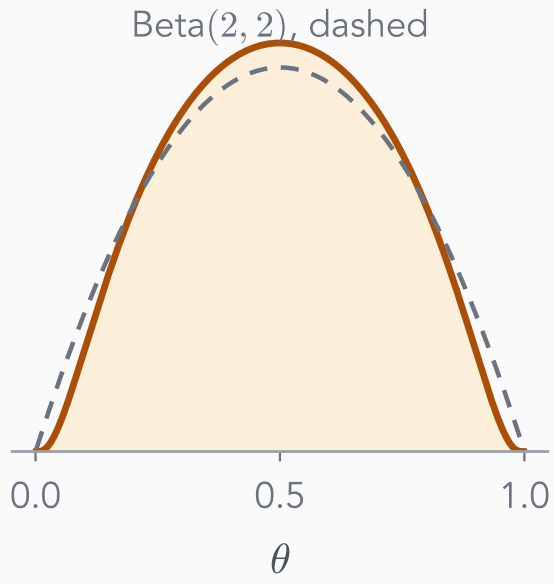
Draw a hidden number $z \sim \mathcal{N}(0, 1)$, squash it: $\theta = \mathrm{sigmoid}(z)$, toss ten times with that θ.

decoder $\theta = \mathrm{sigmoid}(z)$



The VAE's story in miniature: a prior on a code, then a network that turns the code into probabilities.

the prior it implies on θ



The prior it implies on θ is almost Part 1's $\mathrm{Beta}(2, 2)$: nearly the same beliefs.

The hidden quantity is now z, a real number. From here on, z is what we infer.

BAYES' RULE, EXACTLY AS BEFORE

The posterior over z

$$p(z \mid x) = \frac{\mathcal{N}(z; 0, 1)\ \text{sigmoid}(z)^8\ \left(1 - \text{sigmoid}(z)\right)^2}{\int \mathcal{N}(z; 0, 1)\ \text{sigmoid}(z)^8\ \left(1 - \text{sigmoid}(z)\right)^2\, dz}$$

BAYES' RULE, EXACTLY AS BEFORE

The posterior over z

$$p(z \mid x) = \frac{\mathcal{N}(z; 0, 1)\ \mathrm{sigmoid}(z)^8\ \left(1 - \mathrm{sigmoid}(z)\right)^2}{\int \mathcal{N}(z; 0, 1)\ \mathrm{sigmoid}(z)^8\ \left(1 - \mathrm{sigmoid}(z)\right)^2\, dz}$$

The top: easy

A bell curve times a few sigmoids. Pick any z and it is one line of arithmetic.

BAYES' RULE, EXACTLY AS BEFORE

The posterior over z

$$p(z \mid x) = \frac{\mathcal{N}(z; 0, 1)\ \mathrm{sigmoid}(z)^8\ \left(1 - \mathrm{sigmoid}(z)\right)^2}{\int \mathcal{N}(z; 0, 1)\ \mathrm{sigmoid}(z)^8\ \left(1 - \mathrm{sigmoid}(z)\right)^2\, dz}$$

The top: easy

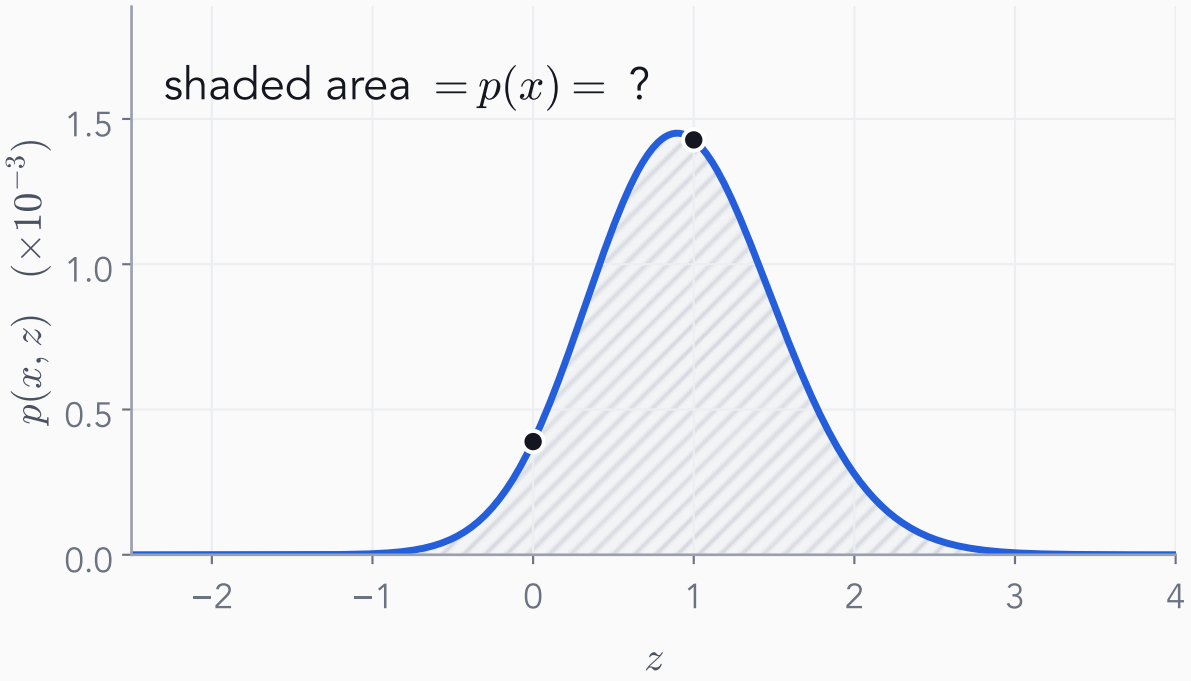
A bell curve times a few sigmoids. Pick any z and it is one line of arithmetic.

The bottom: no formula

No table of integrals has this one. No trick turns it into a Beta function: the bell curve and the sigmoids do not combine.

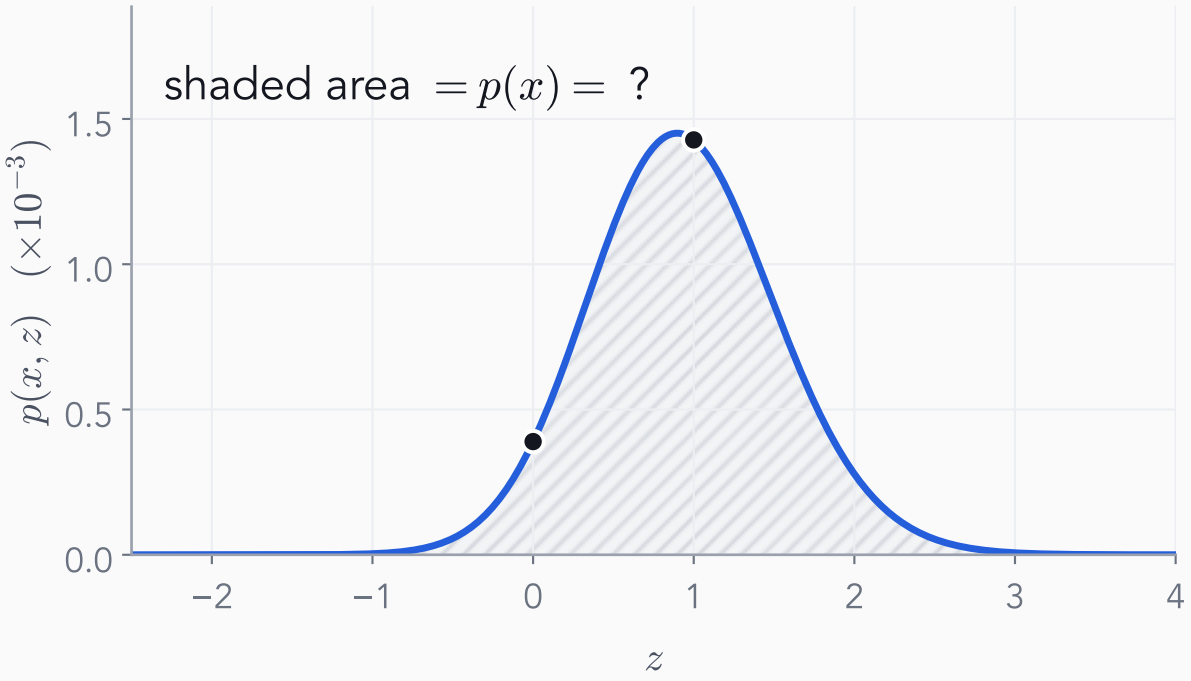
"Intractable" means exactly this: we can write the integral down, but not work it out.

Any point of the curve, but not its area




```
def log_joint(z):           # log p(x, z), any z
    return (Normal(0, 1).log_prob(z)
            + 8 * F.logsigmoid(z)
            + 2 * F.logsigmoid(-z))
```

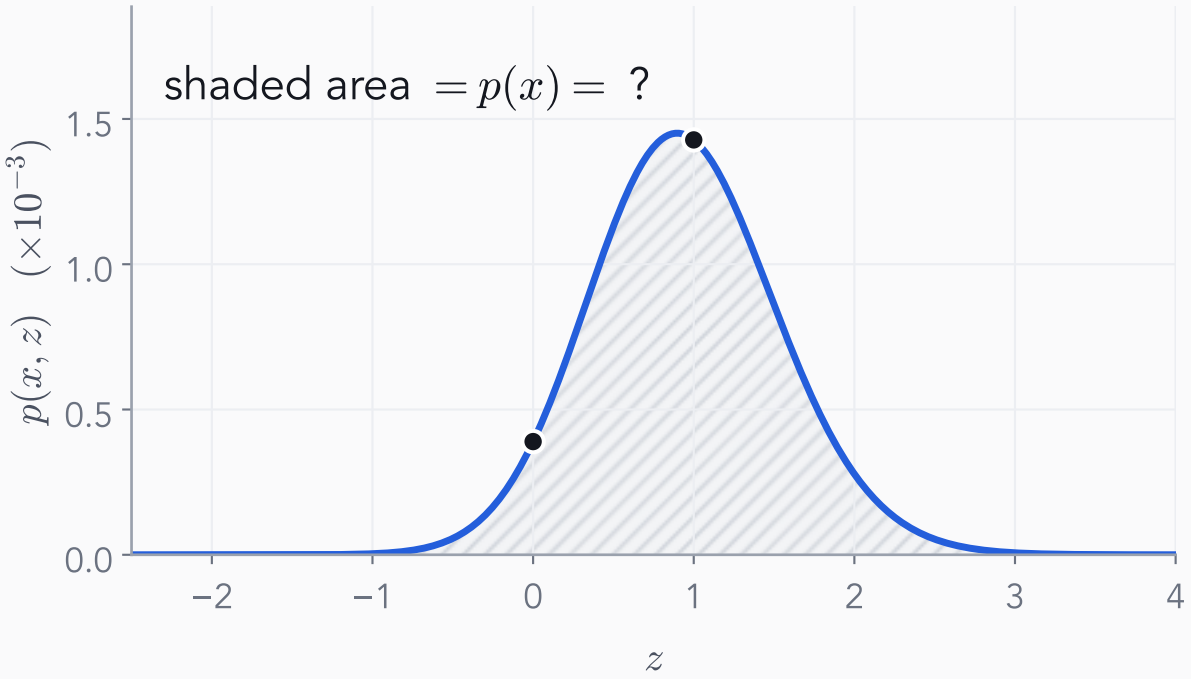
Any point of the curve, but not its area




```python
def log_joint(z):           # log p(x, z), any z
    return (Normal(0, 1).log_prob(z)
            + 8 * F.logsigmoid(z)
            + 2 * F.logsigmoid(-z))
```

The two dots: $\log p(x, z) = -7.85$ at $z = 0$ and -6.552 at $z = 1$.

Any point of the curve, but not its area




```
def log_joint(z):           # log p(x, z), any z
    return (Normal(0, 1).log_prob(z)
            + 8 * F.logsigmoid(z)
            + 2 * F.logsigmoid(-z))
```

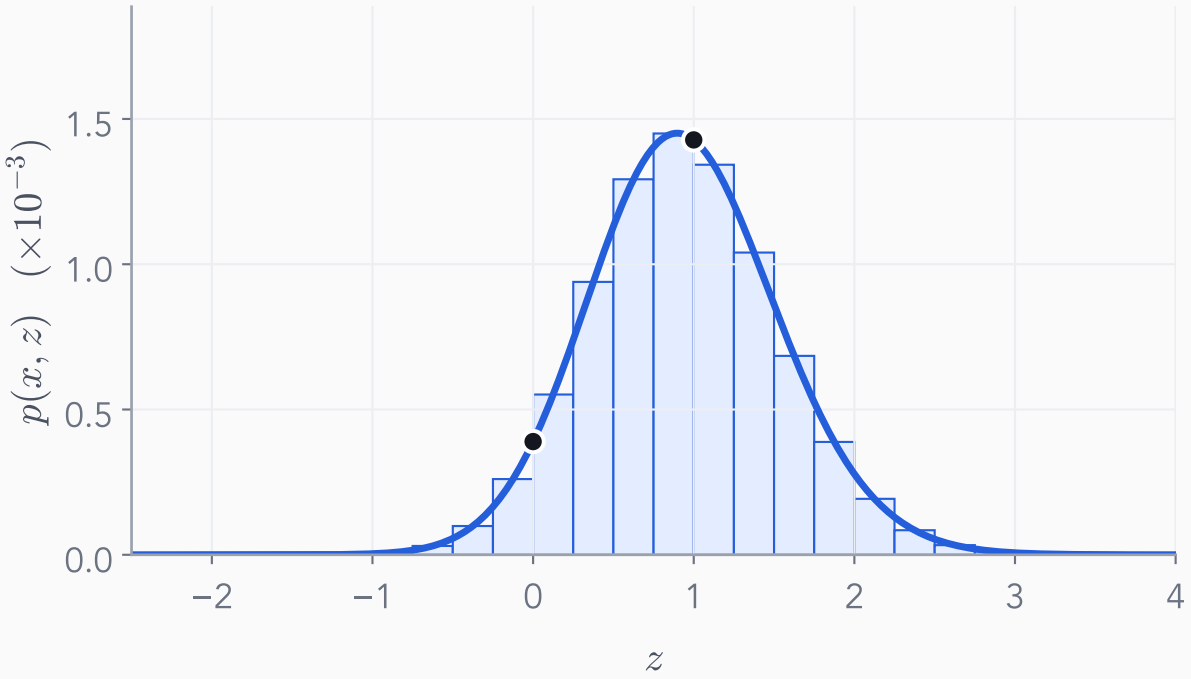
The two dots: $\log p(x, z) = -7.85$ at $z = 0$ and -6.552 at $z = 1$.

The shaded area is $p(x)$. To get it we would have to add up the curve **everywhere**.

We can evaluate the joint at any point. We cannot get its area. Variational inference will need only the first.

IN ONE DIMENSION, THERE IS A WAY OUT

Add up thin rectangles



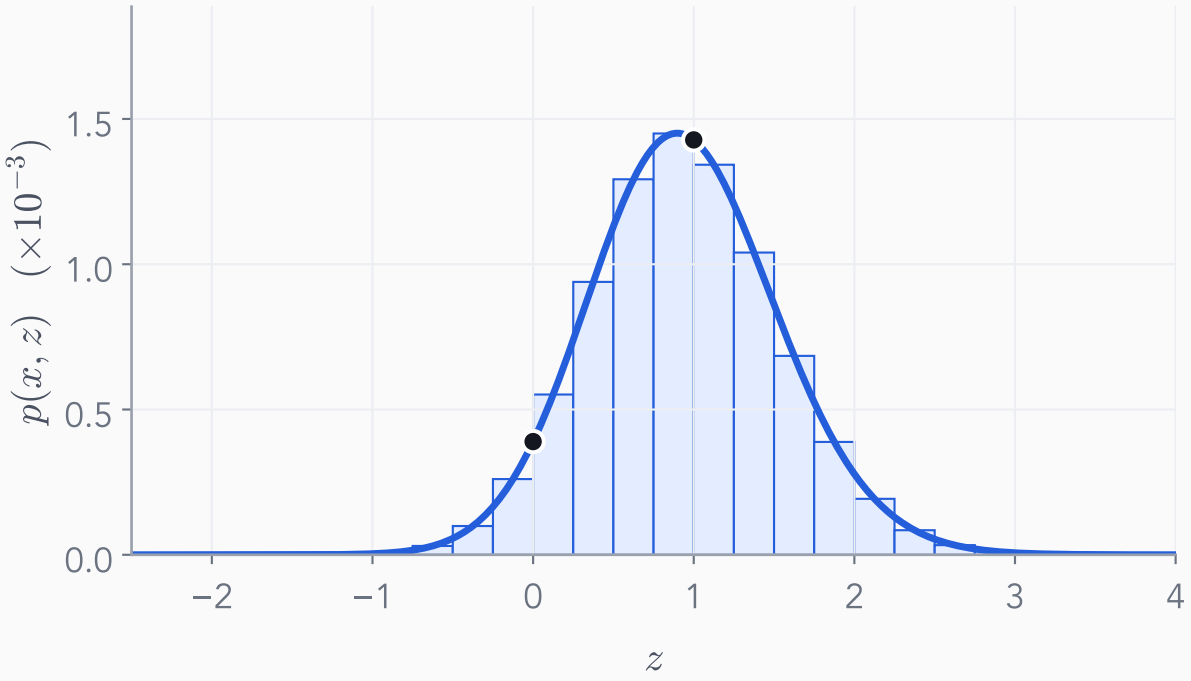
12,001 values of z from -6 to 6, spacing $\Delta z = 0.001$ (26 drawn here):

$$p(x) \approx \sum_k p(x, z_k)\, \Delta z = 0.00210$$

$\log p(x) = -6.164$ nats.

IN ONE DIMENSION, THERE IS A WAY OUT

Add up thin rectangles



12,001 values of z from -6 to 6, spacing $\Delta z = 0.001$ (26 drawn here):

$$p(x) \approx \sum_k p(x, z_k)\, \Delta z = 0.00210$$

$\log p(x) = -6.164$ nats.

Divide the curve by it, and we have the exact posterior: mean 0.94, standard deviation 0.586.

We keep this exact answer only to check every approximation in this lecture. No approximation will be allowed to use it.

IN MANY DIMENSIONS, THE WAY OUT CLOSES

The grid explodes with the number of hidden numbers

numbers in z	grid points (100 per axis)		example
1	100	this coin	

IN MANY DIMENSIONS, THE WAY OUT CLOSES

The grid explodes with the number of hidden numbers

numbers in z	grid points (100 per axis)	example
1	100	this coin
2	10^4	a 2-number VAE code
4	10^8	

IN MANY DIMENSIONS, THE WAY OUT CLOSES

The grid explodes with the number of hidden numbers

numbers in z	grid points (100 per axis)	example
1	100	this coin
2	10^4	a 2-number VAE code
4	10^8	
10	10^{20}	3 million years at 10^6 per second
32	10^{64}	a modest VAE code

IN MANY DIMENSIONS, THE WAY OUT CLOSES

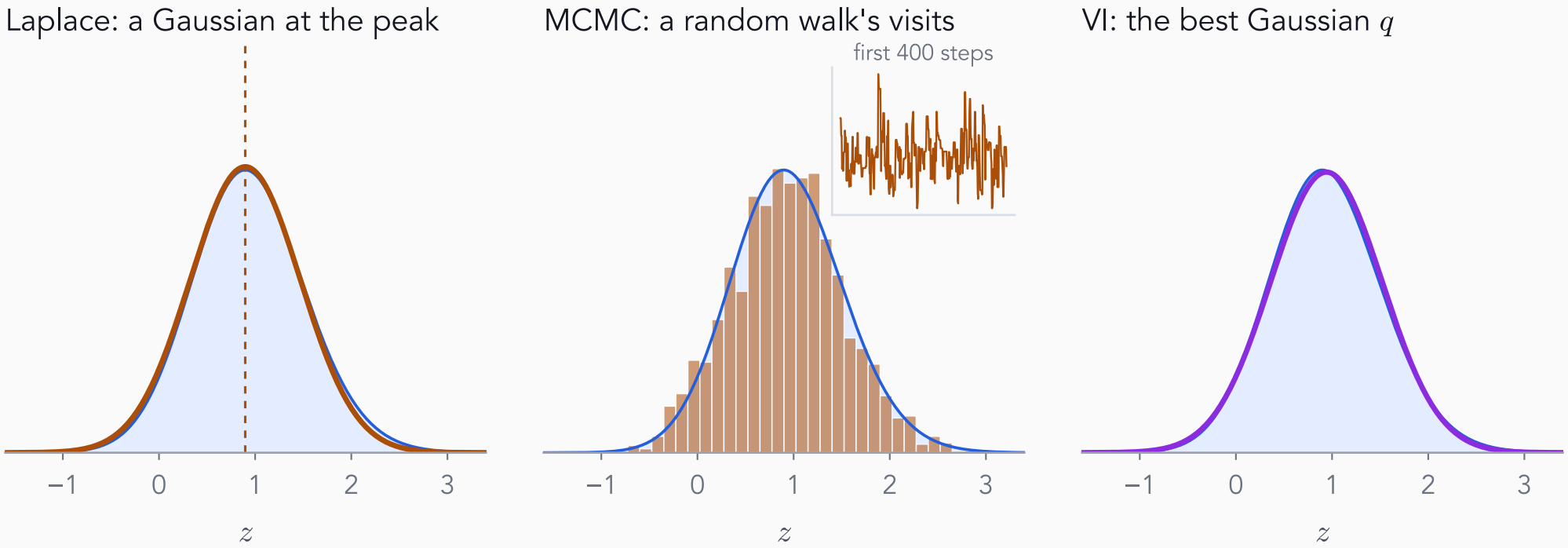
The grid explodes with the number of hidden numbers

numbers in z	grid points (100 per axis)	example
1	100	this coin
2	10^4	a 2-number VAE code
4	10^8	
10	10^{20}	3 million years at 10^6 per second
32	10^{64}	a modest VAE code

And a VAE needs this **for every image** (60,000 of them), **at every training step**, because the decoder keeps changing.

Intractable twice over: no formula, and far too many terms to add up.

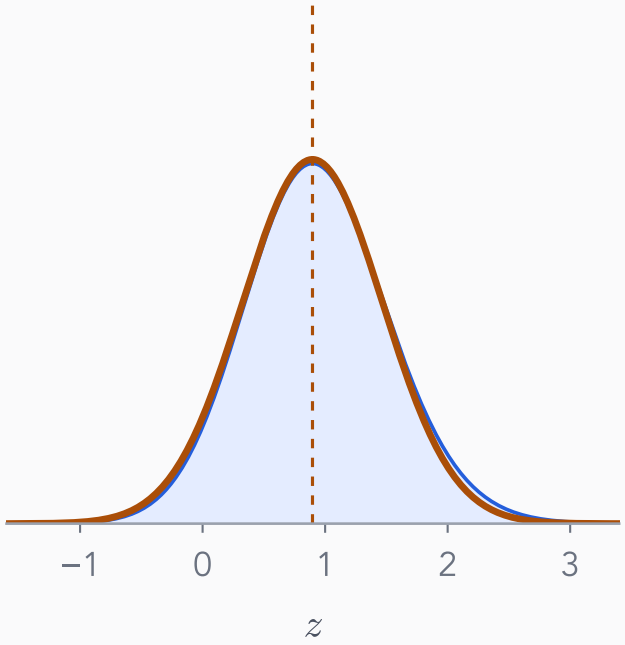
Three ways, on the same posterior



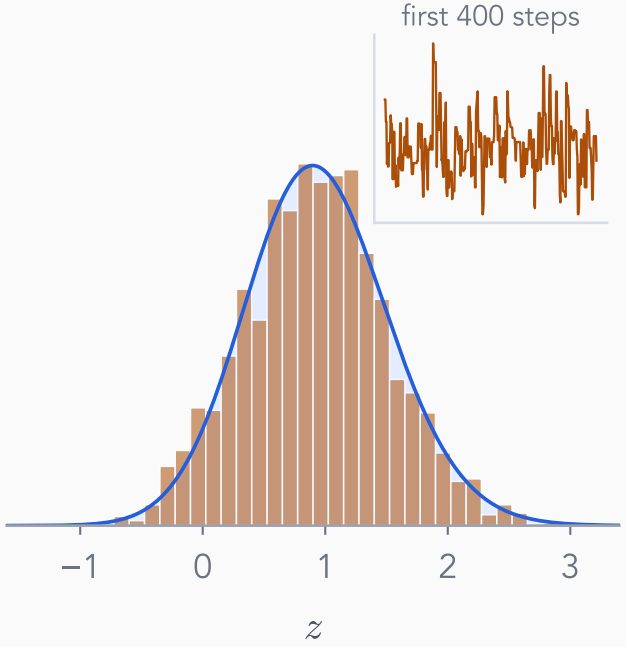
Laplace: a Gaussian fitted to the curvature at the peak, $z = 0.9$. Fast; sees only the peak.

Three ways, on the same posterior

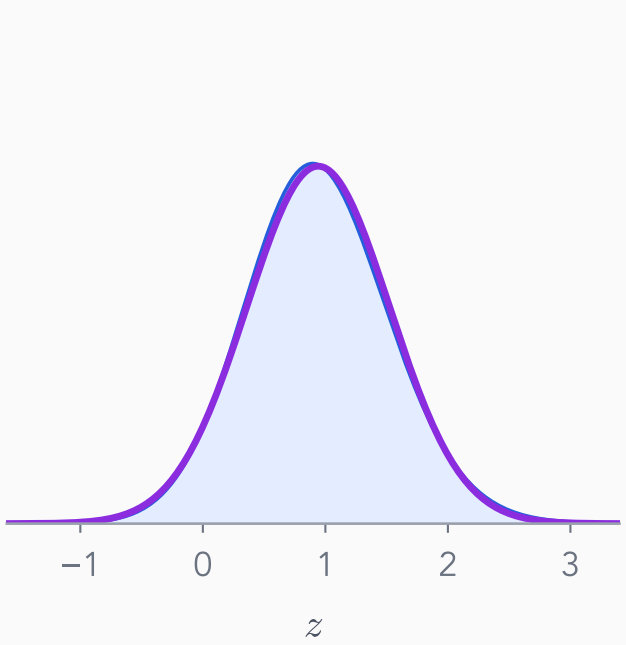
Laplace: a Gaussian at the peak



MCMC: a random walk's visits



VI: the best Gaussian q

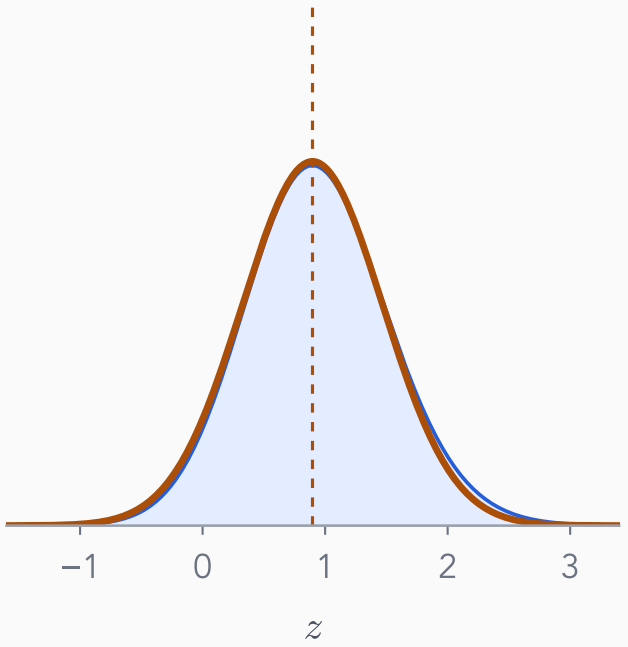


Laplace: a Gaussian fitted to the curvature at the peak, $z = 0.9$. Fast; sees only the peak.

MCMC: a random walk that visits z in proportion to the posterior. Exact in the long run; slow.

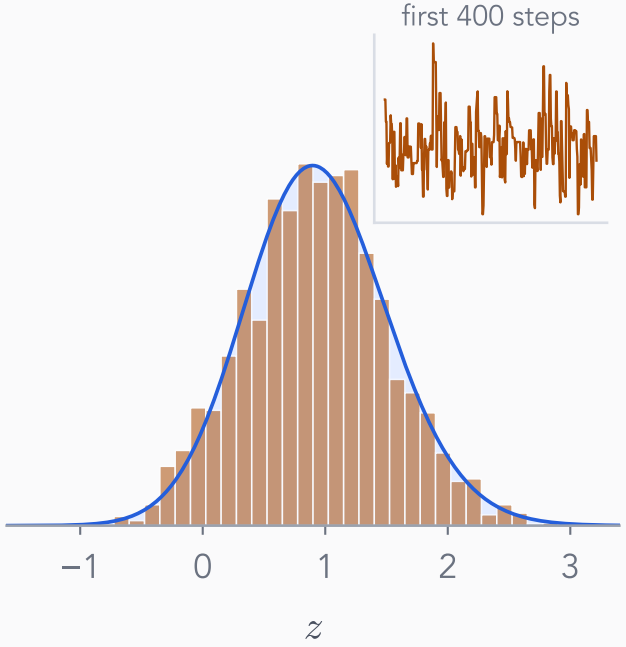
Three ways, on the same posterior

Laplace: a Gaussian at the peak



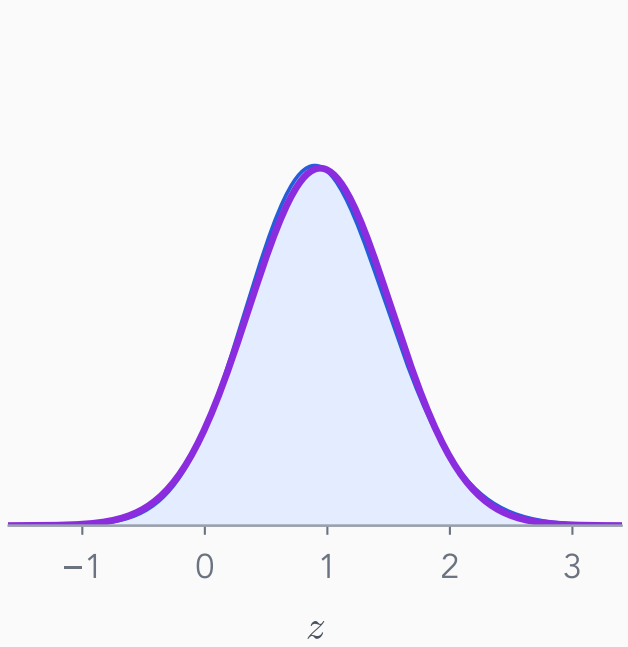
Laplace: a Gaussian fitted to the curvature at the peak, $z = 0.9$. Fast; sees only the peak.

MCMC: a random walk's visits



MCMC: a random walk that visits z in proportion to the posterior. Exact in the long run; slow.

VI: the best Gaussian q



VI: the closest member of a simple family, $\mathcal{N}(0.94,\ 0.58^2)$. Inference becomes optimisation.

> **The rest of today is only the third. It needs two tools first: averages from samples, and a number for "close".**

PART 3 OF 8

Averages, and estimating them from samples

Variational inference's loss is an **average over a distribution**. We will almost never compute it exactly; we will estimate it from a handful of random draws. Here is why that works, and how noisy it is.

✓ Bayes: prior, likelihood, posterior

✓ Why the posterior is hard

3 **Averages from samples**

4 KL divergence

5 Variational inference and the ELBO

6 Climbing the ELBO: gradients through samples

7 Beyond one number

8 Many data points: towards the VAE

NOTATION FOR THE REST OF THE LECTURE

$\mathbb{E}$ means "average over"

$$\mathbb{E}_{\theta \sim p(\theta|x)}[\theta] = 0.0002 \times 0.2 + 0.127 \times 0.5 + 0.873 \times 0.8 = 0.762$$

the three-coin posterior (Part 1): the chance the **next** toss is heads

$\mathbb{E}$ means "average over"

$$\mathbb{E}_{\theta \sim p(\theta|x)}[\theta] = 0.0002 \times 0.2 + 0.127 \times 0.5 + 0.873 \times 0.8 = 0.762$$

the three-coin posterior (Part 1): the chance the **next** toss is heads

$$\mathbb{E}_{\theta \sim \text{Beta}(10,4)}[\theta] = \int_0^1 \theta \, p(\theta \mid x) \, d\theta = \tfrac{10}{14} = 0.714$$

a continuous variable: the sum becomes an integral

NOTATION FOR THE REST OF THE LECTURE

$\mathbb{E}$ means "average over"

$$\mathbb{E}_{\theta \sim p(\theta|x)}[\theta] = 0.0002 \times 0.2 + 0.127 \times 0.5 + 0.873 \times 0.8 = 0.762$$

the three-coin posterior (Part 1): the chance the **next** toss is heads

$$\mathbb{E}_{\theta \sim \text{Beta}(10,4)}[\theta] = \int_0^1 \theta \, p(\theta \mid x) \, d\theta = \tfrac{10}{14} = 0.714$$

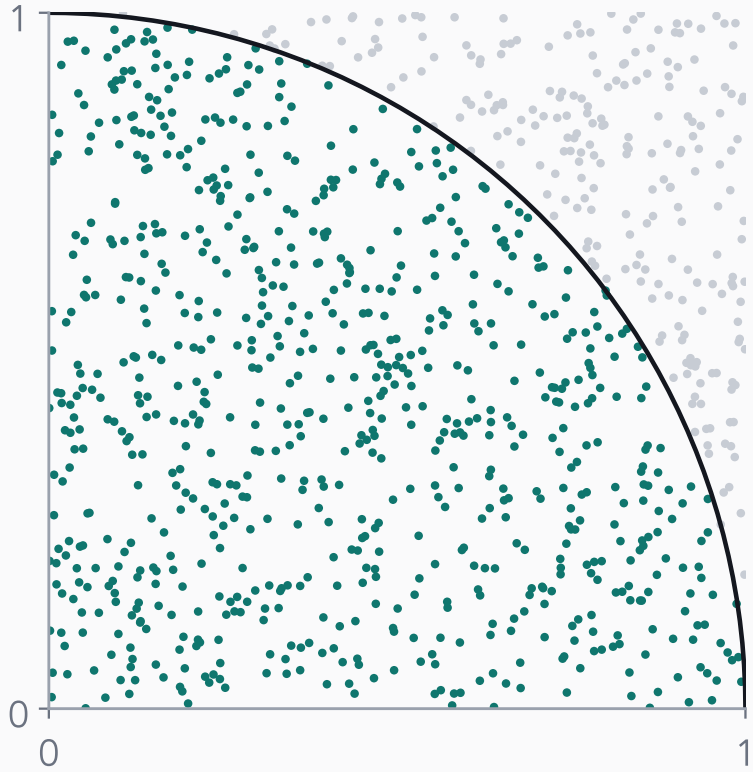
a continuous variable: the sum becomes an integral

$$\mathbb{E}_{z \sim q}\big[f(z)\big] = \int q(z) \, f(z) \, dz$$

in general: weight each value $f(z)$ by how likely z is under q

> **The subscript says which distribution we average under. It will matter: KL and the ELBO are both averages, and the choice of subscript is the key decision in VI.**

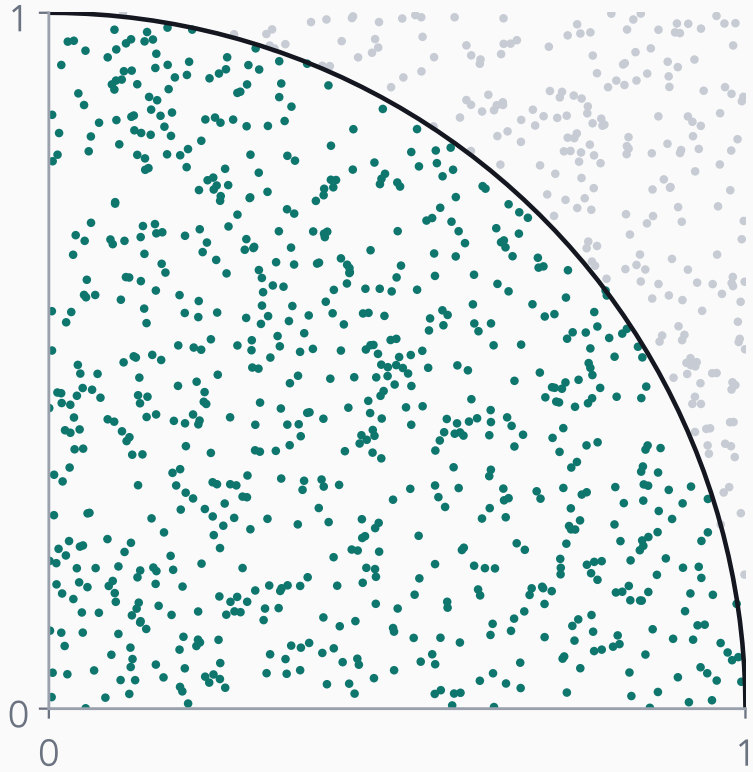
Throw darts at a square to estimate π



Throw darts uniformly at the unit square. The quarter circle covers $\pi/4$ of its area, so

$$\pi = 4\,\mathbb{E}\big[1\{\text{dart inside}\}\big] \approx 4 \times \frac{\text{darts inside}}{\text{darts thrown}}$$

Throw darts at a square to estimate π

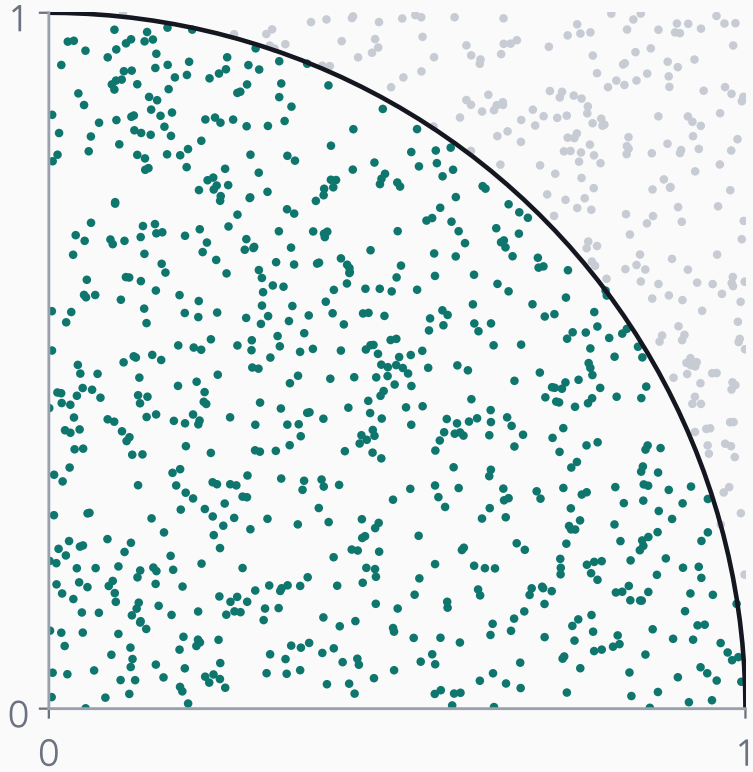


Throw darts uniformly at the unit square. The quarter circle covers $\pi/4$ of its area, so

$$\pi = 4\,\mathbb{E}\big[1\{\text{dart inside}\}\big] \approx 4 \times \frac{\text{darts inside}}{\text{darts thrown}}$$

Here: 771 of 1,000 inside, so $\pi \approx 3.084$.

Throw darts at a square to estimate π



Throw darts uniformly at the unit square. The quarter circle covers $\pi/4$ of its area, so

$$\pi = 4\,\mathbb{E}\big[1\{\text{dart inside}\}\big] \approx 4 \times \frac{\text{darts inside}}{\text{darts thrown}}$$

Here: 771 of 1,000 inside, so $\pi \approx 3.084$.

darts	10	100	1,000	10,000	100,000
spread over 6 runs	0.643	0.14	0.052	0.008	0.005

> **An area, which is an integral, became an average of random yes/no answers. That is the whole idea of Monte Carlo.**

Replace the integral by an average of draws

$$\mathbb{E}_{z\sim q}\big[f(z)\big] \;\approx\; \frac{1}{S}\sum_{s=1}^{S} f(z_s), \qquad z_s \sim q$$

Replace the integral by an average of draws

$$\mathbb{E}_{z\sim q}\big[f(z)\big] \;\approx\; \frac{1}{S}\sum_{s=1}^{S} f(z_s), \qquad z_s \sim q$$

Example: $q = \mathcal{N}(1,\ 0.5^2)$ and $f(z) = z^2$. Exact answer: $\mathbb{E}[z^2] = \mu^2 + \sigma^2 = 1 + 0.25 = 1.25$.

Replace the integral by an average of draws

$$\mathbb{E}_{z\sim q}\big[f(z)\big] \;\approx\; \frac{1}{S}\sum_{s=1}^{S} f(z_s), \qquad z_s \sim q$$

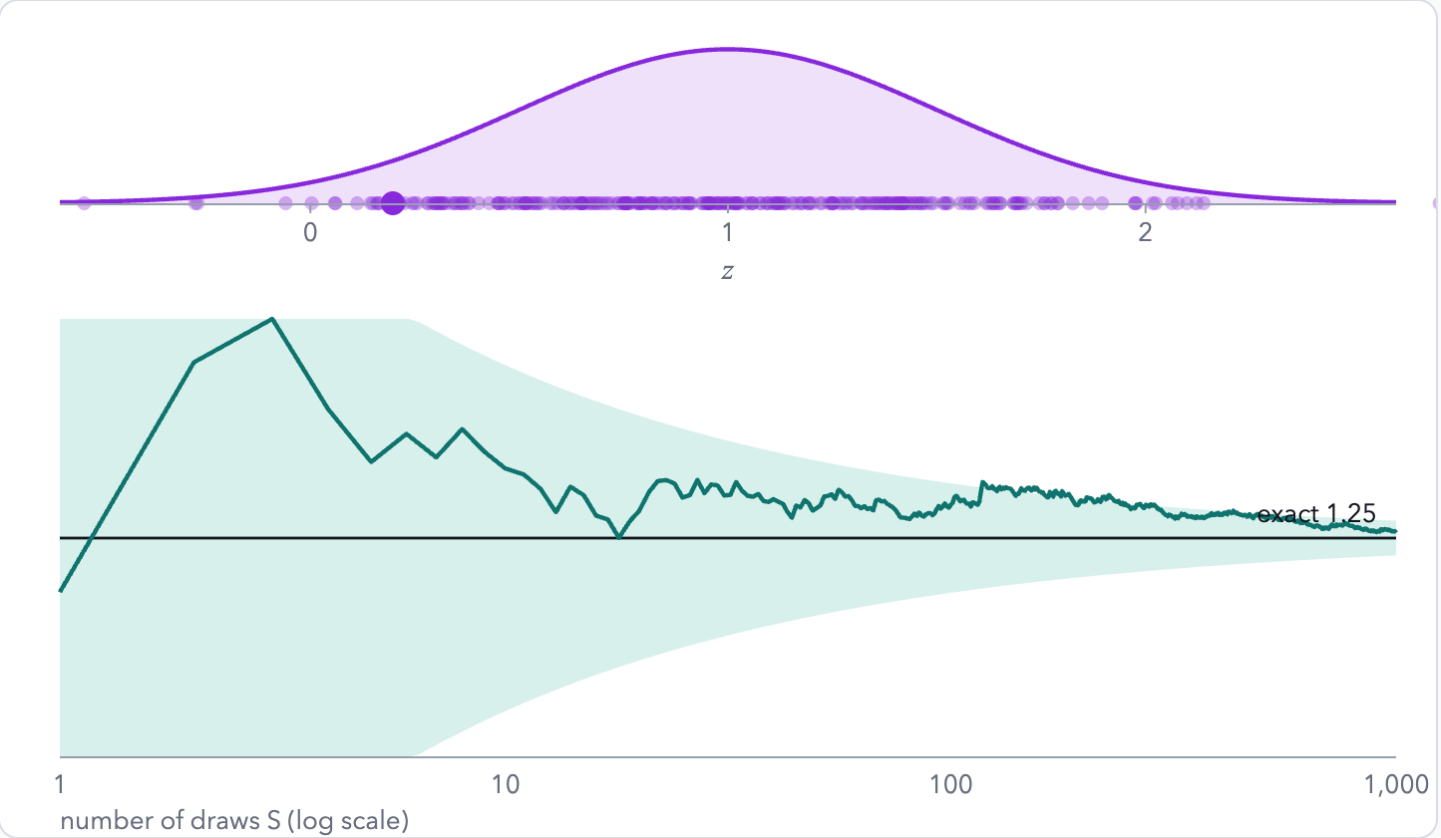
Example: $q = \mathcal{N}(1,\ 0.5^2)$ and $f(z) = z^2$. Exact answer: $\mathbb{E}[z^2] = \mu^2 + \sigma^2 = 1 + 0.25 = 1.25$.

draw s	1	2	3	4	5	average
z_s	1.02	1.68	1.61	0.74	0.85	
$f(z_s) = z_s^2$	1.040	2.822	2.592	0.548	0.722	**1.545**

Five draws: 1.545 instead of 1.25. Not bad, not great. More draws?

TRY IT

Draw more, get closer



draws so far: **1,000**

latest z: **0.20**

estimate 1.270

exact: 1.25

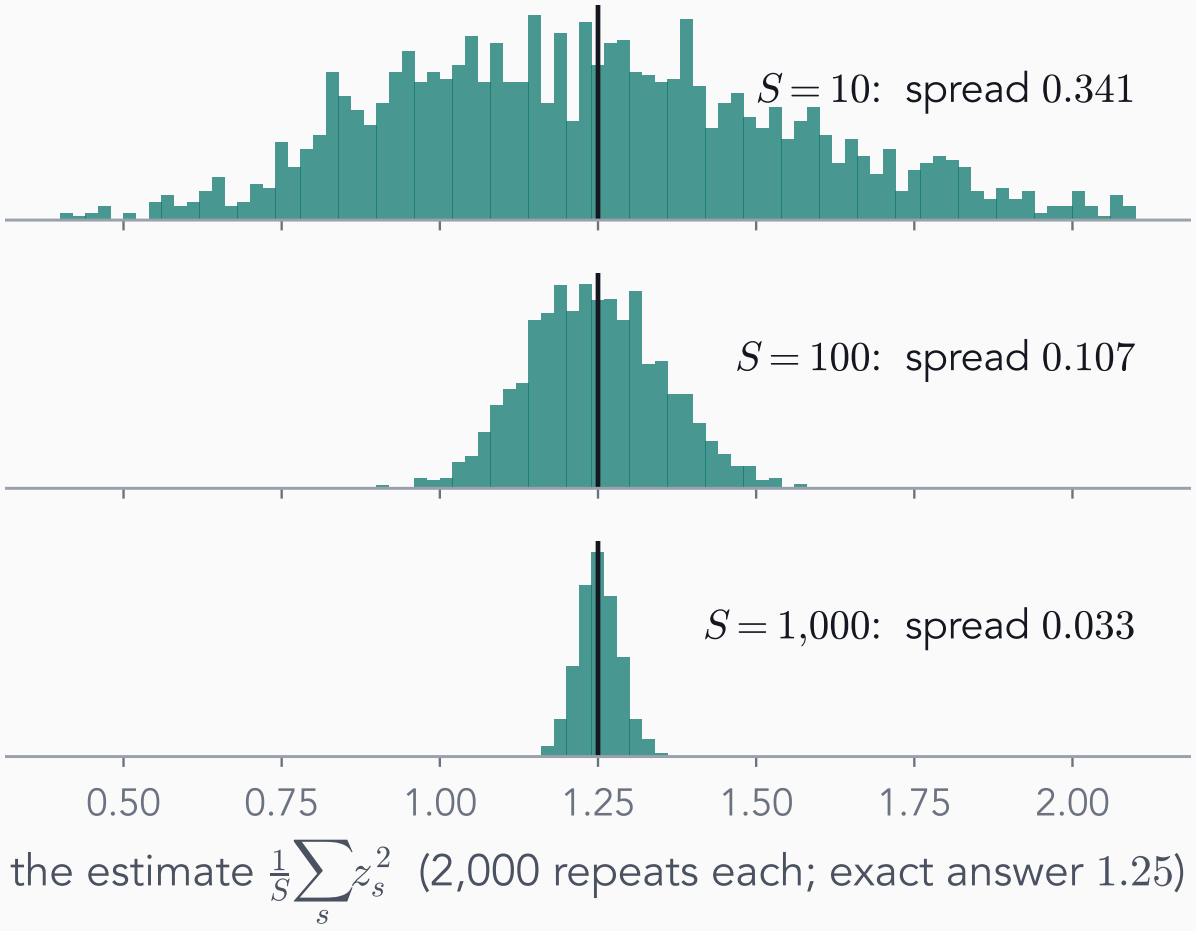
q and its draws — running average

$1.25 \pm 2 \times 1.061/\sqrt{S}$: 95% of runs

More draws: closer to 1.25. The teal band is where 95% of runs end up after S draws; it narrows like $1/\sqrt{S}$.

HOW NOISY IS THE ESTIMATE?

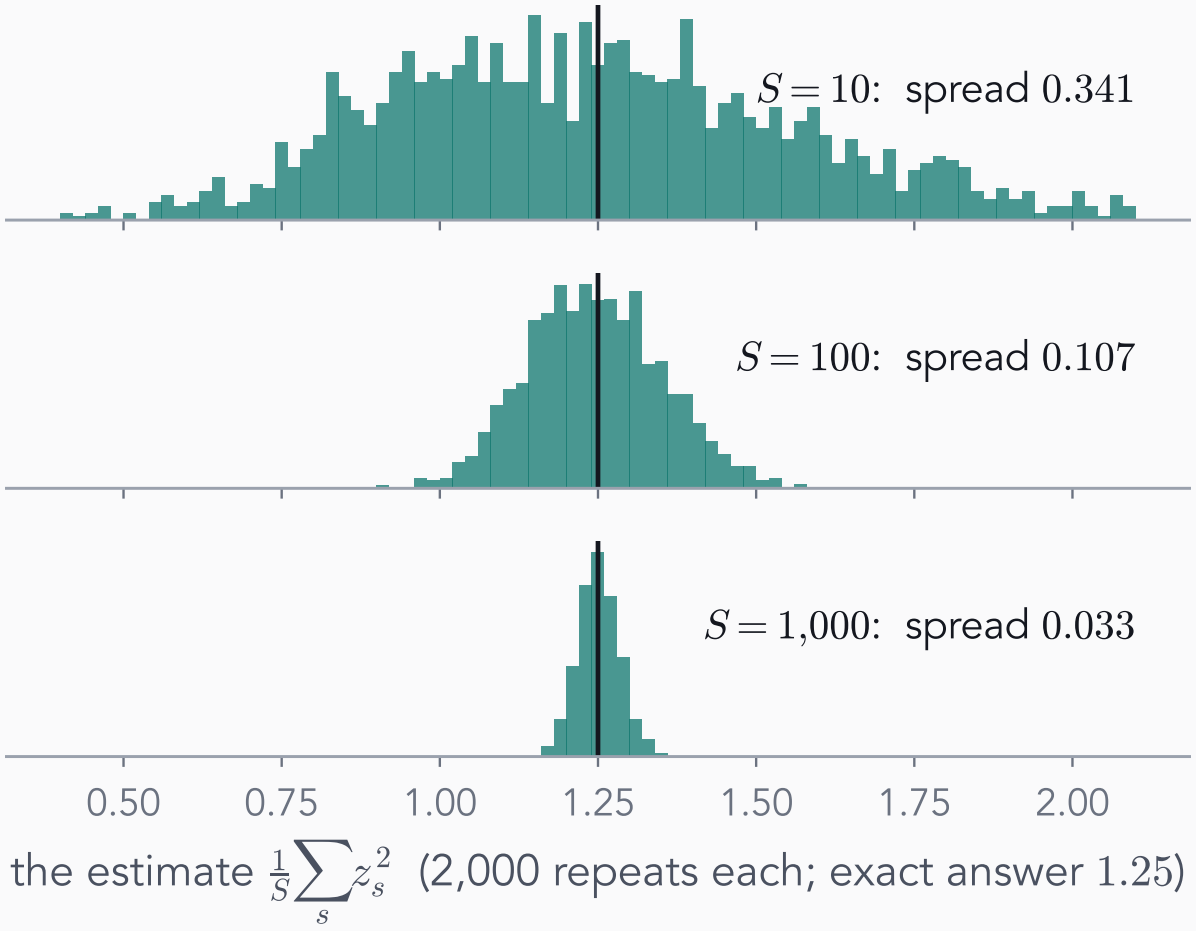
The error shrinks like $1/\sqrt{S}$



One draw of z^2 has a spread (standard deviation) of 1.061.

HOW NOISY IS THE ESTIMATE?

The error shrinks like $1/\sqrt{S}$

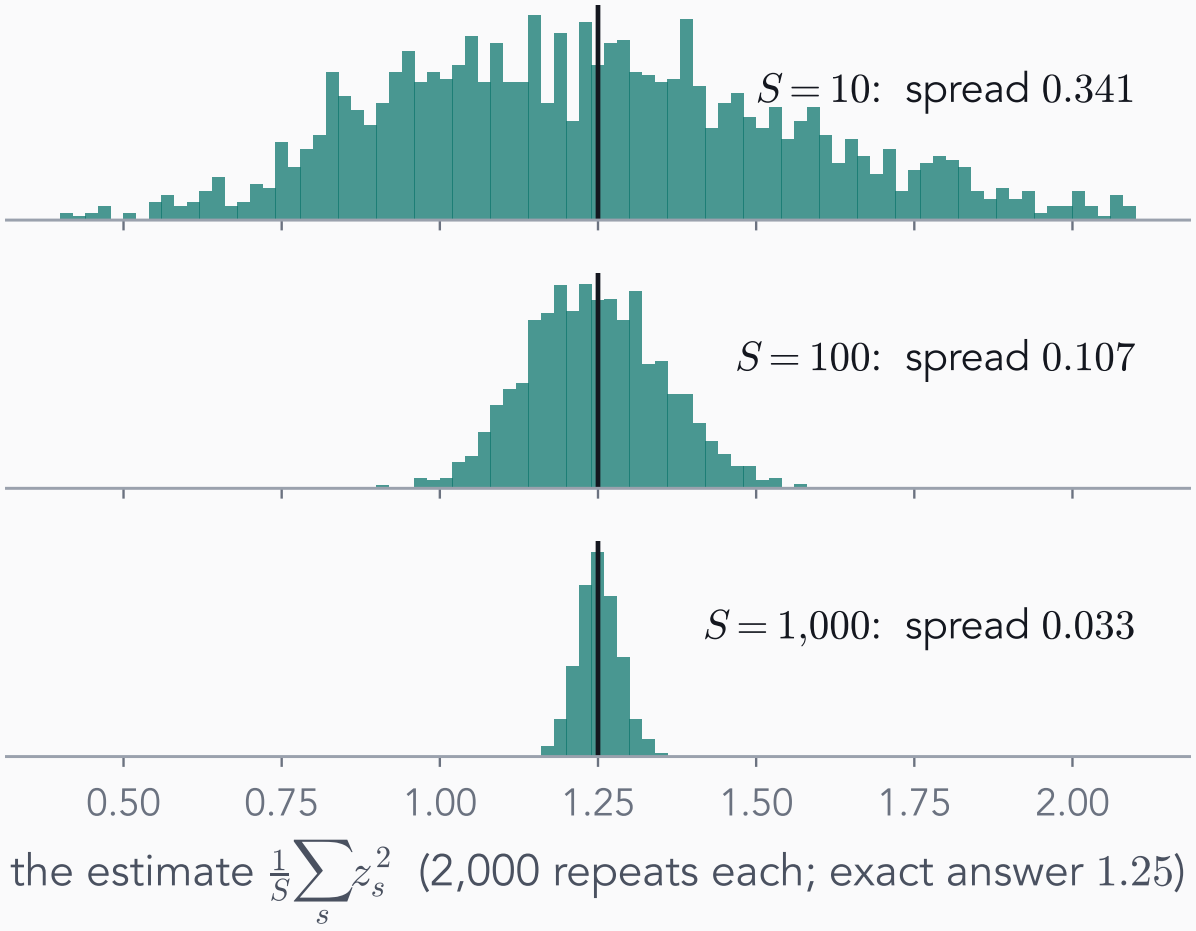


One draw of z^2 has a spread (standard deviation) of 1.061.

An average of S draws has spread $1.061/\sqrt{S}$: 0.335, 0.106, 0.034 for S = 10, 100, 1,000.

HOW NOISY IS THE ESTIMATE?

The error shrinks like $1/\sqrt{S}$



One draw of z^2 has a spread (standard deviation) of 1.061.

An average of S draws has spread $1.061/\sqrt{S}$: 0.335, 0.106, 0.034 for S = 10, 100, 1,000.

Measured over 2,000 repeats: 0.341, 0.107, 0.033.

Centred on the right answer (unbiased), but noisy: 100 times more draws buy 10 times less noise.

IN CODE

Three lines, in any number of dimensions

```
mu, sigma = 1.0, 0.5
z = mu + sigma * torch.randn(1000)   # 1,000 draws from q
estimate = (z**2).mean()             # about 1.25
```

Draw from q: standard noise, stretched by σ and shifted by μ. Remember this line: it returns in Part 6.

IN CODE

Three lines, in any number of dimensions

```
mu, sigma = 1.0, 0.5
z = mu + sigma * torch.randn(1000)   # 1,000 draws from q
estimate = (z**2).mean()             # about 1.25
```

Average f: evaluate f at each draw and take the mean. No integral anywhere.

IN CODE

Three lines, in any number of dimensions

```
mu, sigma = 1.0, 0.5
z = mu + sigma * torch.randn(1000)   # 1,000 draws from q
estimate = (z**2).mean()             # about 1.25
```

The catch: we need draws from the distribution we average under. For a q we chose, easy. For the posterior, that is the very problem we started with.

So variational inference will arrange for every average to be under q, never under the posterior.

PART 4 OF 8

KL divergence: how far apart are two distributions?

Variational inference wants a simple q **close** to the posterior. "Close" needs a number. Build it slowly, from a betting game over two outcomes; then move to bell curves.

✓ Bayes: prior, likelihood, posterior

✓ Why the posterior is hard

✓ Averages from samples

4 **KL divergence**

5 Variational inference and the ELBO

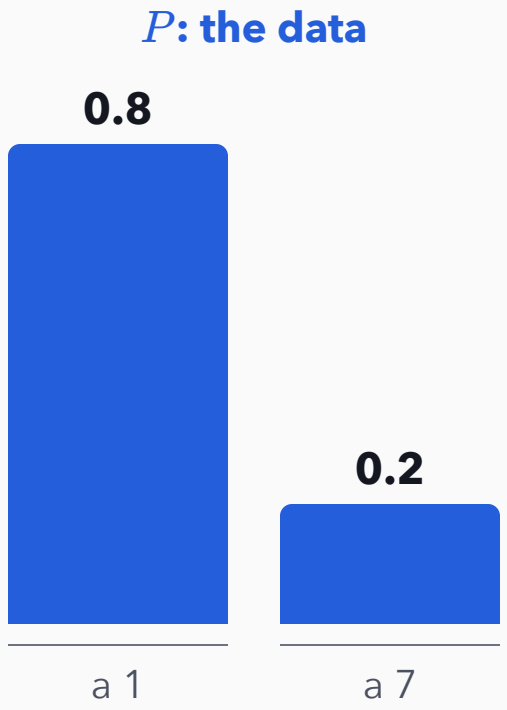
6 Climbing the ELBO: gradients through samples

7 Beyond one number

8 Many data points: towards the VAE

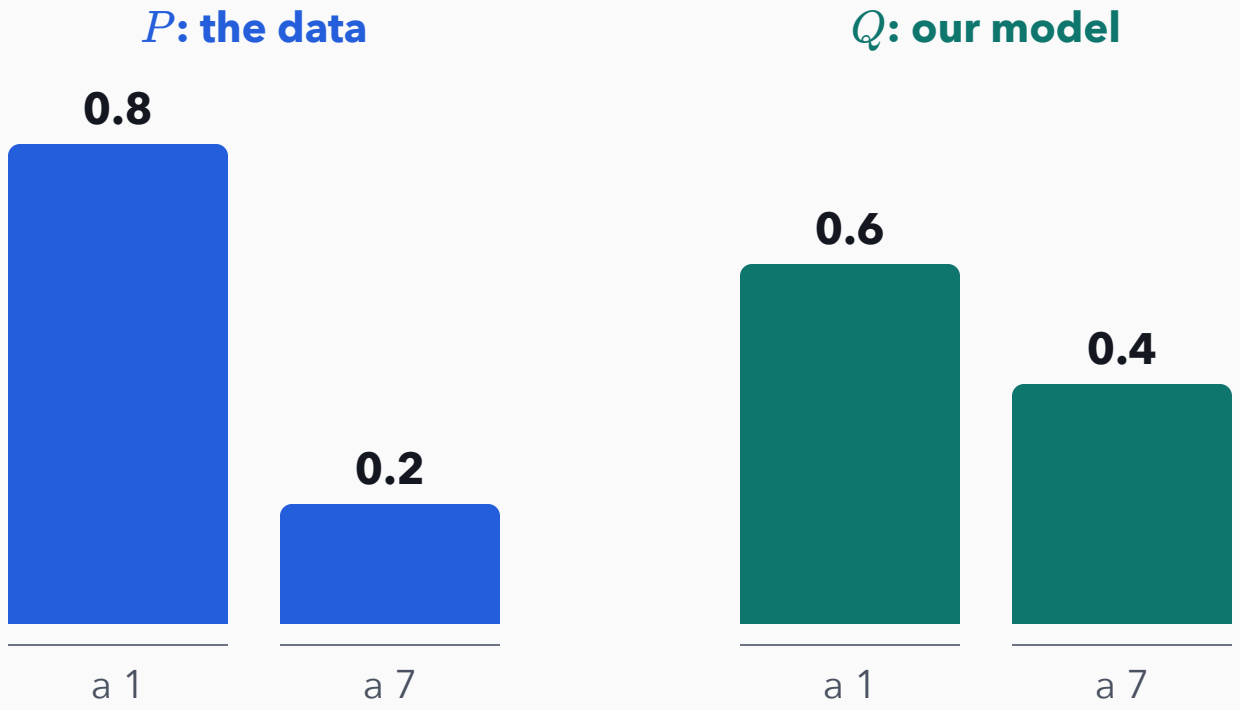
TWO DISTRIBUTIONS OVER THE SAME TWO OUTCOMES

The data says 80/20. The model says 60/40.



TWO DISTRIBUTIONS OVER THE SAME TWO OUTCOMES

The data says 80/20. The model says 60/40.



A dataset of handwritten 1s and 7s, 80% 1s. The model believes 60/40. How wrong is it, as one number?

A BETTING GAME

Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

P **(the truth)**	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8

A BETTING GAME

Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

P (the truth)										
	1	**1**	**1**	**7**	**1**	**1**	**1**	**1**	**7**	**1**
	0.8	0.8	0.8	0.2	0.8	0.8	0.8	0.8	0.2	0.8

$$0.8^8 \times 0.2^2 = 0.0067$$

A BETTING GAME

Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

P **(the truth)**	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8
Q **(our model)**	**1** 0.6	**1** 0.6	**1** 0.6	**7** 0.4	**1** 0.6	**1** 0.6	**1** 0.6	**1** 0.6	**7** 0.4	**1** 0.6

$$0.8^8 \times 0.2^2 = 0.0067$$

A BETTING GAME

Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

P **(the truth)**	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8
Q **(our model)**	**1** 0.6	**1** 0.6	**1** 0.6	**7** 0.4	**1** 0.6	**1** 0.6	**1** 0.6	**1** 0.6	**7** 0.4	**1** 0.6

$$0.8^8 \times 0.2^2 = 0.0067$$

$$0.6^8 \times 0.4^2 = 0.0027$$

A BETTING GAME

Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

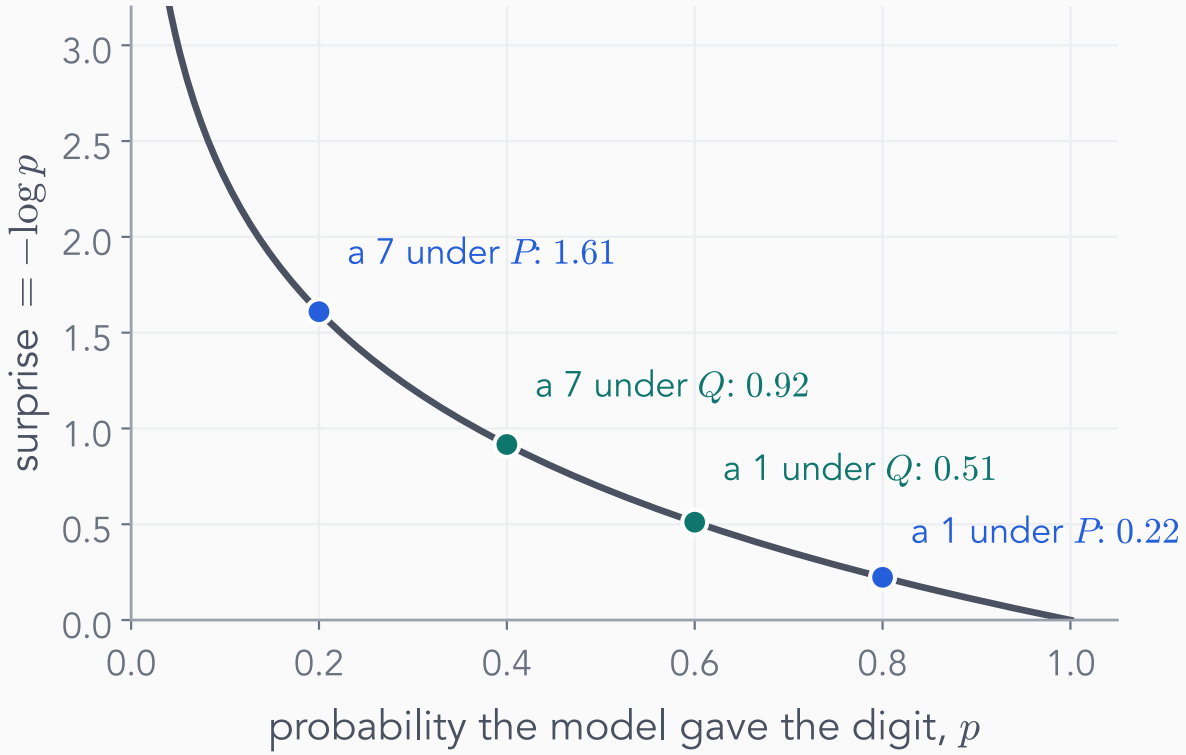
P **(the truth)**	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8	**1** 0.8	**1** 0.8	**1** 0.8	**7** 0.2	**1** 0.8
Q **(our model)**	**1** 0.6	**1** 0.6	**1** 0.6	**7** 0.4	**1** 0.6	**1** 0.6	**1** 0.6	**1** 0.6	**7** 0.4	**1** 0.6

$$0.8^8 \times 0.2^2 = 0.0067$$

$$0.6^8 \times 0.4^2 = 0.0027$$

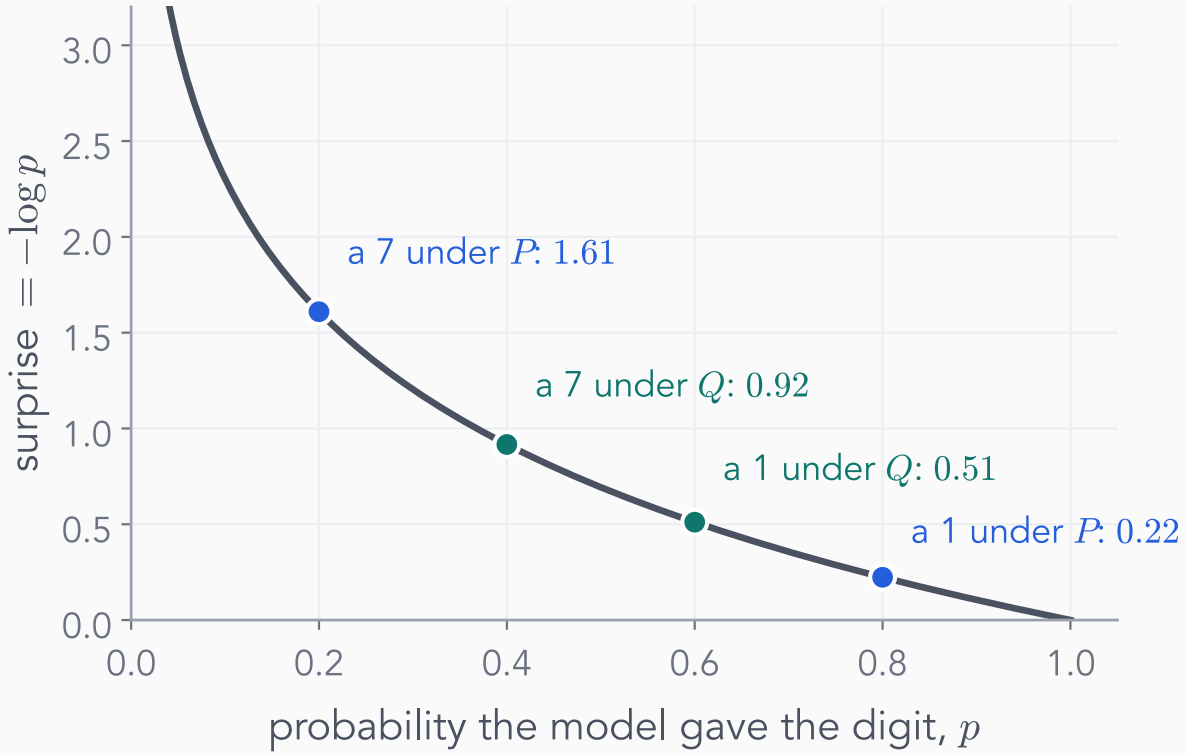
The truth wins, by a factor of $0.0067/0.0027 = 2.5$. KL will measure this gap, per digit.

Surprise $= -\log p$, and why the log



Which is more surprising: snow in Kashmir in January, or snow in Gandhinagar? What should "surprise" do?

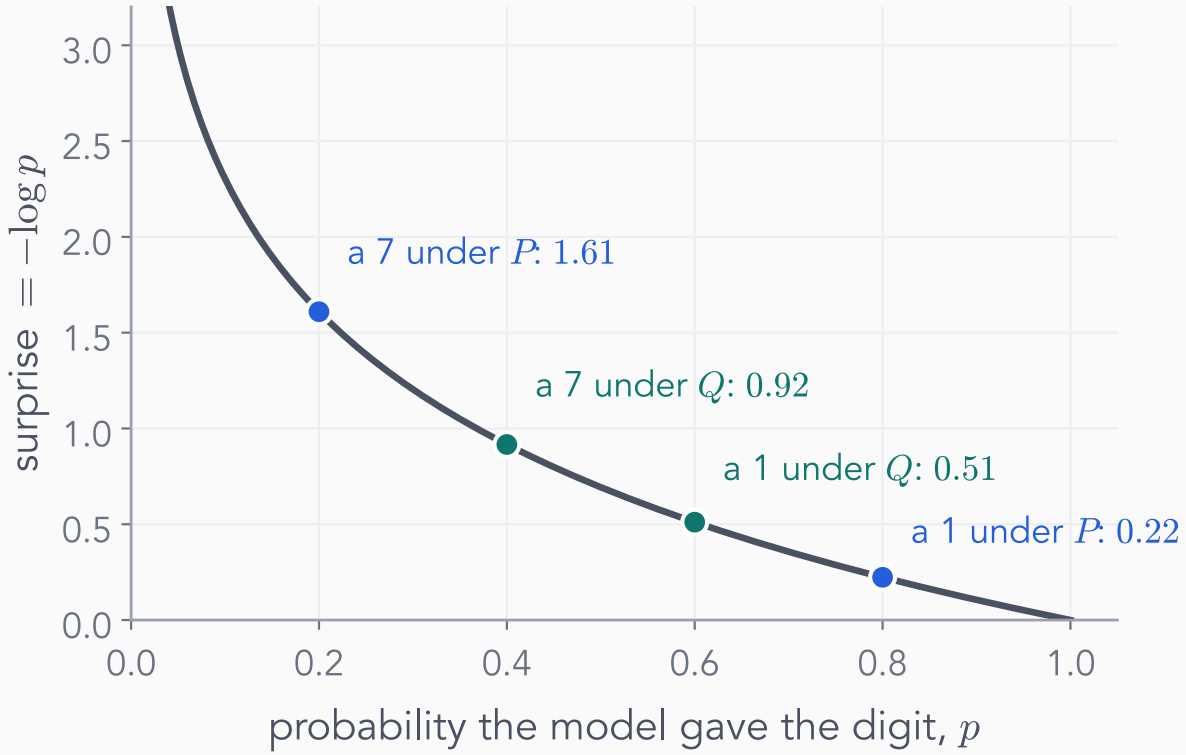
Surprise $= -\log p$, and why the log



Which is more surprising: snow in Kashmir in January, or snow in Gandhinagar? What should "surprise" do?

$1 \cdot$ a certain event: no surprise. $-\log 1 = 0$.

Surprise $= -\log p$, and why the log

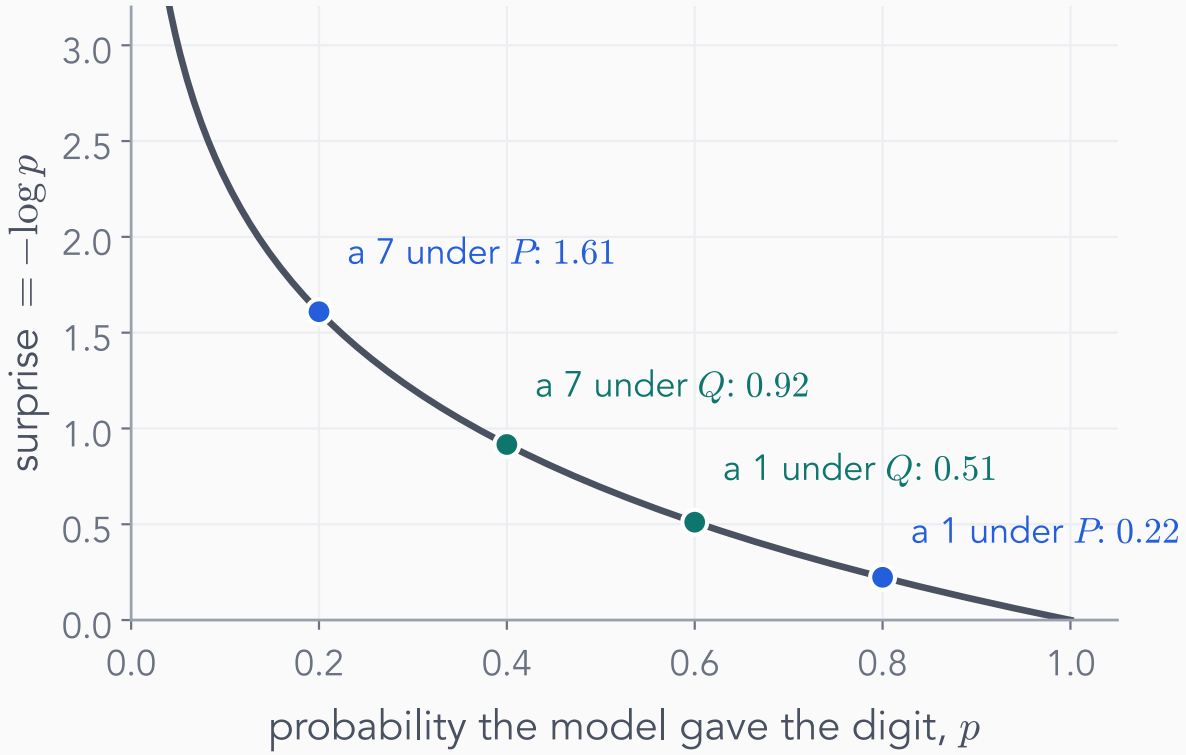


Which is more surprising: snow in Kashmir in January, or snow in Gandhinagar? What should "surprise" do?

1 · a certain event: no surprise. $-\log 1 = 0$.

2 · rarer means more surprising. $-\log p$ grows as p shrinks.

Surprise $= -\log p$, and why the log



Which is more surprising: snow in Kashmir in January, or snow in Gandhinagar? What should "surprise" do?

1 · a certain event: no surprise. $-\log 1 = 0$.

2 · rarer means more surprising. $-\log p$ grows as p shrinks.

3 · two independent digits: probabilities **multiply**, surprises should **add**. Only the log turns products into sums.

> **So the betting-game product becomes a sum of surprises, one per digit. Natural log, so the unit is the nat.**

OUR TWO DIGITS, SCORED BOTH WAYS

The surprise table

outcome	P	Q	surprise if you believe P	surprise if you believe Q
a 1	0.8	0.6	$-\log 0.8 = 0.223$	
a 7	0.2	0.4	$-\log 0.2 = 1.609$	

OUR TWO DIGITS, SCORED BOTH WAYS

The surprise table

outcome	P	Q	surprise if you believe P	surprise if you believe Q
a 1	0.8	0.6	$-\log 0.8 = 0.223$	$-\log 0.6 = 0.511$
a 7	0.2	0.4	$-\log 0.2 = 1.609$	$-\log 0.4 = 0.916$

OUR TWO DIGITS, SCORED BOTH WAYS

The surprise table

outcome	P	Q	surprise if you believe P	surprise if you believe Q
a 1	0.8	0.6	$-\log 0.8 = 0.223$	$-\log 0.6 = 0.511$
a 7	0.2	0.4	$-\log 0.2 = 1.609$	$-\log 0.4 = 0.916$

Each row is one kind of digit. The next two slides average each column, weighted by how often the digit really appears.

Believing Q, a 1 surprises us more than it should (0.511 vs 0.223), a 7 less (0.916 vs 1.609).

AVERAGE SURPRISE, WHEN YOU BELIEVE THE TRUTH

Entropy: $H(P) = 0.500$ nats per digit

$$H(P) = \sum_i P_i \left(-\log P_i\right) = \underbrace{0.8}_{\text{how often a 1}} \times 0.223 + \underbrace{0.2}_{\text{how often a 7}} \times 1.609 = 0.500$$

AVERAGE SURPRISE, WHEN YOU BELIEVE THE TRUTH

Entropy: $H(P) = 0.500$ nats per digit

$$H(P) = \sum_i P_i \left(-\log P_i\right) = \underbrace{0.8}_{\text{how often a 1}} \times 0.223 + \underbrace{0.2}_{\text{how often a 7}} \times 1.609 = 0.500$$

The weights are how often each digit really appears: eight in ten, two in ten. In Part 3's language, an average under P: $H(P) = \mathbb{E}_{x \sim P}[-\log P(x)]$.

Even the perfect model is surprised by 0.500 nats per digit on average: the data are random.

SAME DATA, A DIFFERENT SCORER

Cross-entropy: $H(P, Q) = 0.592$ nats per digit

$$H(P, Q) = \sum_i P_i \left(-\log Q_i\right) = 0.8 \times 0.511 + 0.2 \times 0.916 = 0.592$$

SAME DATA, A DIFFERENT SCORER

Cross-entropy: $H(P, Q) = 0.592$ nats per digit

$$H(P, Q) = \sum_i P_i \left(-\log Q_i\right) = 0.8 \times 0.511 + 0.2 \times 0.916 = 0.592$$

Still weighted by P

the digits still come from the real data: eight 1s, two 7s.

Scored by Q

each digit's surprise now uses the model's probability.

Believing the wrong model costs more surprise on the same data: 0.592 instead of 0.500.

THE EXTRA SURPRISE

KL divergence = cross-entropy − entropy

$$H(P) = 0.500$$

unavoidable: the data are random

together: $H(P, Q) = 0.592$ (more precisely $0.5004 + 0.0915 = 0.5919$)

THE EXTRA SURPRISE

KL divergence = cross-entropy − entropy

$H(P) = 0.500$
unavoidable: the data are random

0.092
KL

together: $H(P, Q) = 0.592$ (more precisely $0.5004 + 0.0915 = 0.5919$)

THE EXTRA SURPRISE

KL divergence = cross-entropy − entropy

$H(P) = 0.500$
unavoidable: the data are random

0.092
KL

together: $H(P, Q) = 0.592$ (more precisely $0.5004 + 0.0915 = 0.5919$)

$$\mathrm{KL}(P \,\|\, Q) = H(P, Q) - H(P) = \sum_i P_i \log \frac{P_i}{Q_i} = 0.092 \text{ nats}$$

KL is the part of the surprise that comes only from using the wrong model. And it is the betting game's gap: $\frac{1}{10} \log 2.5 = 0.092$.

ANOTHER WAY TO READ THE SAME NUMBERS (FROM THE PML COURSE)

In bits: KL is the extra length of the wrong code

symbol	true p	best code for p	length $= -\log_2 p$
A	0.5	`0`	1
B	0.25	`10`	2
C	0.125	`110`	3
D	0.125	`111`	3
average under p			$H(p) = 1.75$

ANOTHER WAY TO READ THE SAME NUMBERS (FROM THE PML COURSE)

In bits: KL is the extra length of the wrong code

symbol	true p	best code for p	length $= -\log_2 p$	code built for q = uniform	length
A	0.5	`0`	1	`00`	2
B	0.25	`10`	2	`01`	2
C	0.125	`110`	3	`10`	2
D	0.125	`111`	3	`11`	2
average under p			$H(p) = 1.75$		$H(p, q) = 2.0$

ANOTHER WAY TO READ THE SAME NUMBERS (FROM THE PML COURSE)

In bits: KL is the extra length of the wrong code

symbol	true p	best code for p	length $= -\log_2 p$	code built for q = uniform	length
A	0.5	`0`	1	`00`	2
B	0.25	`10`	2	`01`	2
C	0.125	`110`	3	`10`	2
D	0.125	`111`	3	`11`	2
average under p			$H(p) = 1.75$		$H(p, q) = 2.0$

Coding p's data with q's code costs $2.0 - 1.75 = 0.25$ extra bits per symbol: $\mathrm{KL}(p \,\|\, q)$ in bits. Surprise is code length.

PART 3'S TOOL, ALREADY USEFUL

KL is an average, so samples can estimate it

$$\mathrm{KL}(P \,\|\, Q) = \mathbb{E}_{x \sim P}\left[\log \frac{P(x)}{Q(x)}\right] \approx \frac{1}{S}\sum_{s=1}^{S} \log \frac{P(x_s)}{Q(x_s)}, \qquad x_s \sim P$$

PART 3'S TOOL, ALREADY USEFUL

KL is an average, so samples can estimate it

$$\mathrm{KL}(P \,\|\, Q) = \mathbb{E}_{x\sim P}\left[\log \frac{P(x)}{Q(x)}\right] \approx \frac{1}{S}\sum_{s=1}^{S} \log \frac{P(x_s)}{Q(x_s)}, \qquad x_s \sim P$$

five draws: 0.092

1, 1, 7, 1, 1 gives $\frac{1}{5}\big(4 \times 0.288 - 0.693\big)$, exactly 80/20 by luck

PART 3'S TOOL, ALREADY USEFUL

KL is an average, so samples can estimate it

$$\mathrm{KL}(P \,\|\, Q) = \mathbb{E}_{x\sim P}\left[\log \frac{P(x)}{Q(x)}\right] \approx \frac{1}{S}\sum_{s=1}^{S} \log \frac{P(x_s)}{Q(x_s)}, \qquad x_s \sim P$$

five draws: 0.092

1, 1, 7, 1, 1 gives $\frac{1}{5}\left(4 \times 0.288 - 0.693\right)$, exactly 80/20 by luck

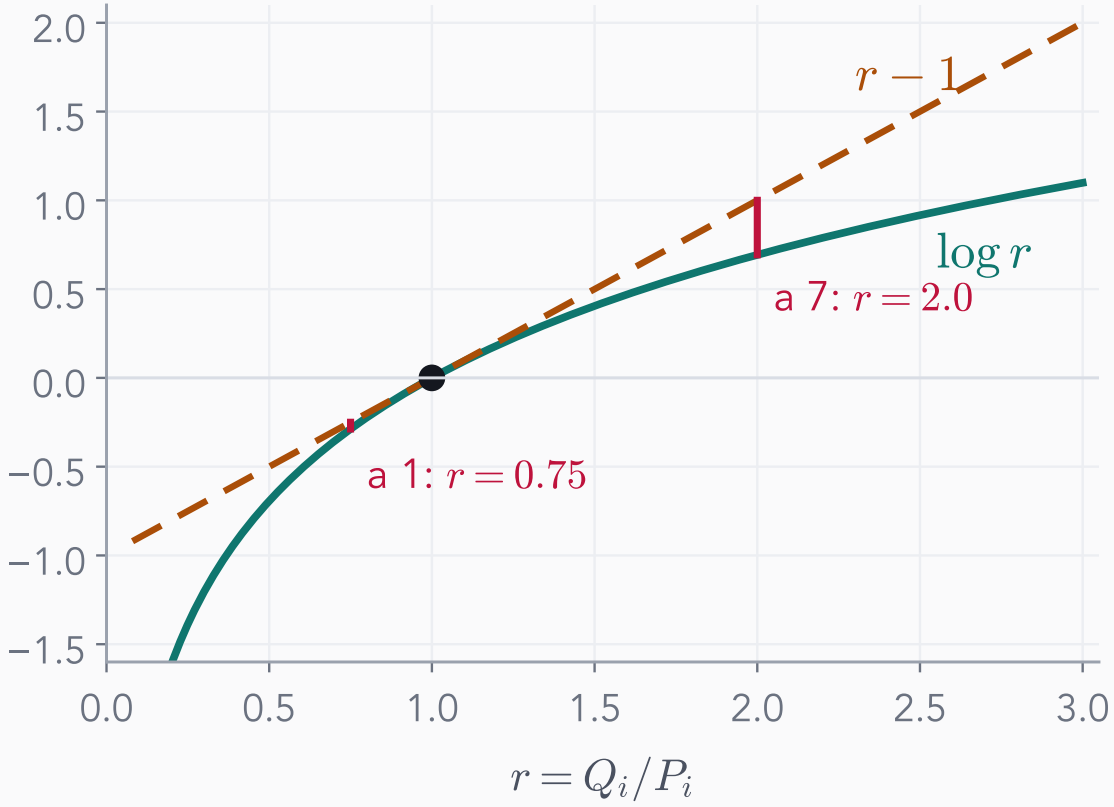
1,000 draws: 0.093

802 ones and the rest sevens, drawn at random

> **Notice what this needs: draws from the first argument, and both probabilities at each draw. Remember that for VI.**

WHY KL CAN NEVER BE NEGATIVE

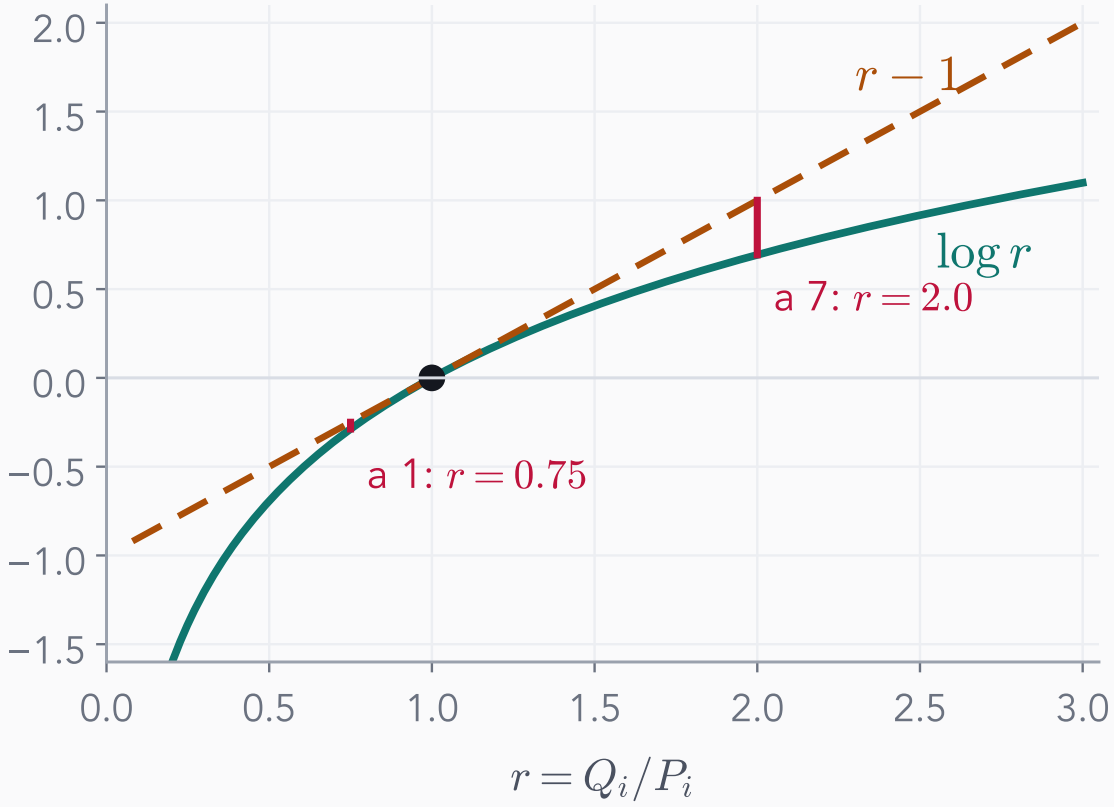
One picture: $\log r$ lies under its tangent



Let $r = Q_i/P_i$. The curve $\log r$ never rises above the line $r - 1$.

WHY KL CAN NEVER BE NEGATIVE

One picture: $\log r$ lies under its tangent

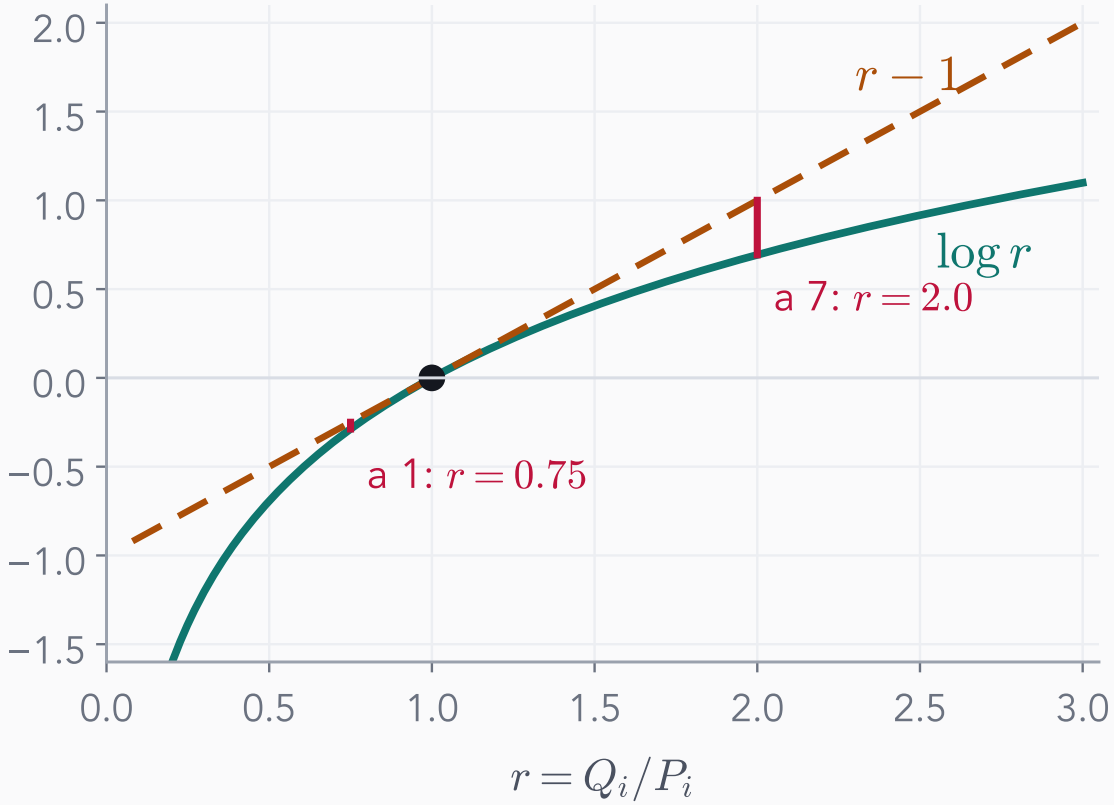


Let $r = Q_i/P_i$. The curve $\log r$ never rises above the line $r - 1$.

$$\mathrm{KL} = \sum_i P_i \left(-\log r_i\right) \;\geq\; \sum_i P_i \left(1 - r_i\right)$$

WHY KL CAN NEVER BE NEGATIVE

One picture: $\log r$ lies under its tangent



Let $r = Q_i/P_i$. The curve $\log r$ never rises above the line $r - 1$.

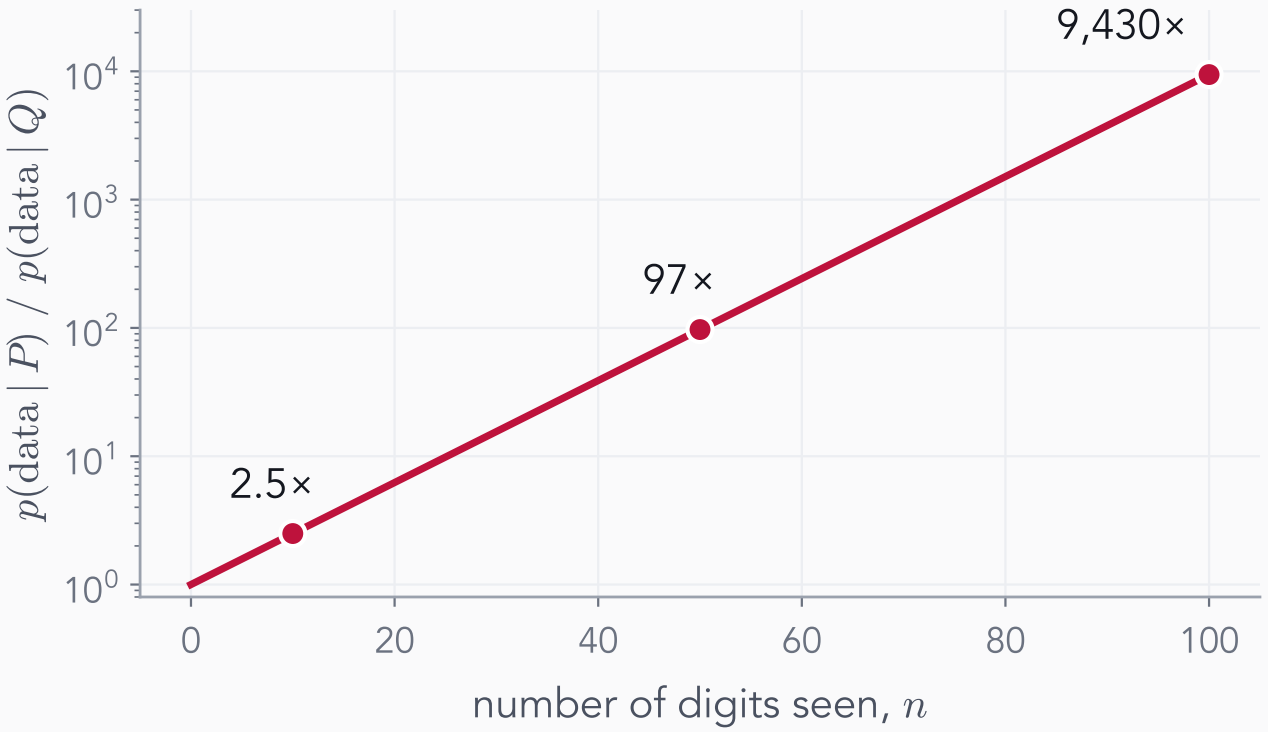
$$\mathrm{KL} = \sum_i P_i \left(-\log r_i\right) \;\geq\; \sum_i P_i \left(1 - r_i\right)$$

$$= \sum_i P_i - \sum_i Q_i = 1 - 1 = 0$$

Equality only when every $r_i = 1$: $Q = P$. This one inequality is what makes the ELBO a lower bound in Part 5.

WHAT THE NUMBER MEANS

KL is how fast evidence piles up



$$\frac{p(\text{data} \mid P)}{p(\text{data} \mid Q)} \approx e^{n\,\mathrm{KL}(P\|Q)}$$

After 10 digits the truth is 2.5× more likely; after 50, 97×; after 100, 9,430×.

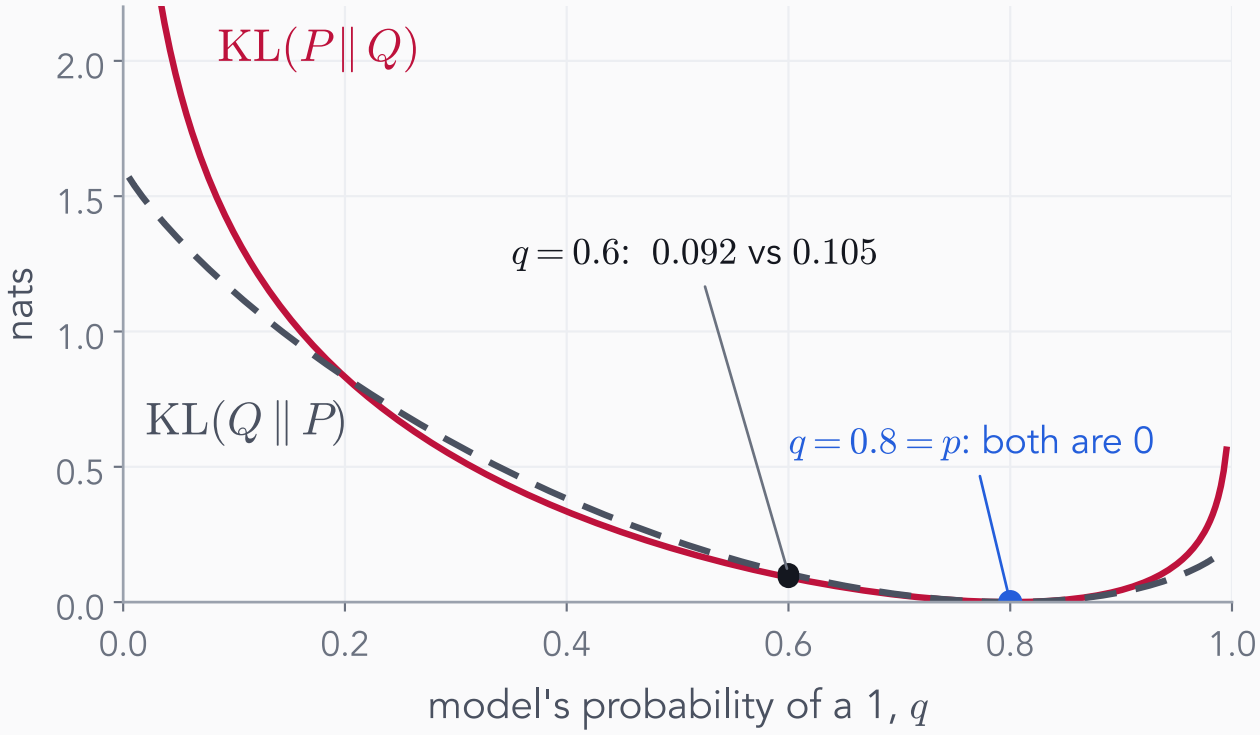
A small KL means the two models are hard to tell apart from data; a large one means a few samples settle it.

NOT A DISTANCE

Swap the arguments, get a different number

$$\mathrm{KL}(Q\|P) = 0.6\log\frac{0.6}{0.8} + 0.4\log\frac{0.4}{0.2}$$

$$= -0.1726 + 0.2773 = 0.1046$$



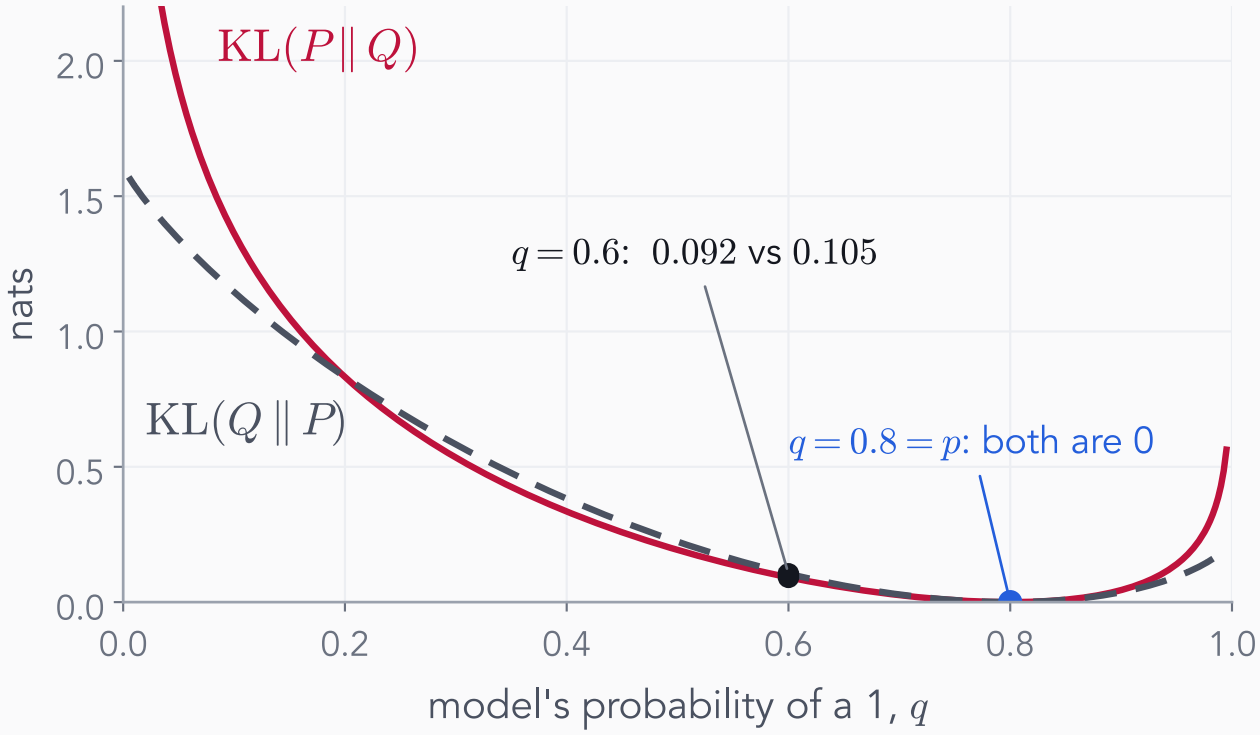
NOT A DISTANCE

Swap the arguments, get a different number

$$\mathrm{KL}(Q\|P) = 0.6\log\frac{0.6}{0.8} + 0.4\log\frac{0.4}{0.2}$$

$$= -0.1726 + 0.2773 = 0.1046$$

$0.105 \neq 0.092$: KL is not symmetric. "How wrong is Q about P" differs from "how wrong is P about Q".



NOT A DISTANCE

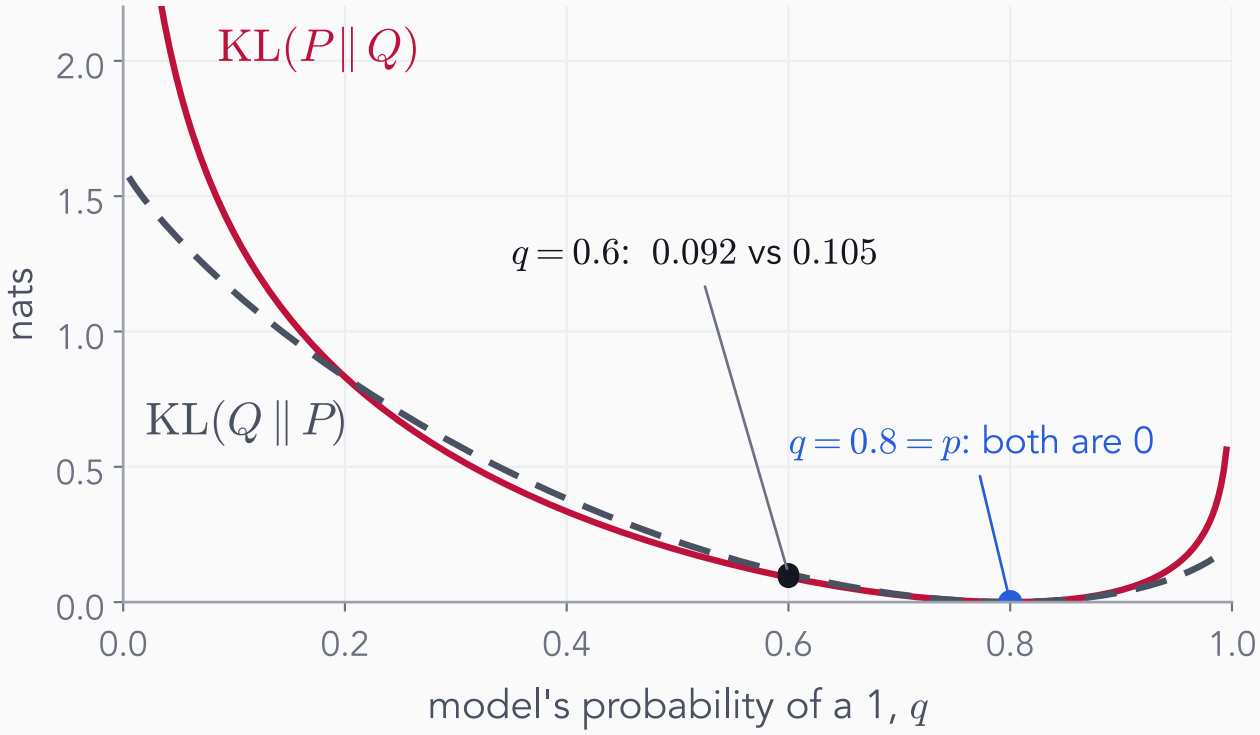
Swap the arguments, get a different number

$$\mathrm{KL}(Q\|P) = 0.6\log\frac{0.6}{0.8} + 0.4\log\frac{0.4}{0.2}$$

$$= -0.1726 + 0.2773 = 0.1046$$

$0.105 \neq 0.092$: KL is not symmetric. "How wrong is Q about P" differs from "how wrong is P about Q".

Push $q \to 0$ and $\mathrm{KL}(P\|Q) \to \infty$. Never give zero probability to something that happens.



THE ORDER MATTERS, SHARPLY

A model that "forgets" 7s

Truth $P = (0.8, 0.2)$. A model $Q' = (0.99, 0.01)$ that almost never expects a 7.

THE ORDER MATTERS, SHARPLY

A model that "forgets" 7s

Truth $P = (0.8, 0.2)$. A model $Q' = (0.99, 0.01)$ that almost never expects a 7.

$$\mathrm{KL}(P \| Q') = 0.43$$

real 7s keep showing up, and each costs $\log(0.2/0.01) = 3.0$ nats

THE ORDER MATTERS, SHARPLY

A model that "forgets" 7s

Truth $P = (0.8, 0.2)$. A model $Q' = (0.99, 0.01)$ that almost never expects a 7.

$$\mathrm{KL}(P \| Q') = 0.43$$

real 7s keep showing up, and each costs $\log(0.2/0.01) = 3.0$ nats

$$\mathrm{KL}(Q' \| P) = 0.18$$

averaged under Q', which almost never draws a 7, so the costly digit rarely counts

Whoever sits first does the weighting. $\mathrm{KL}(P\|Q)$ punishes Q for missing what P does; $\mathrm{KL}(Q\|P)$ barely notices.

Cross-entropy = entropy + KL

$$H(P, Q) = -\sum_i P_i \log Q_i$$

cross-entropy: the loss you train classifiers with

Cross-entropy = entropy + KL

$$H(P,Q) = -\sum_i P_i \log Q_i$$

cross-entropy: the loss you train classifiers with

$$= -\sum_i P_i \log P_i + \sum_i P_i \big(\log P_i - \log Q_i \big)$$

add and subtract $\sum_i P_i \log P_i$

Cross-entropy = entropy + KL

$$H(P, Q) = -\sum_i P_i \log Q_i$$

cross-entropy: the loss you train classifiers with

$$= -\sum_i P_i \log P_i + \sum_i P_i \big(\log P_i - \log Q_i \big)$$

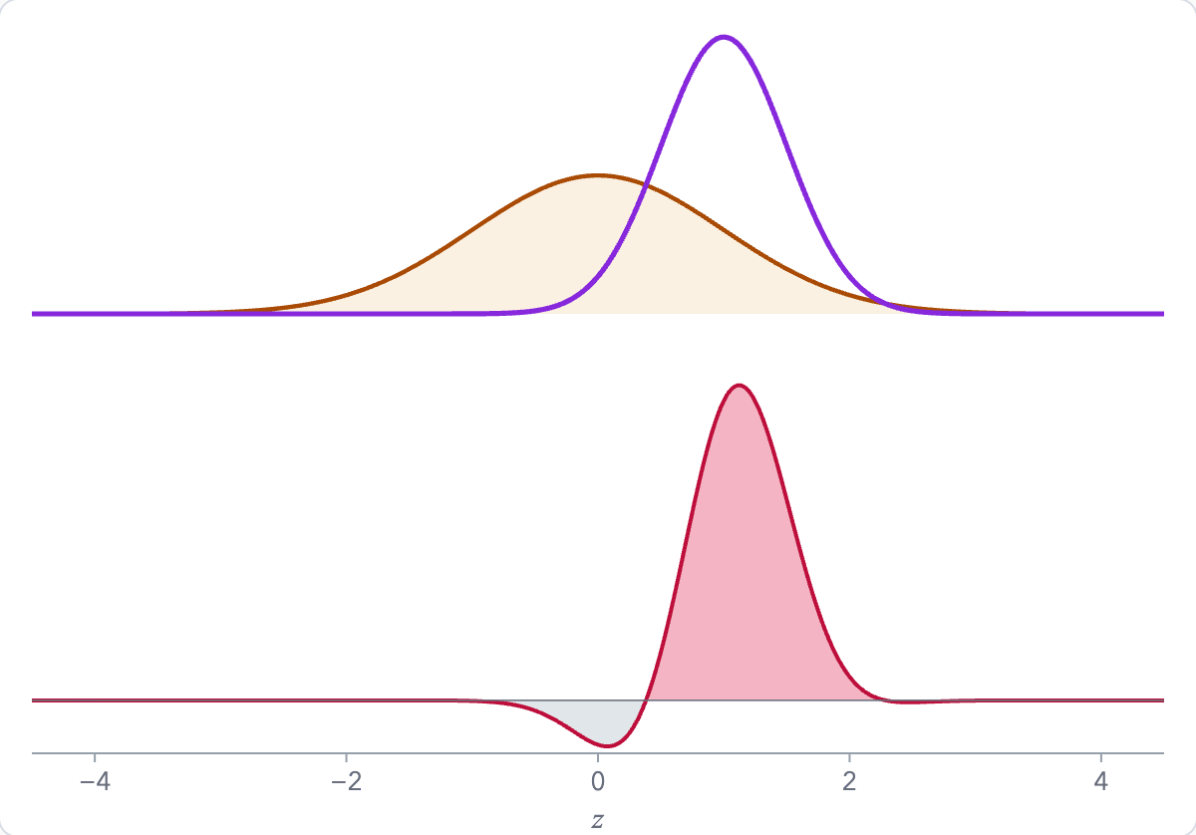
add and subtract $\sum_i P_i \log P_i$

$$= \underbrace{H(P)}_{\text{entropy: fixed by the data}} + \mathrm{KL}(P \,\|\, Q)$$

Lowering cross-entropy can only lower the KL part. So maximum likelihood minimises $\mathrm{KL}(\text{data} \,\|\, \text{model})$: data first.

FROM BARS TO BELL CURVES

KL between two bell curves is an area



$$\mathrm{KL}(q\|p) = \int q(z)\,\log\frac{q(z)}{p(z)}\,dz$$

$p = \mathcal{N}(0,1)$ fixed (orange). Move $q = \mathcal{N}(\mu,\sigma^2)$ (purple):

μ **1.00**

σ **0.50**

shaded area (numerically): **0.818**

formula: $\mathrm{KL}(q\|p) =$ **0.818**

the other way: $\mathrm{KL}(p\|q) =$ **2.807**

Start: $q = \mathcal{N}(1, 0.5^2)$ against $p = \mathcal{N}(0,1)$. The red area is 0.818 nats one way, 2.807 the other.

THE FORMULA, STEP 1 OF 2

Write out the log of the ratio

$$\log q(z) = -\log \sigma - \tfrac{1}{2}\log 2\pi - \frac{(z-\mu)^2}{2\sigma^2}$$

log of $\mathcal{N}(z; \mu, \sigma^2)$

THE FORMULA, STEP 1 OF 2

Write out the log of the ratio

$$\log q(z) = \underbrace{-\log \sigma - \tfrac{1}{2}\log 2\pi - \frac{(z-\mu)^2}{2\sigma^2}}_{\text{log of } \mathcal{N}(z;\mu,\sigma^2)}$$

$$\log p(z) = \underbrace{-\tfrac{1}{2}\log 2\pi - \frac{z^2}{2}}_{\text{log of } \mathcal{N}(z;0,1)}$$

THE FORMULA, STEP 1 OF 2

Write out the log of the ratio

$$\log q(z) = -\log \sigma - \tfrac{1}{2}\log 2\pi - \frac{(z-\mu)^2}{2\sigma^2}$$

log of $\mathcal{N}(z; \mu, \sigma^2)$

$$\log p(z) = -\tfrac{1}{2}\log 2\pi - \frac{z^2}{2}$$

log of $\mathcal{N}(z; 0, 1)$

$$\log \frac{q(z)}{p(z)} = -\log \sigma - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2}$$

subtract: the $\frac{1}{2}\log 2\pi$ terms cancel

Three terms. Next: average each one over $z \sim q$.

THE FORMULA, STEP 2 OF 2

Average each term over q

$\mathbb{E}_q\left[(z-\mu)^2\right] = \sigma^2$ (that is the variance)

$\mathbb{E}_q\left[z^2\right] = \mu^2 + \sigma^2$ (Part 3's example)

THE FORMULA, STEP 2 OF 2

Average each term over q

$\mathbb{E}_q\left[(z-\mu)^2\right] = \sigma^2$ (that is the variance)

$\mathbb{E}_q\left[z^2\right] = \mu^2 + \sigma^2$ (Part 3's example)

$$\mathrm{KL} = -\log\sigma - \frac{\sigma^2}{2\sigma^2} + \frac{\mu^2+\sigma^2}{2} = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log\sigma^2\right)$$

THE FORMULA, STEP 2 OF 2

Average each term over q

$\mathbb{E}_q\left[(z-\mu)^2\right] = \sigma^2$ (that is the variance)

$\mathbb{E}_q\left[z^2\right] = \mu^2 + \sigma^2$ (Part 3's example)

$$\mathrm{KL} = -\log\sigma - \frac{\sigma^2}{2\sigma^2} + \frac{\mu^2+\sigma^2}{2} = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log\sigma^2\right)$$

$$\mu = 1,\ \sigma = 0.5: \quad \tfrac{1}{2}\left(1 + 0.25 - 1 + 1.386\right) = 0.818\ \checkmark$$

Moving the centre costs $\frac{1}{2}\mu^2$; a width other than 1 costs $\frac{1}{2}(\sigma^2 - 1 - \log\sigma^2)$. This is the KL term of every VAE.

No formula for the KL? Estimate it by sampling

```
p = MixtureSameFamily(Categorical(torch.tensor([0.5, 0.5])),
                      Normal(torch.tensor([-2., 2.]), torch.tensor([0.5, 0.5])))
kl_divergence(Normal(0., 2.06), p)
# NotImplementedError: No KL(p || q) is implemented

def kl_via_sampling(q, p, n=100_000):
    z = q.sample([n])                          # draws from the FIRST argument
    return (q.log_prob(z) - p.log_prob(z)).mean()
```

Two bumps at ± 2: a mixture. PyTorch has KL formulas for pairs of standard families, not for this pair.

No formula for the KL? Estimate it by sampling

```
p = MixtureSameFamily(Categorical(torch.tensor([0.5, 0.5])),
                      Normal(torch.tensor([-2., 2.]), torch.tensor([0.5, 0.5])))
kl_divergence(Normal(0., 2.06), p)
# NotImplementedError: No KL(p || q) is implemented

def kl_via_sampling(q, p, n=100_000):
    z = q.sample([n])                          # draws from the FIRST argument
    return (q.log_prob(z) - p.log_prob(z)).mean()
```

Part 3's Monte Carlo, applied to the definition. For two candidate q's:

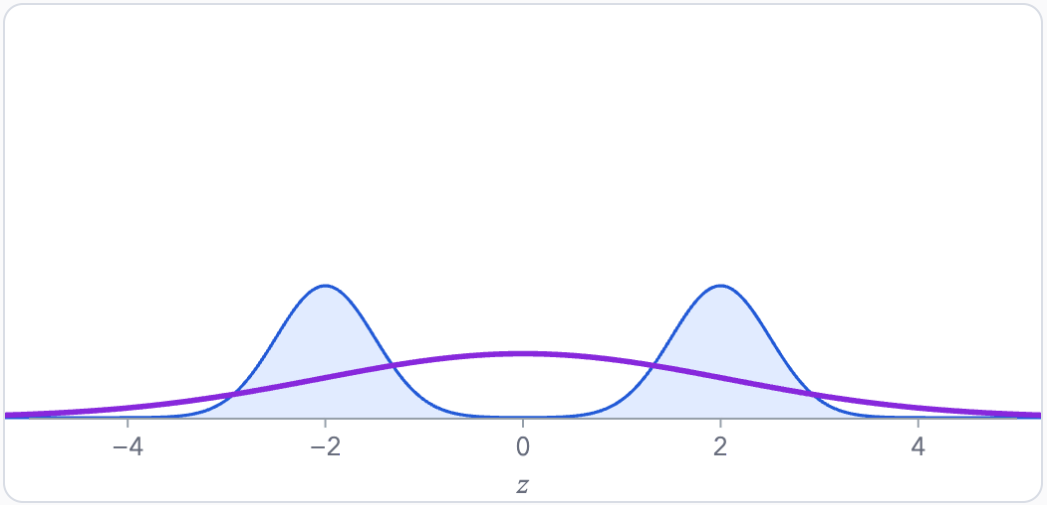
q	sampling	grid
wide, $\mathcal{N}(0,\ 2.06^2)$	2.087	2.095
one bump, $\mathcal{N}(-2,\ 0.5^2)$	0.693	0.693

$\mathrm{KL}(q \,\|\, p)$ prefers the one-bump q by a wide margin. Can we then optimise q through these samples? That needs Part 6; first, watch it happen.

FIT ONE BELL CURVE TO TWO BUMPS, BOTH WAYS

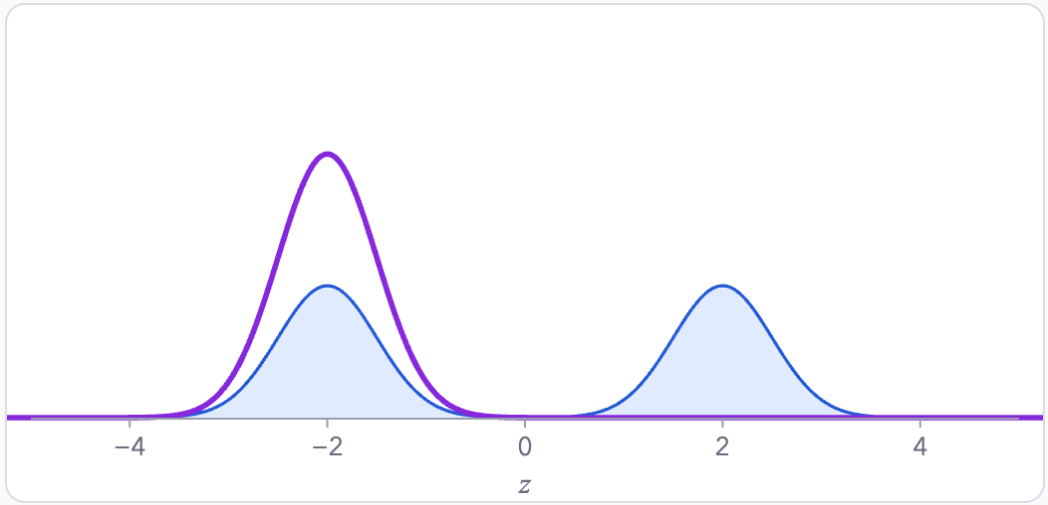
The direction of the KL decides what you get

minimise $\mathrm{KL}(p \,\|\, q)$: data first



centre **0.00** · width **2.06** · KL **0.724**

minimise $\mathrm{KL}(q \,\|\, p)$: q first



centre **−2.00** · width **0.50** · KL **0.693**

play reset gradient step **600**

> **Data first: cover everything, $\mathcal{N}(0.0,\ 2.06^2)$. q first: pick one bump, $\mathcal{N}(-2.0,\ 0.5^2)$. Mode-covering vs mode-seeking.**

WHY THE TWO DIRECTIONS BEHAVE SO DIFFERENTLY

Whoever sits first does the weighting

$$\mathrm{KL}(p\|q) = \mathbb{E}_p\big[\log p - \log q\big]$$

Averaged where the **data** are. Anywhere p has mass and q is tiny, $-\log q$ is huge. So q stretches to cover all of p, even the empty middle.

WHY THE TWO DIRECTIONS BEHAVE SO DIFFERENTLY

Whoever sits first does the weighting

$$\mathrm{KL}(p\|q) = \mathbb{E}_p\big[\log p - \log q\big]$$

Averaged where the **data** are. Anywhere p has mass and q is tiny, $-\log q$ is huge. So q stretches to cover all of p, even the empty middle.

$$\mathrm{KL}(q\|p) = \mathbb{E}_q\big[\log q - \log p\big]$$

Averaged where q puts mass. Anywhere q has mass and p is tiny, $-\log p$ is huge. So q avoids the gaps and hides inside one bump.

Variational inference will minimise $\mathrm{KL}(q \,\|\, \text{posterior})$: the second kind. Expect q to be cautious, never too wide, often too narrow.

PART 5 OF 8

Variational inference: pick the closest simple q

We have a posterior we cannot compute (Part 2), averages from samples (Part 3) and a number for "close" (Part 4). Put them together, **one move per slide**, and the integral we could not do disappears from the problem.

✓ Bayes: prior, likelihood, posterior

✓ Why the posterior is hard

✓ Averages from samples

✓ KL divergence

5 **Variational inference and the ELBO**

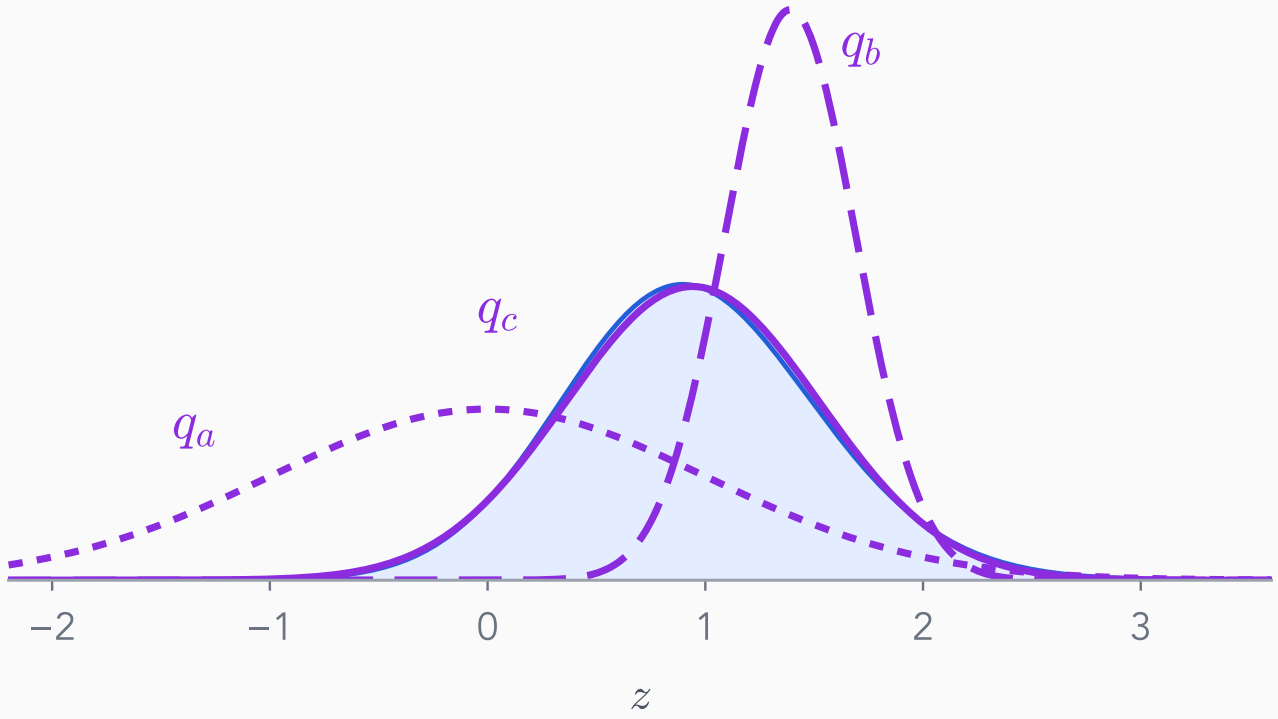
6 Climbing the ELBO: gradients through samples

7 Beyond one number

8 Many data points: towards the VAE

STEP 1: CHOOSE A FAMILY OF SIMPLE SHAPES

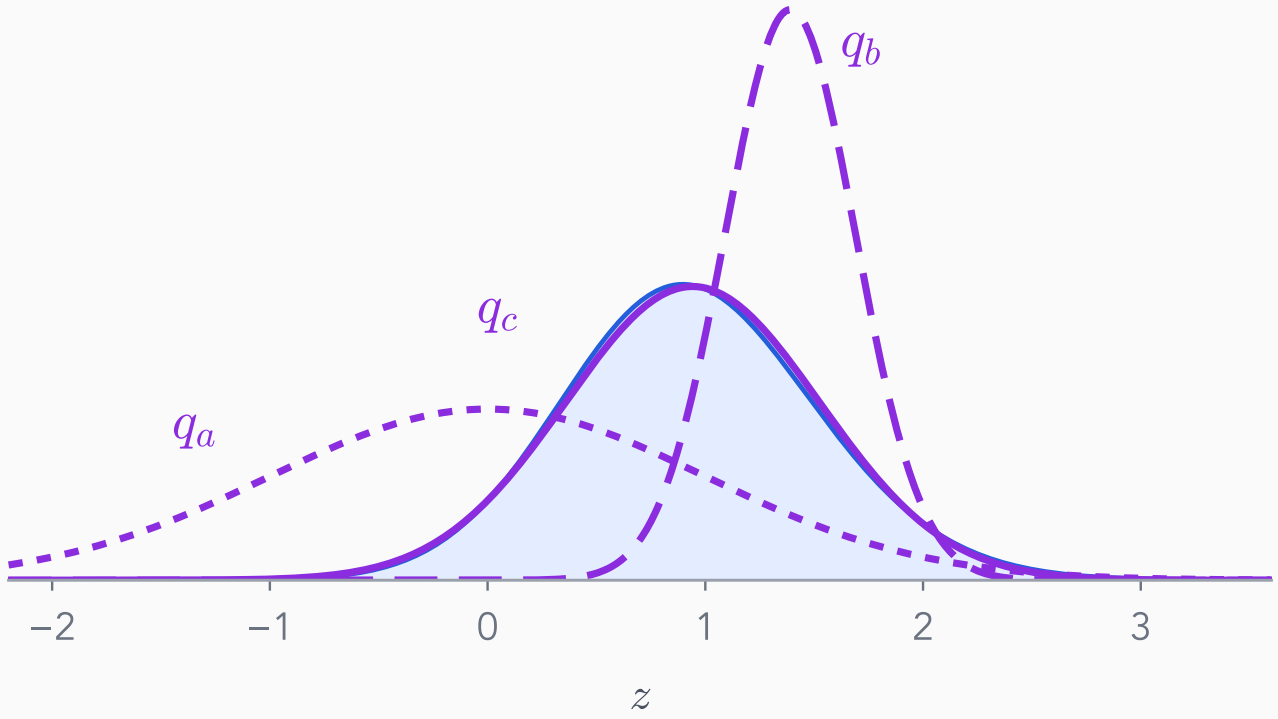
Every bell curve $q(z) = \mathcal{N}(m, s^2)$: two dials



Blue: the coin's posterior over z (Part 2's grid, for checking only). Purple: three members of the family.

STEP 1: CHOOSE A FAMILY OF SIMPLE SHAPES

Every bell curve $q(z) = \mathcal{N}(m, s^2)$: two dials

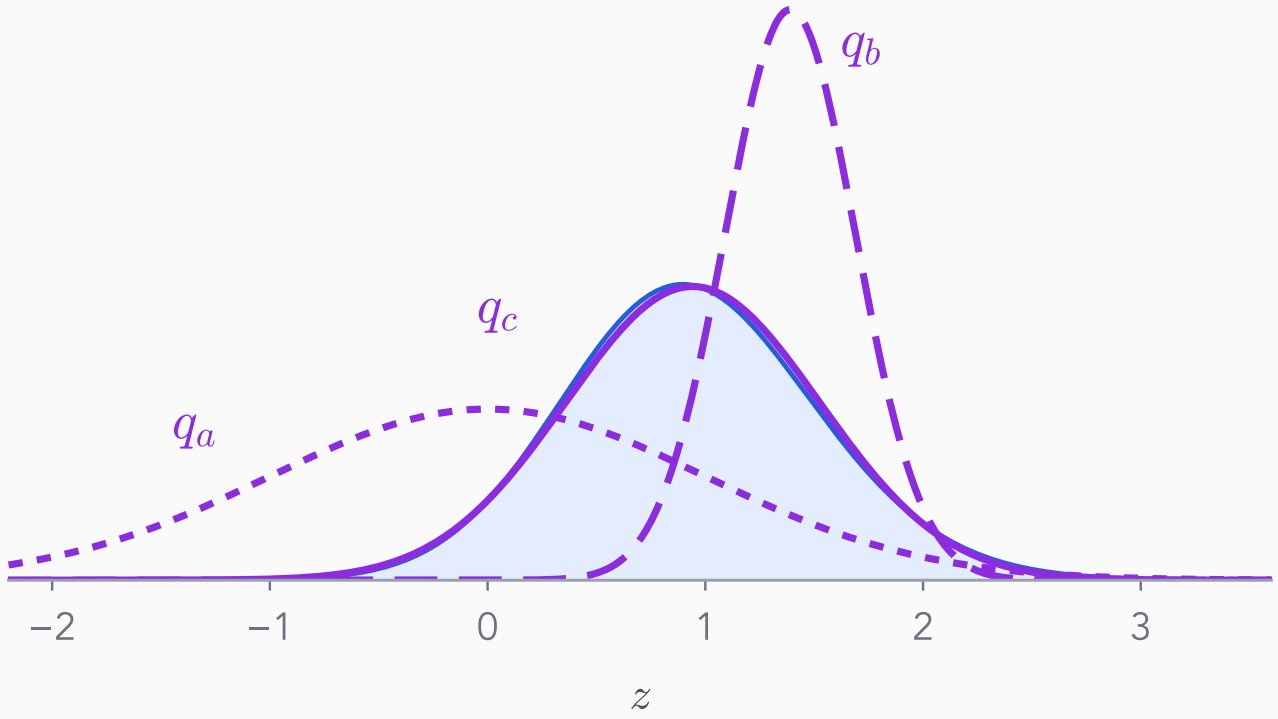


Blue: the coin's posterior over z (Part 2's grid, for checking only). Purple: three members of the family.

Which is closest? Guess first.

STEP 1: CHOOSE A FAMILY OF SIMPLE SHAPES

Every bell curve $q(z) = \mathcal{N}(m, s^2)$: two dials



Blue: the coin's posterior over z (Part 2's grid, for checking only). Purple: three members of the family.

Which is closest? Guess first.

q	m	s	$\mathrm{KL}(q \,\|\, \text{posterior})$
q_a: the prior	0.00	1.00	1.897
q_b: narrow, at $\log 4$	1.39	0.30	0.627
q_c	0.94	0.58	**0.002**

Step 2: of all the bell curves, take the one with the smallest KL to the posterior. That is variational inference.

THE GOAL, WRITTEN DOWN

Find the q with the smallest KL to the posterior

$$q^\star = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(z) \,\|\, p(z \mid x) \big)$$

$\mathcal{Q}$: the shapes we allow

Gaussians $\mathcal{N}(m, s^2)$. Two numbers to find. (The VAE: a few dozen.)

THE GOAL, WRITTEN DOWN

Find the q with the smallest KL to the posterior

$$q^\star = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(z) \,\|\, p(z \mid x) \big)$$

$\mathcal{Q}$: the shapes we allow

Gaussians $\mathcal{N}(m, s^2)$. Two numbers to find. (The VAE: a few dozen.)

Why q first?

KL averages under its first argument (Part 4). We can draw from q; we cannot draw from the posterior (Part 3's catch).

THE GOAL, WRITTEN DOWN

Find the q with the smallest KL to the posterior

$$q^\star = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(z) \,\|\, p(z \mid x) \big)$$

$\mathcal{Q}$: the shapes we allow

Gaussians $\mathcal{N}(m, s^2)$. Two numbers to find. (The VAE: a few dozen.)

Why q first?

KL averages under its first argument (Part 4). We can draw from q; we cannot draw from the posterior (Part 3's catch).

What it will do

q first means mode-seeking (Part 4's two bumps): q will hide inside the posterior rather than spread over it.

Inference has become optimisation: two numbers, one objective. Before any maths, try it by hand.

TRY IT

You are the optimiser



Move $q = \mathcal{N}(m, s^2)$ (purple) onto the posterior (blue):

m **0.00**

s **1.00**

$\mathrm{KL}(q \,\|\, \text{posterior}) =$ **1.897**

The best possible: 0.0019 nats, at $m = 0.94$, $s = 0.58$.

We cheated: that readout uses the exact posterior from Part 2's grid. In a real problem there is no grid. Can we score q without it?

THE CATCH

The KL to the posterior contains the evidence

$$\mathrm{KL}\big(q \,\|\, p(z \mid x)\big) = \mathbb{E}_{z\sim q}\big[\log q(z) - \log p(z \mid x)\big]$$

THE CATCH

The KL to the posterior contains the evidence

$$\mathrm{KL}\big(q \,\|\, p(z \mid x)\big) = \mathbb{E}_{z\sim q}\big[\log q(z) - \log p(z \mid x)\big]$$

$$p(z \mid x) = \frac{p(x, z)}{p(x)}$$

THE CATCH

The KL to the posterior contains the evidence

$$\mathrm{KL}\big(q \,\|\, p(z \mid x)\big) = \mathbb{E}_{z\sim q}\big[\log q(z) - \log p(z \mid x)\big]$$

$$p(z \mid x) = \frac{p(x, z)}{p(x)}$$

To score q this way we need $p(x)$: the integral of Part 2. We are back where we started.

Four short moves get the evidence out of the way. Watch where $\log p(x)$ goes.

Write out the KL

$$\mathrm{KL}\big(q(z) \,\|\, p(z \mid x)\big)$$

$$= \mathbb{E}_{z\sim q}\Big[\, \log q(z) - \log p(z \mid x) \,\Big]$$

Write out the KL

$$\mathrm{KL}\big(q(z) \,\|\, p(z \mid x)\big)$$

$$= \mathbb{E}_{z\sim q}\Big[\, \log q(z) - \log p(z \mid x) \,\Big]$$

The definition from Part 4: the average, under the first argument, of the log of the ratio. The highlighted term is the one we cannot compute.

Bayes: the posterior is the joint over the evidence

$$\mathbb{E}_q\big[\log q(z) - \log p(z \mid x)\big]$$

$$= \mathbb{E}_q\big[\log q(z) - \log p(x, z)\big] + \log p(x)$$

Bayes: the posterior is the joint over the evidence

$$\mathbb{E}_q\big[\log q(z) - \log p(z \mid x)\big]$$

$$= \mathbb{E}_q\big[\log q(z) - \log p(x, z)\big] \; + \; \log p(x)$$

$\log p(z \mid x) = \log p(x, z) - \log p(x)$. The evidence has no z in it: the same number for every draw, so it comes out of the average.

The joint $p(x, z) = p(z)\,p(x \mid z)$ is Part 2's one line of code. All the difficulty is now in one number.

Move the evidence to the other side

$$\mathrm{KL}\big(q \,\|\, p(z \mid x)\big) = \mathbb{E}_q\big[\log q(z) - \log p(x, z)\big] + \log p(x)$$

$$\log p(x) = \underbrace{\mathbb{E}_q\big[\log p(x, z) - \log q(z)\big]}_{\mathrm{ELBO}(q)} + \mathrm{KL}\big(q \,\|\, p(z \mid x)\big)$$

Move the evidence to the other side

$$\mathrm{KL}\big(q \,\|\, p(z \mid x)\big) = \mathbb{E}_q\big[\log q(z) - \log p(x, z)\big] + \log p(x)$$

$$\log p(x) = \underbrace{\mathbb{E}_q\big[\log p(x, z) - \log q(z)\big]}_{\mathrm{ELBO}(q)} + \mathrm{KL}\big(q \,\|\, p(z \mid x)\big)$$

Rearrange, and give the highlighted average a name: the **ELBO**. Every piece of it is computable: the joint (one line of code) and q (ours, so we can evaluate it and draw from it).

Why "Evidence Lower BOund"? The last move.

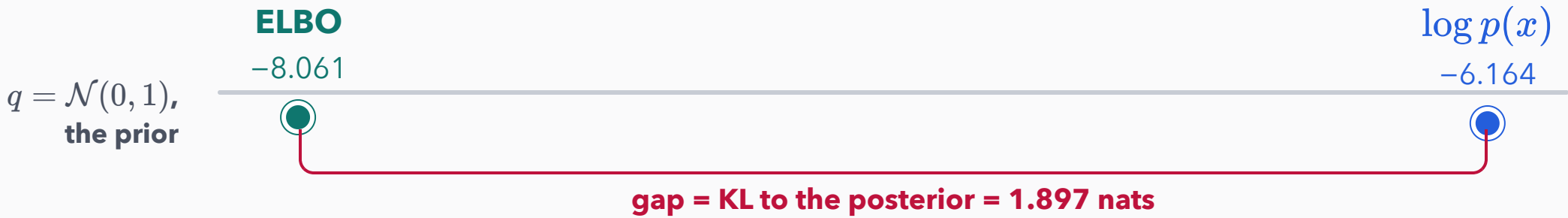
The KL is never negative, so the ELBO is a lower bound

$$\log p(x) = \mathrm{ELBO}(q) + \underbrace{\mathrm{KL}\big(q \,\|\, p(z \mid x)\big)}_{\geq 0} \;\geq\; \mathrm{ELBO}(q)$$

The KL is never negative, so the ELBO is a lower bound

$$\log p(x) = \mathrm{ELBO}(q) + \underbrace{\mathrm{KL}\big(q \,\|\, p(z \mid x)\big)}_{\geq 0} \;\geq\; \mathrm{ELBO}(q)$$

Our coin, two choices of q. The right end, $\log p(x) = -6.164$, is fixed by the model and the data: it does not move when q changes.



The KL is never negative, so the ELBO is a lower bound

$$\log p(x) = \mathrm{ELBO}(q) + \underbrace{\mathrm{KL}\big(q \,\|\, p(z \mid x)\big)}_{\geq 0} \;\geq\; \mathrm{ELBO}(q)$$

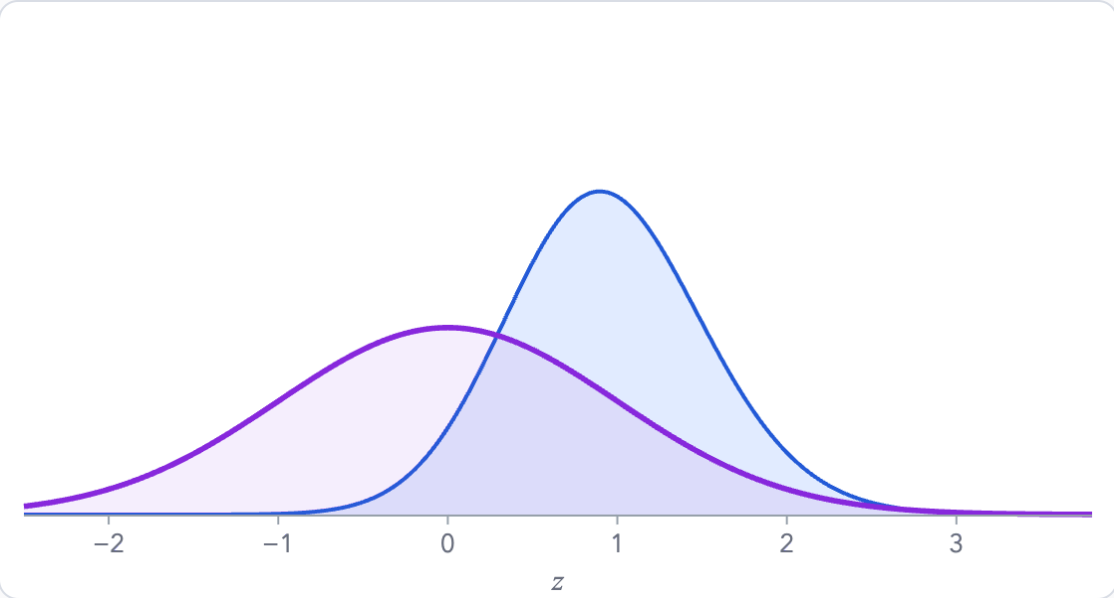
Our coin, two choices of q. The right end, $\log p(x) = -6.164$, is fixed by the model and the data: it does not move when q changes.



> **The total cannot move, so raising the ELBO must shrink the KL. Maximise the ELBO, and q approaches the posterior, without ever computing $p(x)$.**

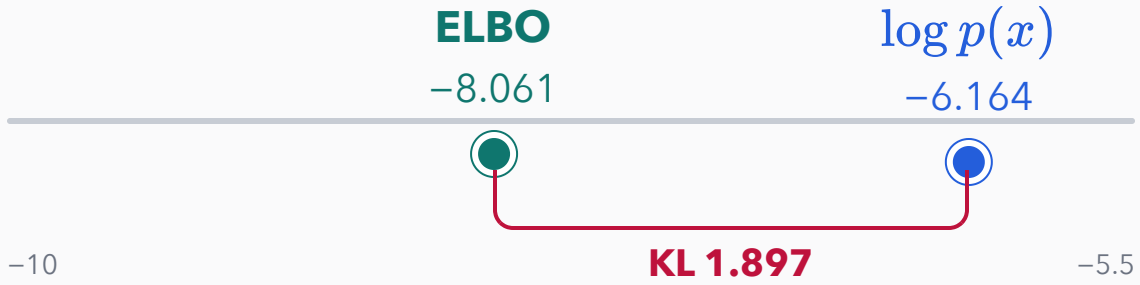
TRY IT

Move q: the ELBO and the gap change, their sum never does



m **0.00**

s **1.00**



> Same dials as before, but now the score is the ELBO, which needs no grid. Push the ELBO up and the gap closes by exactly the same amount.

CHECK IT WITH NUMBERS YOU CAN DO BY HAND

A two-value hidden cause: neat or messy writer?

$z \in \{\text{neat}, \text{messy}\}$, prior $\frac{1}{2}, \frac{1}{2}$. Observation x: a thick stroke.

$$p(x \mid \text{neat}) = 0.8, \quad p(x \mid \text{messy}) = 0.2$$

CHECK IT WITH NUMBERS YOU CAN DO BY HAND

A two-value hidden cause: neat or messy writer?

$z \in \{\text{neat}, \text{messy}\}$, prior $\frac{1}{2}, \frac{1}{2}$. Observation x: a thick stroke.

$$p(x \mid \text{neat}) = 0.8, \quad p(x \mid \text{messy}) = 0.2$$

evidence $p(x) = 0.5$ · true posterior $(0.8,\ 0.2)$

CHECK IT WITH NUMBERS YOU CAN DO BY HAND

A two-value hidden cause: neat or messy writer?

$z \in \{\text{neat}, \text{messy}\}$, prior $\frac{1}{2}, \frac{1}{2}$. Observation x: a thick stroke.

$$p(x \mid \text{neat}) = 0.8, \quad p(x \mid \text{messy}) = 0.2$$

evidence $p(x) = 0.5$ · true posterior $(0.8,\ 0.2)$

Our guess $q = (q_0,\ 1 - q_0)$:

q_0 = `0.60`

quantity	value
$\mathbb{E}_q[\log p(x \mid z)]$	−0.778
$-\,\mathrm{KL}(q \,\|\, \text{prior})$	−0.020
ELBO	**−0.798**
$+\,\mathrm{KL}(q \,\|\, \text{posterior})$	0.105
$\log p(x)$	**−0.693**

Every row is a sum of two terms you can work out on paper. At $q_0 = 0.8$ the gap is exactly 0.

The joint is prior times likelihood

$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\log p(x, z) - \log q(z)\big]$$

$$= \mathbb{E}_q\Big[\log p(x \mid z) + \log p(z) - \log q(z)\Big]$$

The joint is prior times likelihood

$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\log p(x, z) - \log q(z)\big]$$

$$= \mathbb{E}_q\Big[\log p(x \mid z) + \log p(z) - \log q(z)\Big]$$

$$= \mathbb{E}_q\big[\log p(x \mid z)\big] \;+\; \mathbb{E}_q\big[\log p(z) - \log q(z)\big]$$

The log of a product is a sum of logs; the average of a sum is the sum of the averages. Group the likelihood on its own, and the two terms that only involve z together.

Recognise the second average: a KL to the prior

$$\mathbb{E}_q\big[\log p(z) - \log q(z)\big]$$

$$= -\,\mathbb{E}_q\left[\log \frac{q(z)}{p(z)}\right] = -\mathrm{KL}\big(q(z) \,\|\, p(z)\big)$$

Recognise the second average: a KL to the prior

$$\mathbb{E}_q\big[\log p(z) - \log q(z)\big]$$

$$= -\mathbb{E}_q\left[\log \frac{q(z)}{p(z)}\right] = -\mathrm{KL}\big(q(z) \,\|\, p(z)\big)$$

Flip the sign to flip the fraction, then Part 4's definition. And for $q = \mathcal{N}(m, s^2)$, $p(z) = \mathcal{N}(0, 1)$, Part 4 already has the formula: $\frac{1}{2}\left(m^2 + s^2 - 1 - \log s^2\right)$.

Two KLs now appear: to the posterior (the gap, not computable) and to the prior (inside the ELBO, by formula). Do not mix them up.

PUT THE PIECES TOGETHER

The ELBO, in symbols and in words

$$\text{ELBO} = \underbrace{\mathbb{E}_{z \sim q}\big[\log p(x \mid z)\big]}_{\text{explain the data}} - \underbrace{\text{KL}\big(q(z) \,\|\, p(z)\big)}_{\text{stay close to the prior}}$$

PUT THE PIECES TOGETHER

The ELBO, in symbols and in words

$$\text{ELBO} = \underbrace{\mathbb{E}_{z\sim q}\big[\log p(x \mid z)\big]}_{\text{explain the data}} - \underbrace{\mathrm{KL}\big(q(z) \,\|\, p(z)\big)}_{\text{stay close to the prior}}$$

For the coin: the average, over draws z from q, of $8 \log \text{sigmoid}(z) + 2 \log(1 - \text{sigmoid}(z))$.

For the coin: $\frac{1}{2}\left(m^2 + s^2 - 1 - \log s^2\right)$, no sampling needed.

In the VAE these two terms get names: the reconstruction term and the KL term. Same two terms, a bigger x.

TRY IT

The two terms pull against each other

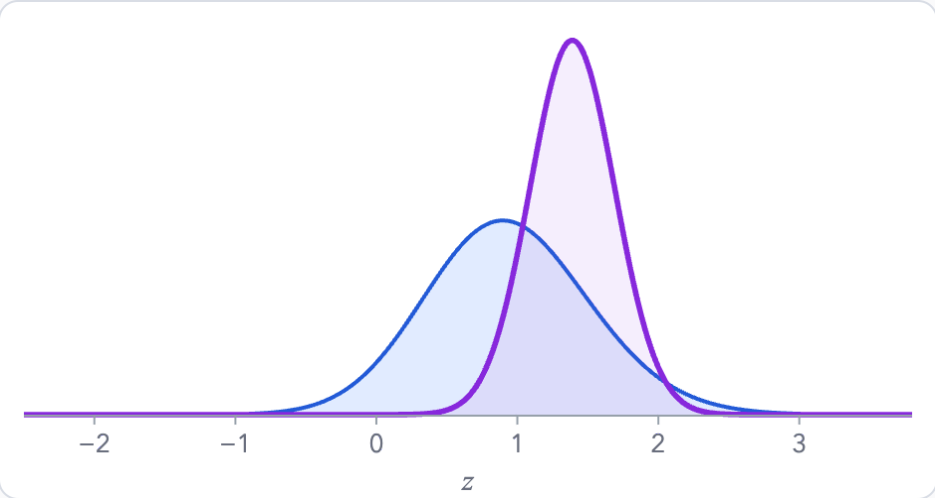
m **1.39**

s **0.30**

explain the data: $\mathbb{E}_q[\log p(x \mid z)]$	−5.076
− stay near the prior: $\mathrm{KL}(q \,\|\, p(z))$	1.715
ELBO	**−6.791**

TRY IT

The two terms pull against each other



m **1.39**

s **0.30**

explain the data: $\mathbb{E}_q[\log p(x \mid z)]$	−5.076
− stay near the prior: $\mathrm{KL}(q \,\|\, p(z))$	1.715
ELBO	**−6.791**

	$\mathbb{E}_q[\log p(x \mid z)]$	$\mathrm{KL}(q \,\|\, p(z))$	ELBO
q_a, the prior	−8.061	0.000	−8.061
q_b, narrow at $\log 4$	−5.076	1.715	−6.791
q_c, the best	**−5.513**	**0.653**	**−6.166**

> **The prior explains the data badly; the narrow q explains it best but strays far from the prior. The best q is the compromise.**

END OF PART 5

Three ways to read the same number

$$\mathrm{ELBO}(q) \le \log p(x)$$

a lower bound on the evidence, for every q

END OF PART 5

Three ways to read the same number

$$\mathrm{ELBO}(q) \le \log p(x)$$

a lower bound on the evidence, for every q

$$\mathrm{ELBO}(q) = \log p(x) - \mathrm{KL}\big(q \,\|\, p(z \mid x)\big)$$

the evidence minus the gap: maximising it pulls q onto the posterior

END OF PART 5

Three ways to read the same number

$$\mathrm{ELBO}(q) \le \log p(x)$$

a lower bound on the evidence, for every q

$$\mathrm{ELBO}(q) = \log p(x) - \mathrm{KL}\big(q \,\|\, p(z \mid x)\big)$$

the evidence minus the gap: maximising it pulls q onto the posterior

$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\log p(x \mid z)\big] - \mathrm{KL}\big(q \,\|\, p(z)\big)$$

fit minus cost: the form we compute, and (with a minus sign) the VAE's loss

Next: actually climb it, with samples and gradients, and watch q move.

PART 6 OF 8

Climbing the ELBO with samples and gradients

The ELBO is an average under q. Estimate it from a few draws (Part 3), then climb it by gradient ascent on m and s. One step needs care: **a gradient through a random draw**.

- ✓ Bayes: prior, likelihood, posterior
- ✓ Why the posterior is hard
- ✓ Averages from samples
- ✓ KL divergence
- ✓ Variational inference and the ELBO
- 6 **Climbing the ELBO: gradients through samples**
- 7 Beyond one number
- 8 Many data points: towards the VAE

STEP 1: ESTIMATE THE ELBO FROM DRAWS

Three lines, using only things we can compute

```
def log_joint(z):                    # Part 2's one line
    return (Normal(0, 1).log_prob(z)
            + 8 * F.logsigmoid(z) + 2 * F.logsigmoid(-z))

q = Normal(0.0, 1.0)                 # start: q is the prior
z = q.sample((S,))                   # S draws from q
elbo = (log_joint(z) - q.log_prob(z)).mean()
```

The joint: prior times likelihood, in logs. Any z, one line.

STEP 1: ESTIMATE THE ELBO FROM DRAWS

Three lines, using only things we can compute

```
def log_joint(z):                    # Part 2's one line
    return (Normal(0, 1).log_prob(z)
            + 8 * F.logsigmoid(z) + 2 * F.logsigmoid(-z))

q = Normal(0.0, 1.0)                 # start: q is the prior
z = q.sample((S,))                   # S draws from q
elbo = (log_joint(z) - q.log_prob(z)).mean()
```

Draws from q: easy, because we chose q.

STEP 1: ESTIMATE THE ELBO FROM DRAWS

Three lines, using only things we can compute

```
def log_joint(z):                  # Part 2's one line
    return (Normal(0, 1).log_prob(z)
            + 8 * F.logsigmoid(z) + 2 * F.logsigmoid(-z))

q = Normal(0.0, 1.0)                    # start: q is the prior
z = q.sample((S,))                      # S draws from q
elbo = (log_joint(z) - q.log_prob(z)).mean()
```

The ELBO's definition, averaged:

$$\mathbb{E}_q[\log p(x, z) - \log q(z)].$$

STEP 1: ESTIMATE THE ELBO FROM DRAWS

Three lines, using only things we can compute

```
def log_joint(z):                    # Part 2's one line
    return (Normal(0, 1).log_prob(z)
            + 8 * F.logsigmoid(z) + 2 * F.logsigmoid(-z))

q = Normal(0.0, 1.0)                 # start: q is the prior
z = q.sample((S,))                   # S draws from q
elbo = (log_joint(z) - q.log_prob(z)).mean()
```

The ELBO's definition, averaged:
$\mathbb{E}_q[\log p(x, z) - \log q(z)]$.

draws S	estimate
10	−8.377
100	−7.884
10,000	−8.064
exact	**−8.061**

> No $p(x)$, no grid. With 10 draws the estimate wobbles by about 1.08 nats; that is fine for climbing, as Part 3 promised.

STEP 2: THE GRADIENT. THE SUBTLE PART.

The gradient of an average whose draws move

$$\frac{\partial}{\partial m}\,\mathbb{E}_{z\sim\mathcal{N}(m,\,s^2)}\big[f(z)\big] \;=\; ?$$

STEP 2: THE GRADIENT. THE SUBTLE PART.

The gradient of an average whose draws move

$$\frac{\partial}{\partial m}\,\mathbb{E}_{z\sim\mathcal{N}(m,\,s^2)}\big[f(z)\big] \;=\; ?$$

Move m and the draws themselves move. The randomness depends on what we differentiate.

STEP 2: THE GRADIENT. THE SUBTLE PART.

The gradient of an average whose draws move

$$\frac{\partial}{\partial m}\,\mathbb{E}_{z\sim\mathcal{N}(m,\,s^2)}\big[f(z)\big] \;=\; ?$$

Move m and the draws themselves move. The randomness depends on what we differentiate.

A toy with a known answer (Part 3's): $q = \mathcal{N}(\mu, \sigma^2)$ with $\mu = 1$, $\sigma = 0.5$, and $f(z) = z^2$. Since $\mathbb{E}[z^2] = \mu^2 + \sigma^2$, the exact gradient is $\dfrac{\partial}{\partial \mu}(\mu^2 + \sigma^2) = 2\mu = 2$.

Two standard ways to estimate it from draws: the score function, and reparameterisation.

WAY 1

The score function (the log-derivative trick)

$$\frac{\partial}{\partial \mu} \int q_\mu(z)\, f(z)\, dz = \int \frac{\partial q_\mu(z)}{\partial \mu}\, f(z)\, dz$$

f does not depend on μ; only q does

WAY 1

The score function (the log-derivative trick)

$$\frac{\partial}{\partial \mu} \int q_\mu(z)\, f(z)\, dz = \int \frac{\partial q_\mu(z)}{\partial \mu}\, f(z)\, dz$$

f does not depend on μ; only q does

$$= \int q_\mu(z)\, \frac{\partial \log q_\mu(z)}{\partial \mu}\, f(z)\, dz$$

$\partial q = q\, \partial \log q$: multiply and divide by q

WAY 1

The score function (the log-derivative trick)

$$\frac{\partial}{\partial \mu} \int q_\mu(z)\, f(z)\, dz = \int \frac{\partial q_\mu(z)}{\partial \mu}\, f(z)\, dz$$

f does not depend on μ; only q does

$$= \int q_\mu(z)\, \frac{\partial \log q_\mu(z)}{\partial \mu}\, f(z)\, dz$$

$\partial q = q\, \partial \log q$: multiply and divide by q

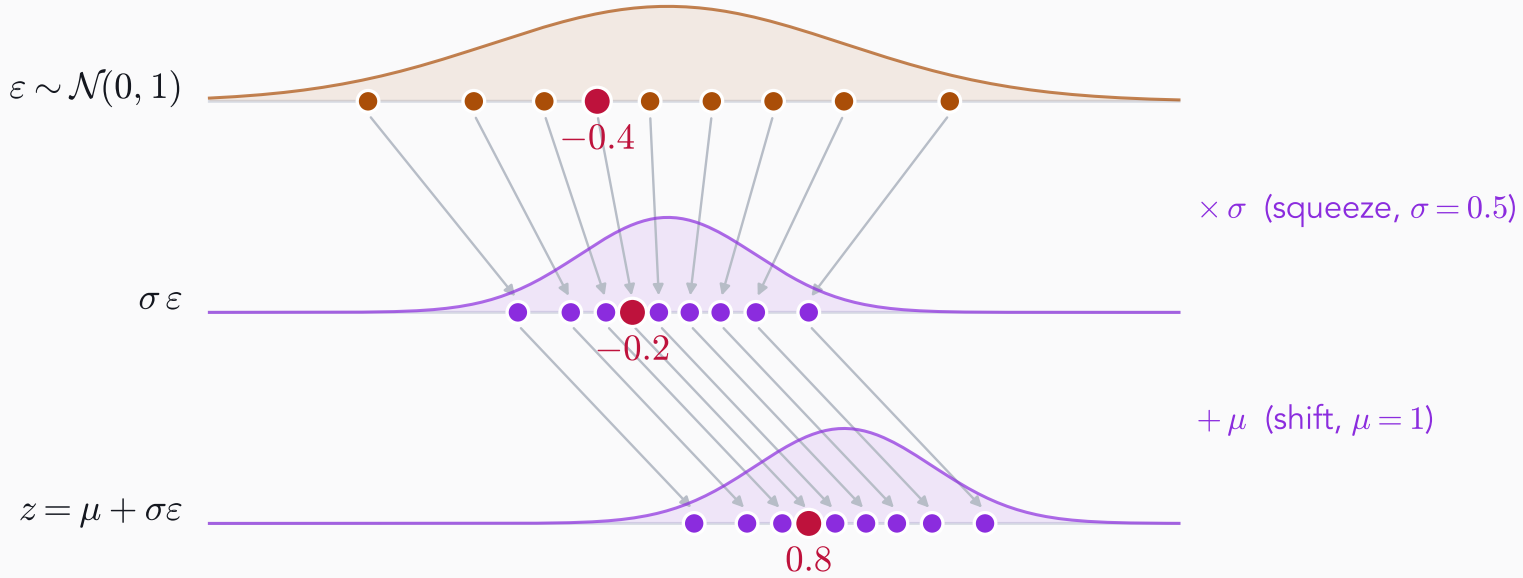
$$= \mathbb{E}_{z \sim q}\left[f(z)\, \frac{z - \mu}{\sigma^2} \right]$$

an average under q again; for a Gaussian, $\partial \log q / \partial \mu = (z - \mu)/\sigma^2$

Estimate it with draws: average $f(z)\,(z - \mu)/\sigma^2$. Works for any q and any f, even an f you cannot differentiate.

WAY 2

Reparameterisation: stretch and shift standard noise



$$z = \mu + \sigma\varepsilon,\ \varepsilon \sim \mathcal{N}(0,1) \quad \Rightarrow \quad \mathbb{E}_{z\sim q}\big[f(z)\big] = \mathbb{E}_{\varepsilon}\big[f(\mu+\sigma\varepsilon)\big] \quad \Rightarrow \quad \frac{\partial}{\partial\mu}\mathbb{E} = \mathbb{E}_{\varepsilon}\big[f'(\mu+\sigma\varepsilon)\big]$$

> **Now the randomness, ε, does not depend on μ. The parameter sits inside f's argument, and the ordinary chain rule applies.**

ONE DRAW, BOTH WAYS

$$\varepsilon = -0.4\text{, so } z = 1 + 0.5 \times (-0.4) = 0.8$$

Score function

$$f(z)\,\frac{z-\mu}{\sigma^2} = 0.64 \times \frac{-0.2}{0.25} = -0.512$$

ONE DRAW, BOTH WAYS

$$\varepsilon = -0.4\text{, so } z = 1 + 0.5 \times (-0.4) = 0.8$$

Score function

$$f(z)\,\frac{z-\mu}{\sigma^2} = 0.64 \times \frac{-0.2}{0.25} = -0.512$$

Reparameterisation

$$f'(z)\,\frac{\partial z}{\partial \mu} = 2z \times 1 = 1.6$$

ONE DRAW, BOTH WAYS

$$\varepsilon = -0.4\text{, so } z = 1 + 0.5 \times (-0.4) = 0.8$$

Score function

$$f(z)\,\frac{z-\mu}{\sigma^2} = 0.64 \times \frac{-0.2}{0.25} = -0.512$$

Reparameterisation

$$f'(z)\,\frac{\partial z}{\partial \mu} = 2z \times 1 = 1.6$$

> The exact answer is 2. One draw says -0.51 one way and 1.6 the other. Both are right **on average**; how noisy is each?

TRY IT

Same draws, same average, very different noise



draws: **300** (the first is $\varepsilon = -0.4$)

score function: mean **1.98**, spread **4.90**

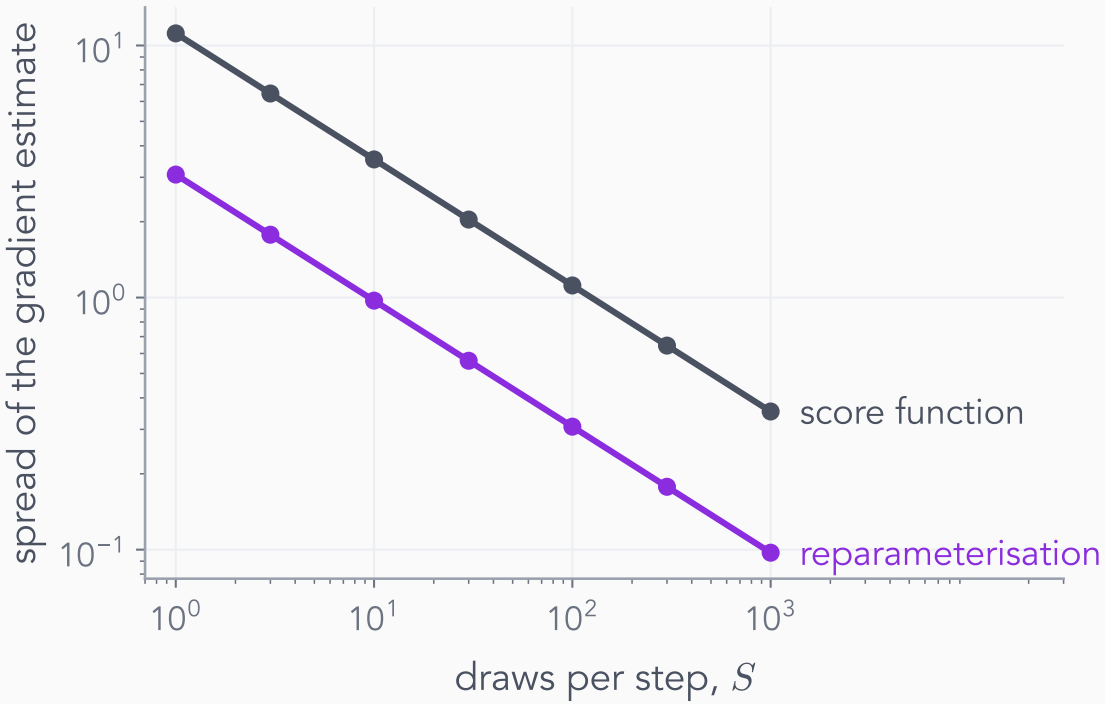
reparameterisation: mean **1.99**, spread **0.99**

exact: 2 (red line); diamonds: running means

> **Over a million draws: spread 4.65 for the score function, 1.0 for reparameterisation. The score function needs about 22× more draws for the same accuracy.**

ON THE COIN'S ELBO, THE SAME STORY

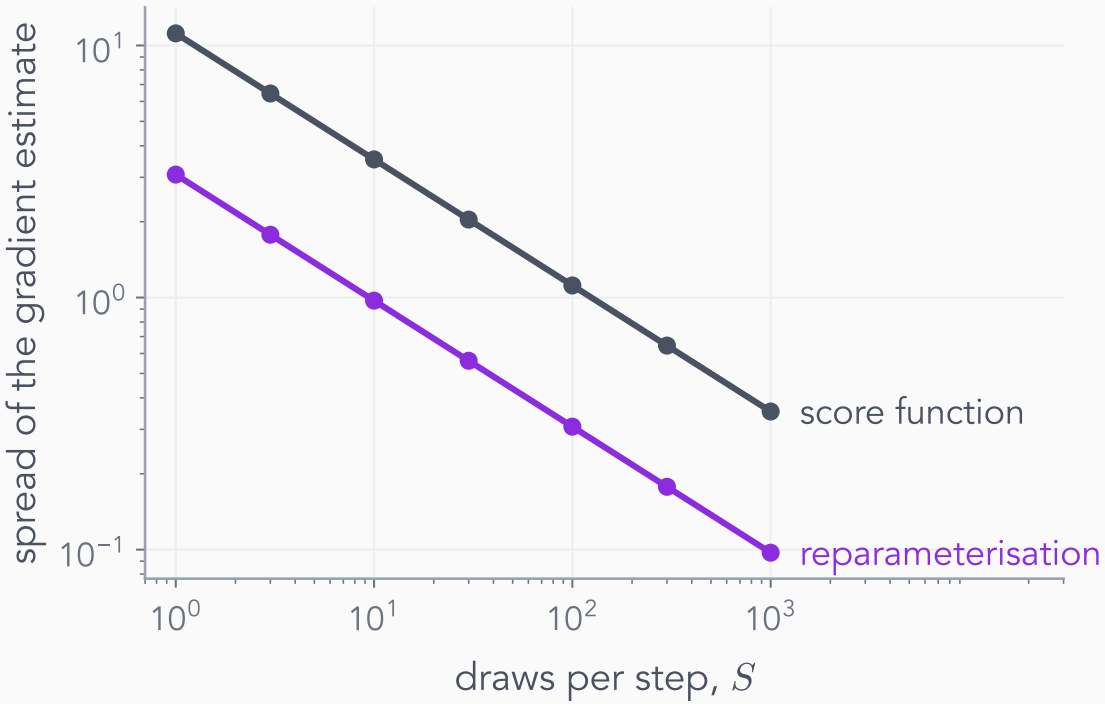
Reparameterisation is the low-noise choice



$\partial\,\mathrm{ELBO}/\partial m$ at $q = \mathcal{N}(0, 1)$. Exact: 3.0 (push m up, towards more heads).

ON THE COIN'S ELBO, THE SAME STORY

Reparameterisation is the low-noise choice

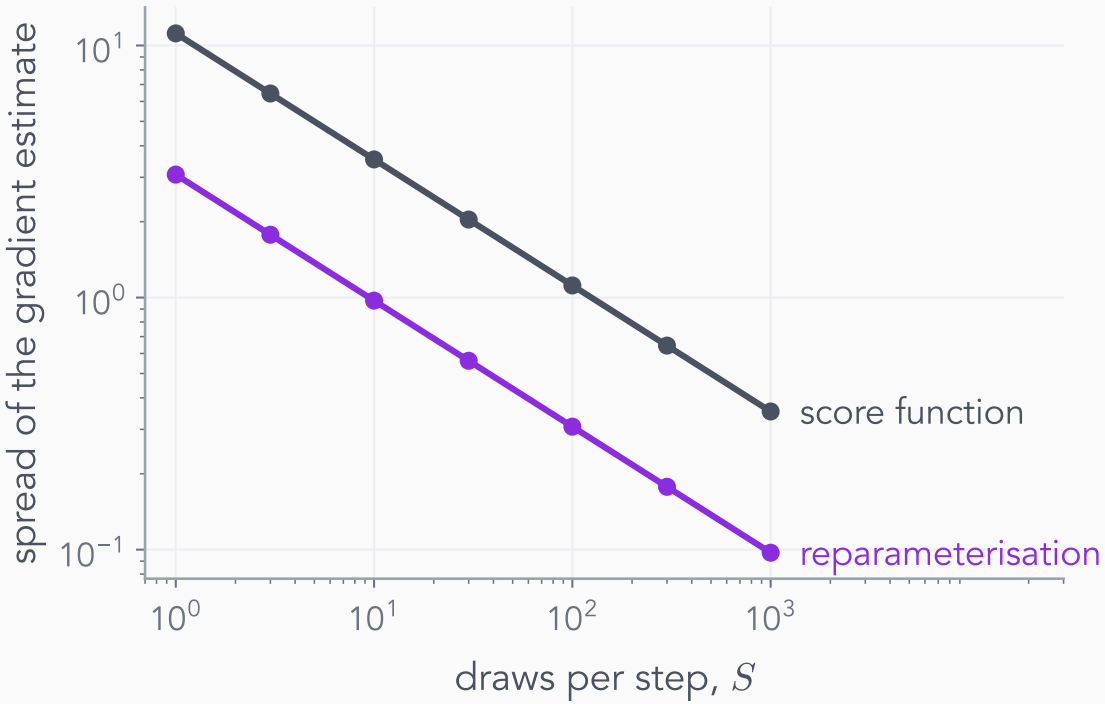


$\partial\,\mathrm{ELBO}/\partial m$ at $q = \mathcal{N}(0, 1)$. Exact: 3.0 (push m up, towards more heads).

Spread of a one-draw estimate: 11.17 (score function) vs 3.08 (reparameterisation).

ON THE COIN'S ELBO, THE SAME STORY

Reparameterisation is the low-noise choice



$\partial\,\mathrm{ELBO}/\partial m$ at $q = \mathcal{N}(0, 1)$. Exact: 3.0 (push m up, towards more heads).

Spread of a one-draw estimate: 11.17 (score function) vs 3.08 (reparameterisation).

Both shrink like $1/\sqrt{S}$, but the score function needs about 13× more draws to match.

That is why VI with Gaussians, and the VAE, use reparameterisation. The score function returns when z is discrete and $f'(z)$ does not exist.

IN PYTORCH (AS IN THE PML COURSE'S VI NOTEBOOK)

`rsample`, not `sample`

```
mu    = torch.tensor(1.0, requires_grad=True)
sigma = torch.tensor(0.5, requires_grad=True)
q = Normal(mu, sigma)

z = q.sample()
(z**2).backward()
# RuntimeError: element 0 of tensors does not require grad
#               and does not have a grad_fn

z = q.rsample()                # = mu + sigma * eps, eps ~ N(0, 1)
(z**2).backward()
mu.grad                        # 2 z: one reparameterised draw
```

The toy again: $\mu = 1, \sigma = 0.5, f(z) = z^2$.

IN PYTORCH (AS IN THE PML COURSE'S VI NOTEBOOK)

`rsample`, not `sample`

```
mu    = torch.tensor(1.0, requires_grad=True)
sigma = torch.tensor(0.5, requires_grad=True)
q = Normal(mu, sigma)

z = q.sample()
(z**2).backward()
# RuntimeError: element 0 of tensors does not require grad
#               and does not have a grad_fn

z = q.rsample()                 # = mu + sigma * eps, eps ~ N(0, 1)
(z**2).backward()
mu.grad                         # 2 z: one reparameterised draw
```

`sample()` returns a plain number: the graph is cut at z, so no gradient can reach μ.

`rsample`, not `sample`

```
mu    = torch.tensor(1.0, requires_grad=True)
sigma = torch.tensor(0.5, requires_grad=True)
q = Normal(mu, sigma)

z = q.sample()
(z**2).backward()
# RuntimeError: element 0 of tensors does not require grad
#               and does not have a grad_fn

z = q.rsample()                  # = mu + sigma * eps, eps ~ N(0, 1)
(z**2).backward()
mu.grad                          # 2 z: one reparameterised draw
```

`rsample()` ("reparameterised sample") draws ε and returns $\mu + \sigma\varepsilon$: same distribution, but a graph from z back to μ and σ.

Writing `mu + sigma * torch.randn(...)` by hand does exactly the same thing.

The whole loop: two numbers, gradient ascent on the ELBO

```
def neg_elbo(q, S=10):
    z = q.rsample((S,))                          # S reparameterised draws
    return -(log_joint(z) - q.log_prob(z)).mean()

m     = torch.tensor(0.0, requires_grad=True)   # start at the prior
log_s = torch.tensor(0.0, requires_grad=True)   # s = exp(log_s) = 1
opt = torch.optim.Adam([m, log_s], lr=0.05)

for step in range(400):
    loss = neg_elbo(Normal(m, log_s.exp()))
    opt.zero_grad(); loss.backward(); opt.step()
```

The loss: minus the ELBO estimate of two slides ago, now with `rsample` so gradients flow.

The whole loop: two numbers, gradient ascent on the ELBO

```
def neg_elbo(q, S=10):
    z = q.rsample((S,))                          # S reparameterised draws
    return -(log_joint(z) - q.log_prob(z)).mean()

m     = torch.tensor(0.0, requires_grad=True)    # start at the prior
log_s = torch.tensor(0.0, requires_grad=True)    # s = exp(log_s) = 1
opt = torch.optim.Adam([m, log_s], lr=0.05)

for step in range(400):
    loss = neg_elbo(Normal(m, log_s.exp()))
    opt.zero_grad(); loss.backward(); opt.step()
```

Only q is learned: two numbers. We learn $\log s$ so that s stays positive.

The whole loop: two numbers, gradient ascent on the ELBO

```
def neg_elbo(q, S=10):
    z = q.rsample((S,))                          # S reparameterised draws
    return -(log_joint(z) - q.log_prob(z)).mean()

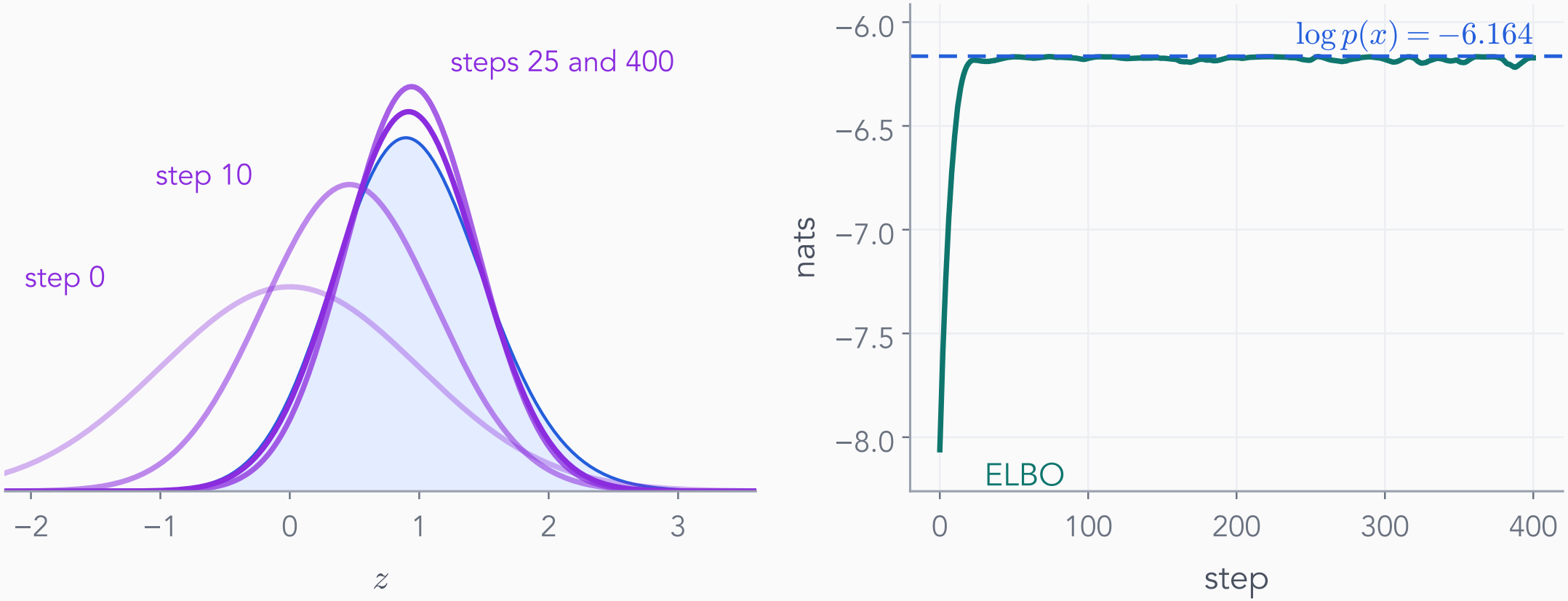
m     = torch.tensor(0.0, requires_grad=True)    # start at the prior
log_s = torch.tensor(0.0, requires_grad=True)    # s = exp(log_s) = 1
opt = torch.optim.Adam([m, log_s], lr=0.05)

for step in range(400):
    loss = neg_elbo(Normal(m, log_s.exp()))
    opt.zero_grad(); loss.backward(); opt.step()
```

The same three-line loop as fitting any model. 400 steps of 10 draws each: $m = 0.92$, $s = 0.54$. The best Gaussian: 0.94, 0.58.

PRESS PLAY

Watch q climb the ELBO, live



Left: q (purple) slides onto the posterior. Right: the ELBO climbs to the dashed $\log p(x) = -6.164$ and stops just under it.

THE RECORDED RUN, IN NUMBERS

Most of the gap closes in 25 steps

step	m	s	ELBO	gap $= \mathrm{KL}(q \,\|\, \text{posterior})$
0 · the prior	0.0	1.0	−8.061	1.897

THE RECORDED RUN, IN NUMBERS

Most of the gap closes in 25 steps

step	m	s	ELBO	gap $= \mathrm{KL}(q \,\|\, \text{posterior})$
0 · the prior	0.0	1.0	−8.061	1.897
10	0.46	0.67	−6.549	0.386
25	0.94	0.51	−6.184	0.02

THE RECORDED RUN, IN NUMBERS

Most of the gap closes in 25 steps

step	m	s	ELBO	gap $= \mathrm{KL}(q \,\|\, \text{posterior})$
0 · the prior	0.0	1.0	−8.061	1.897
10	0.46	0.67	−6.549	0.386
25	0.94	0.51	−6.184	0.02
50	0.96	0.57	−6.167	0.003
100	0.87	0.57	−6.174	0.01
400	0.92	0.54	−6.172	0.008

THE RECORDED RUN, IN NUMBERS

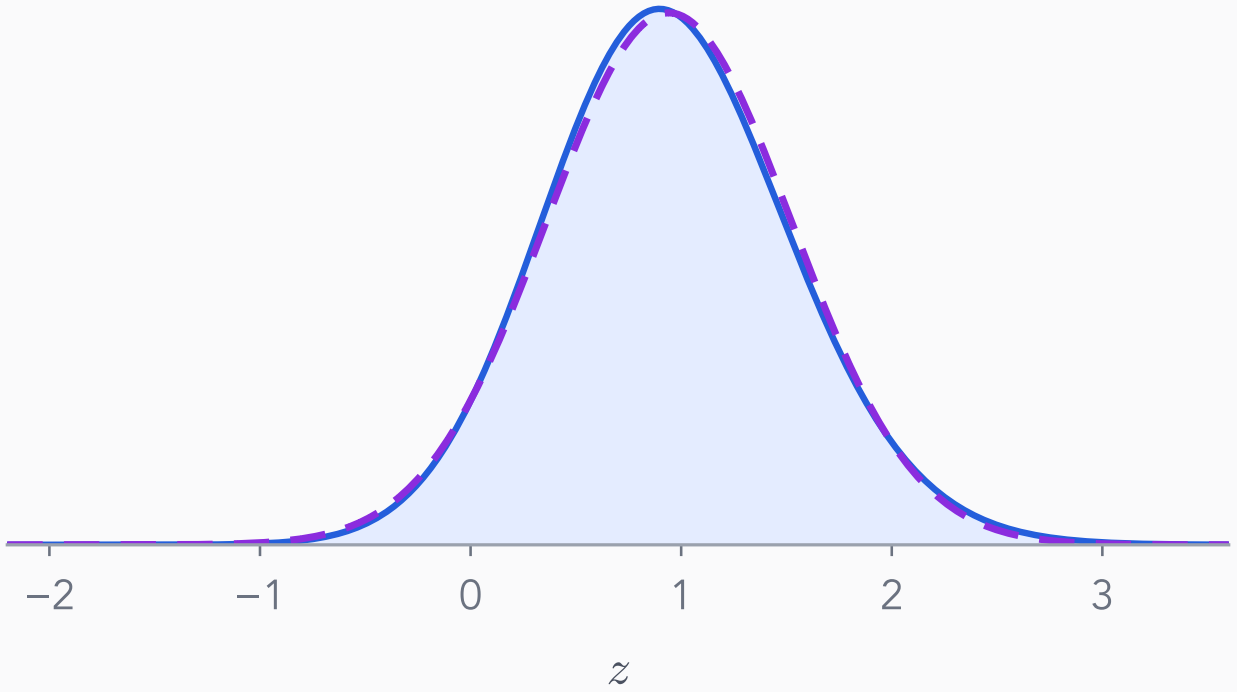
Most of the gap closes in 25 steps

step	m	s	ELBO	gap $= \mathrm{KL}(q \,\|\, \text{posterior})$
0 · the prior	0.0	1.0	−8.061	1.897
10	0.46	0.67	−6.549	0.386
25	0.94	0.51	−6.184	0.02
50	0.96	0.57	−6.167	0.003
100	0.87	0.57	−6.174	0.01
400	0.92	0.54	−6.172	0.008
best possible	**0.94**	**0.58**	**−6.166**	**0.0019**

> After step 25 the noise from 10 draws keeps q jittering within about 0.01 nats of the best. One draw per step works too: averaged over its last 100 steps, $m = 0.94$, $s = 0.58$.

HOW GOOD IS THE ANSWER?

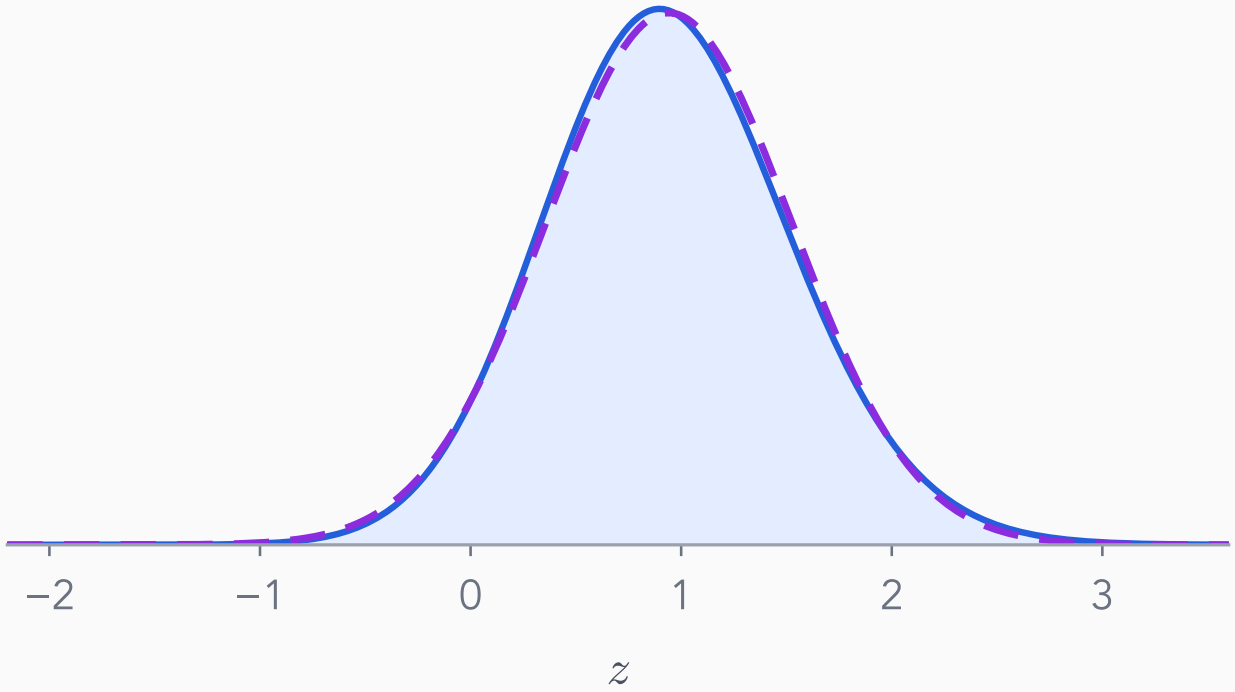
For this posterior, a bell curve is almost perfect



	exact posterior	best q
mean of z	0.94	0.94
spread of z	0.586	0.583
chance the next toss is heads	0.706	0.706

HOW GOOD IS THE ANSWER?

For this posterior, a bell curve is almost perfect



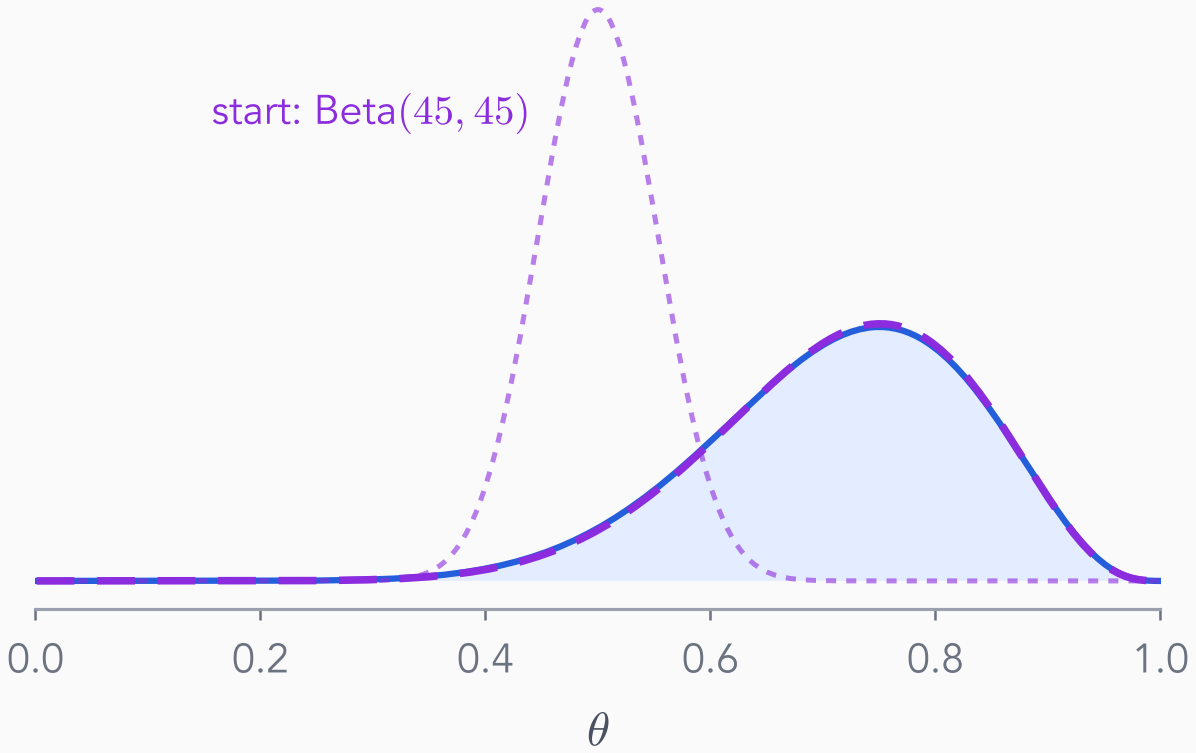
	exact posterior	best q
mean of z	0.94	0.94
spread of z	0.586	0.583
chance the next toss is heads	0.706	0.706

The leftover gap, 0.0019 nats, is the posterior's slight right skew, which no bell curve can match.

VI found, without ever computing $p(x)$, an answer that matches the exact one to two decimals. Part 7: shapes where it cannot.

A CHECK WHERE THE ANSWER IS KNOWN (FROM THE PML COURSE)

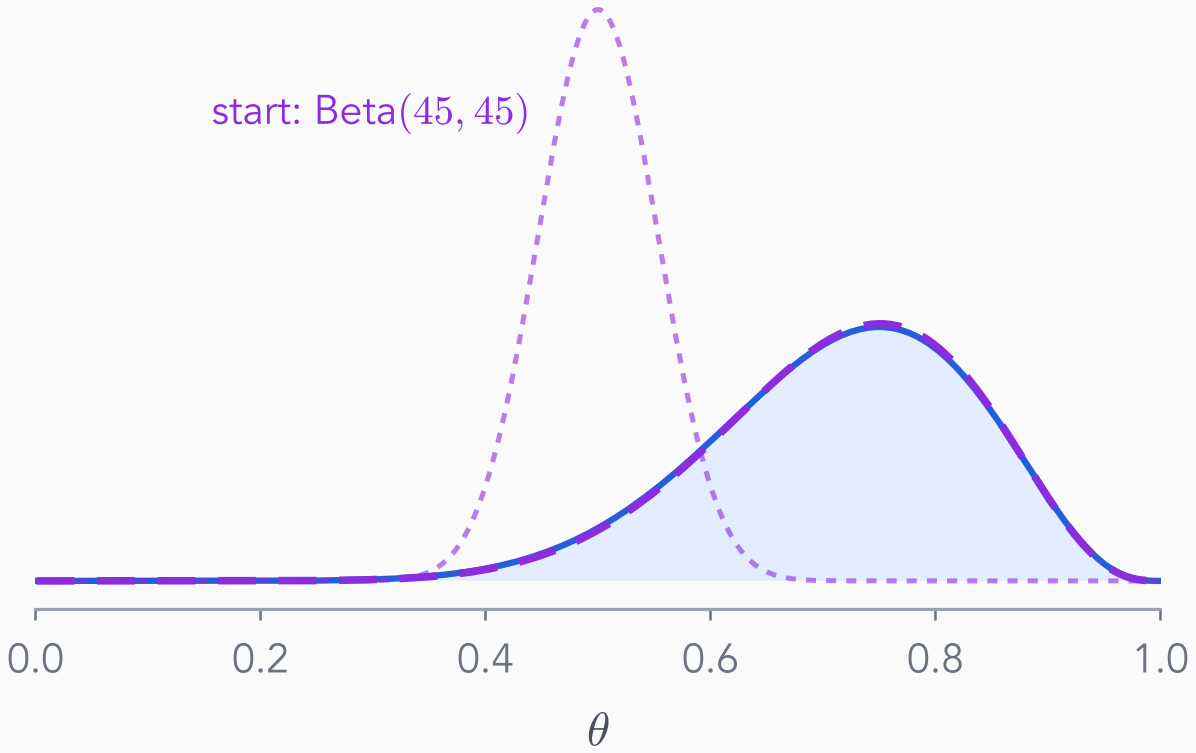
On Part 1's coin, a Beta q finds the exact posterior



Back to the $\mathrm{Beta}(2, 2)$ prior on θ, whose posterior we know: $\mathrm{Beta}(10, 4)$.

A CHECK WHERE THE ANSWER IS KNOWN (FROM THE PML COURSE)

On Part 1's coin, a Beta q finds the exact posterior

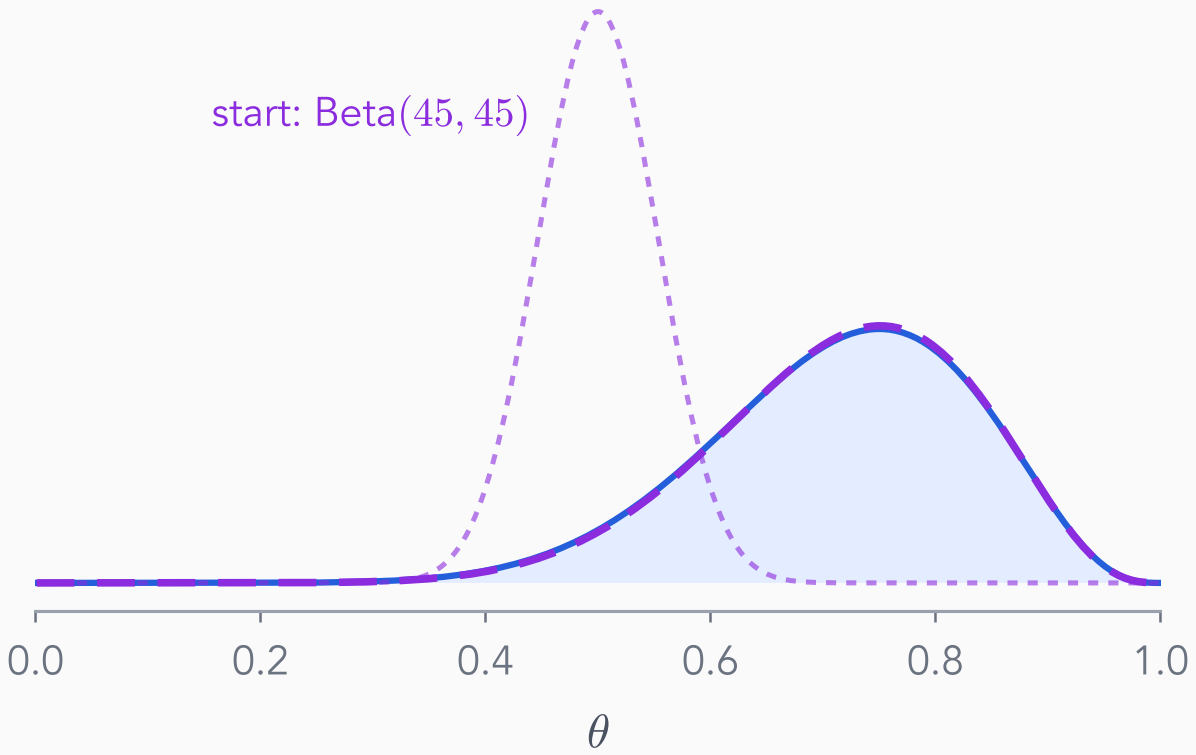


Back to the $\mathrm{Beta}(2, 2)$ prior on θ, whose posterior we know: $\mathrm{Beta}(10, 4)$.

Let $q = \mathrm{Beta}(\alpha, \beta)$, start at $\mathrm{Beta}(45, 45)$, and run the same loop: `rsample`, 100 draws per step, Adam, 3,000 steps.

A CHECK WHERE THE ANSWER IS KNOWN (FROM THE PML COURSE)

On Part 1's coin, a Beta q finds the exact posterior



Back to the $\mathrm{Beta}(2, 2)$ prior on θ, whose posterior we know: $\mathrm{Beta}(10, 4)$.

Let $q = \mathrm{Beta}(\alpha, \beta)$, start at $\mathrm{Beta}(45, 45)$, and run the same loop: `rsample`, 100 draws per step, Adam, 3,000 steps.

It ends near $\mathrm{Beta}(10.24,\ 4.06)$: ELBO −6.167 against $\log p(x) = -6.167$, a gap of 0.0002 nats.

> **When the family contains the posterior, VI finds it. The gap is a measure of how much the family is missing.**

PART 7 OF 8

Two numbers and beyond: where a simple q falls short

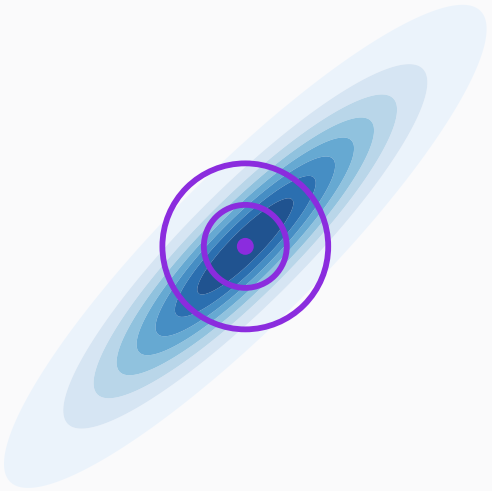
The VAE's q is a Gaussian with **one width per coordinate**, and no tilt. Fit that kind of q to shapes in a plane, and see exactly what it gets wrong, and why.

- ✓ Bayes: prior, likelihood, posterior
- ✓ Why the posterior is hard
- ✓ Averages from samples
- ✓ KL divergence
- ✓ Variational inference and the ELBO
- ✓ Climbing the ELBO: gradients through samples
- 7 **Beyond one number**
- 8 Many data points: towards the VAE

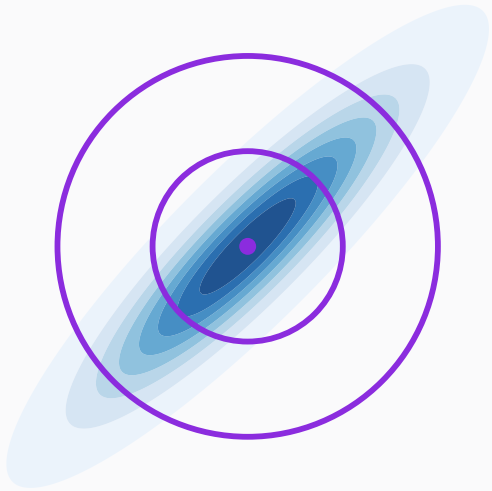
A TILTED TARGET, A q THAT CANNOT TILT

Mean-field q on a correlated Gaussian: too narrow

minimise $\mathrm{KL}(q \,\|\, p)$: too narrow



minimise $\mathrm{KL}(p \,\|\, q)$: too wide

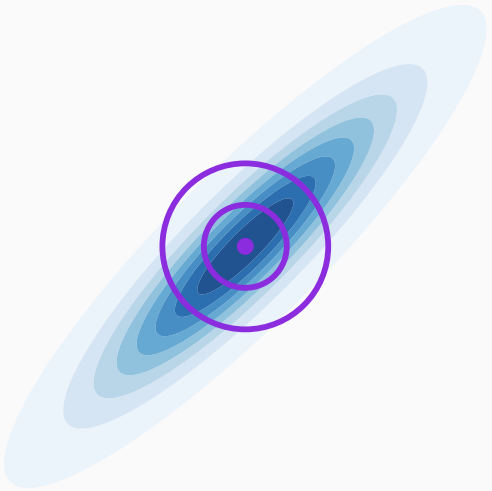


$$q(z_1, z_2) = \mathcal{N}(z_1;\, m_1, s_1^2)\, \mathcal{N}(z_2;\, m_2, s_2^2)$$

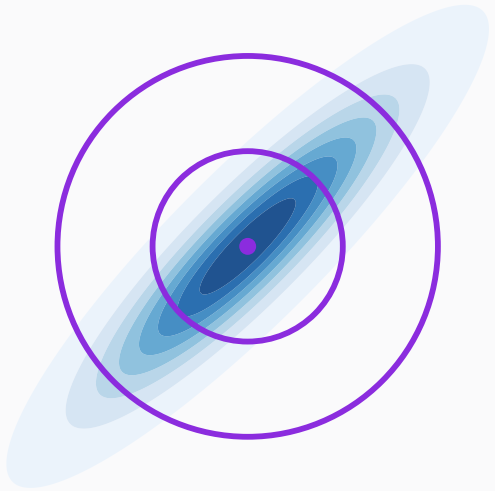
One width per coordinate, independent under q: **mean-field**. Target: correlation $\rho = 0.9$, each coordinate's spread 1.

Mean-field q on a correlated Gaussian: too narrow

minimise $\mathrm{KL}(q \,\|\, p)$: too narrow



minimise $\mathrm{KL}(p \,\|\, q)$: too wide



$$q(z_1, z_2) = \mathcal{N}(z_1;\, m_1, s_1^2)\, \mathcal{N}(z_2;\, m_2, s_2^2)$$

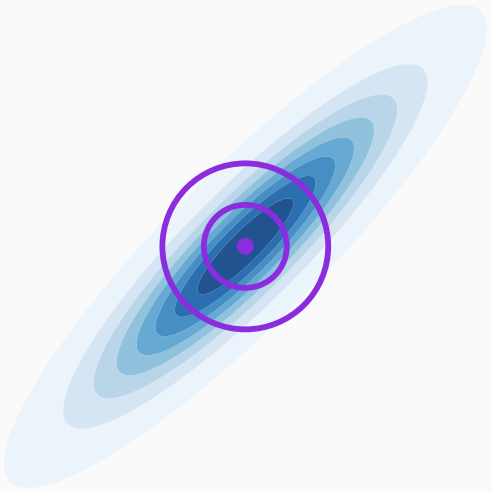
One width per coordinate, independent under q: **mean-field**. Target: correlation $\rho = 0.9$, each coordinate's spread 1.

Left, VI's direction: q must not put mass where p is thin, so it shrinks inside the ellipse. Width $\sqrt{1 - \rho^2} = 0.436$, not 1.

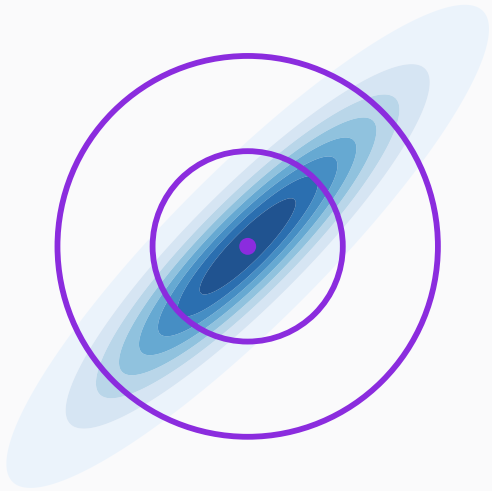
A TILTED TARGET, A q THAT CANNOT TILT

Mean-field q on a correlated Gaussian: too narrow

minimise $\mathrm{KL}(q \,\|\, p)$: too narrow



minimise $\mathrm{KL}(p \,\|\, q)$: too wide



$$q(z_1, z_2) = \mathcal{N}(z_1;\, m_1, s_1^2)\, \mathcal{N}(z_2;\, m_2, s_2^2)$$

One width per coordinate, independent under q: **mean-field**. Target: correlation $\rho = 0.9$, each coordinate's spread 1.

Left, VI's direction: q must not put mass where p is thin, so it shrinks inside the ellipse. Width $\sqrt{1-\rho^2} = 0.436$, not 1.

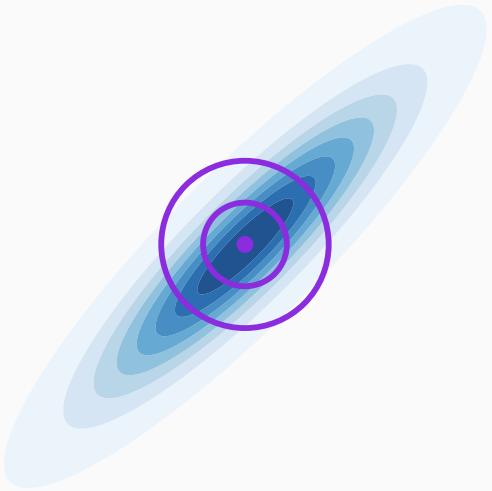
Right, the other direction: wide enough to cover, width 1, but lots of mass where p has none.

> **Mean-field VI underestimates uncertainty. Both fits pay the same KL here, 0.830 nats, by opposite mistakes.**

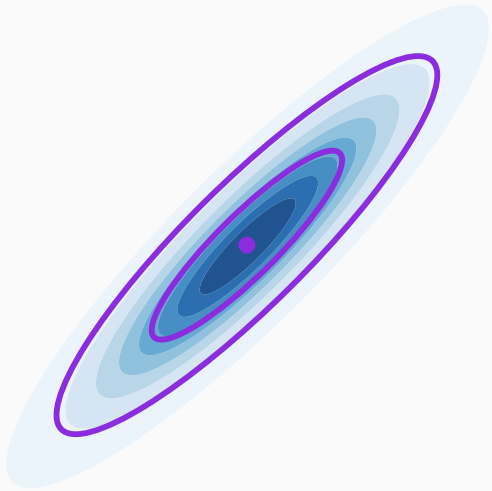
GIVE q A TILT

A full-covariance q fits the tilted target exactly

diagonal q (mean-field)



full-covariance q



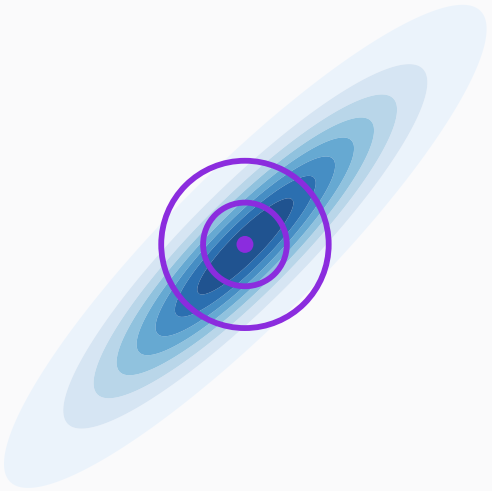
$$z = m + L\varepsilon, \quad L = \begin{pmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{pmatrix}, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Reparameterisation in 2-D: one extra number, ℓ_{21}, lets q tilt.

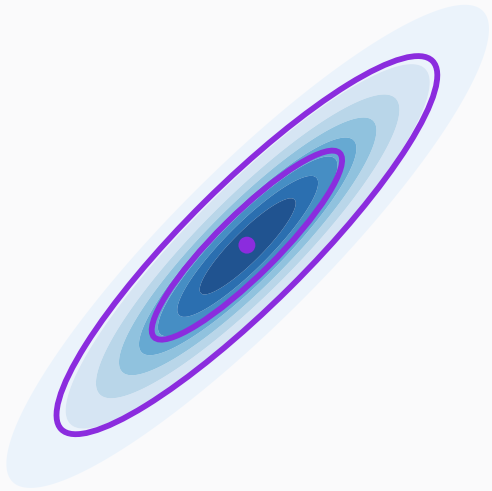
GIVE q A TILT

A full-covariance q fits the tilted target exactly

diagonal q (mean-field)



full-covariance q



$$z = m + L\varepsilon, \quad L = \begin{pmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{pmatrix}, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Reparameterisation in 2-D: one extra number, ℓ_{21}, lets q tilt.

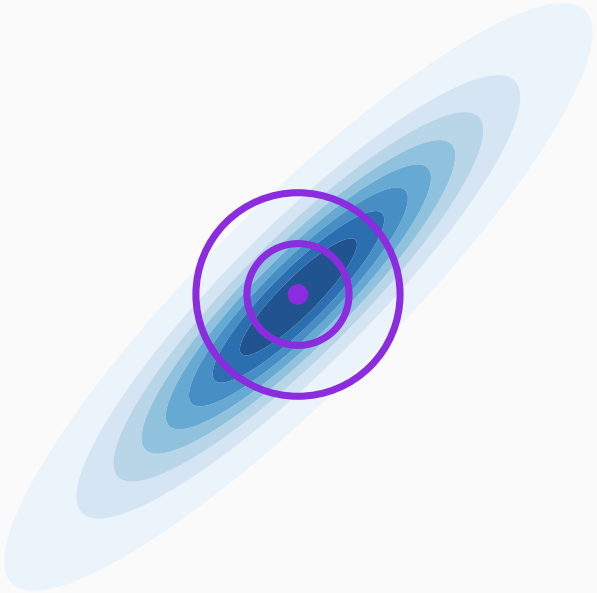
Both fitted by the same loop (reverse KL, `rsample`, Adam). Diagonal: KL 0.828. Full: KL 0.000, correlation 0.90.

With d numbers in z: d widths for a diagonal q, $d(d+1)/2$ for a full one. The VAE keeps diagonal: cheaper, and its decoder adapts.

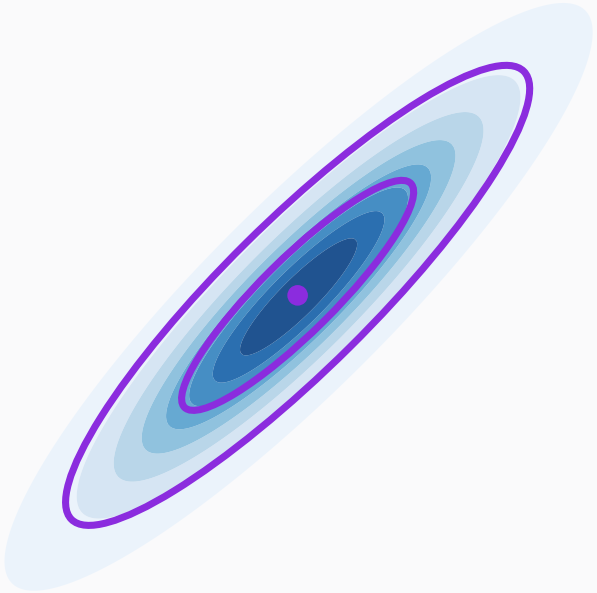
TRY IT

Three targets, two kinds of q, press play

diagonal q (mean-field)



full-covariance q



A CURVED TARGET

A Gaussian can stretch and tilt, but it cannot bend

diagonal q



full-covariance q



A banana: $z_1 \sim \mathcal{N}(0, 1.2^2)$, then z_2 centred on the curve $0.6\, z_1^2 - 1.2$.

A CURVED TARGET

A Gaussian can stretch and tilt, but it cannot bend

diagonal q



full-covariance q



A banana: $z_1 \sim \mathcal{N}(0, 1.2^2)$, then z_2 centred on the curve $0.6\, z_1^2 - 1.2$.

Diagonal q: KL 0.542. Full q: KL 0.542. The same.

A CURVED TARGET

A Gaussian can stretch and tilt, but it cannot bend

diagonal q



full-covariance q



A banana: $z_1 \sim \mathcal{N}(0, 1.2^2)$, then z_2 centred on the curve $0.6\, z_1^2 - 1.2$.

Diagonal q: KL 0.542. Full q: KL 0.542. The same.

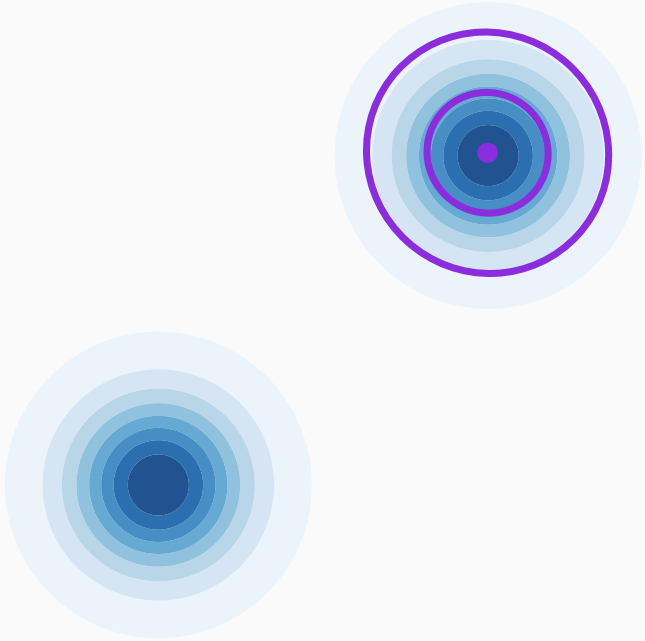
The banana is symmetric, so the best tilt is no tilt. Both q's sit in the bend and miss the arms.

More parameters of the same kind do not help. The family is the limit, not the optimiser.

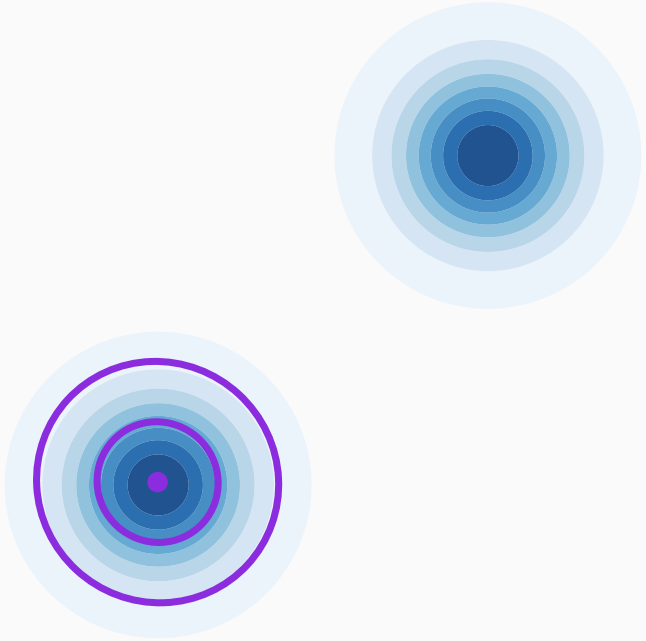
TWO SEPARATE BUMPS, IN A PLANE

Reverse KL picks one bump, and pays $\log 2$

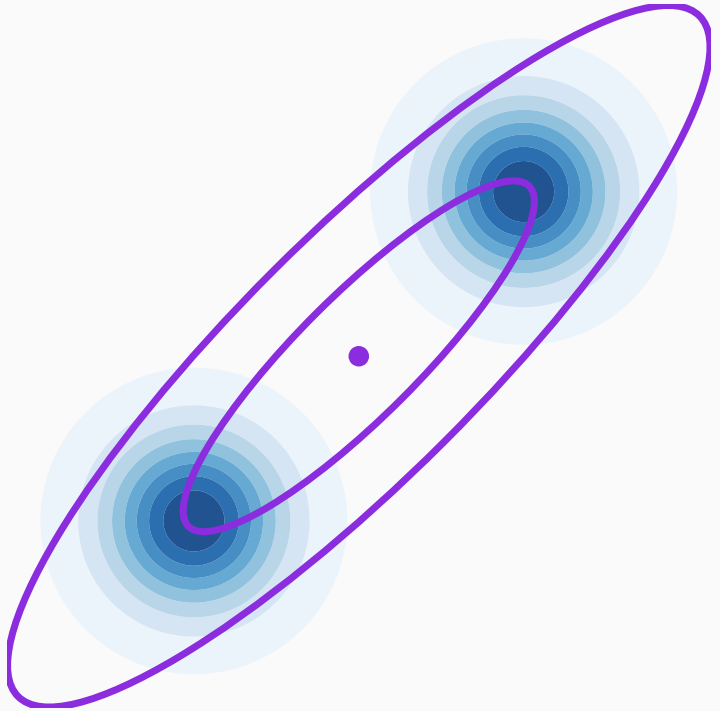
$\mathrm{KL}(q \,\|\, p)$, start right of centre



$\mathrm{KL}(q \,\|\, p)$, start left of centre



$\mathrm{KL}(p \,\|\, q)$: cover both



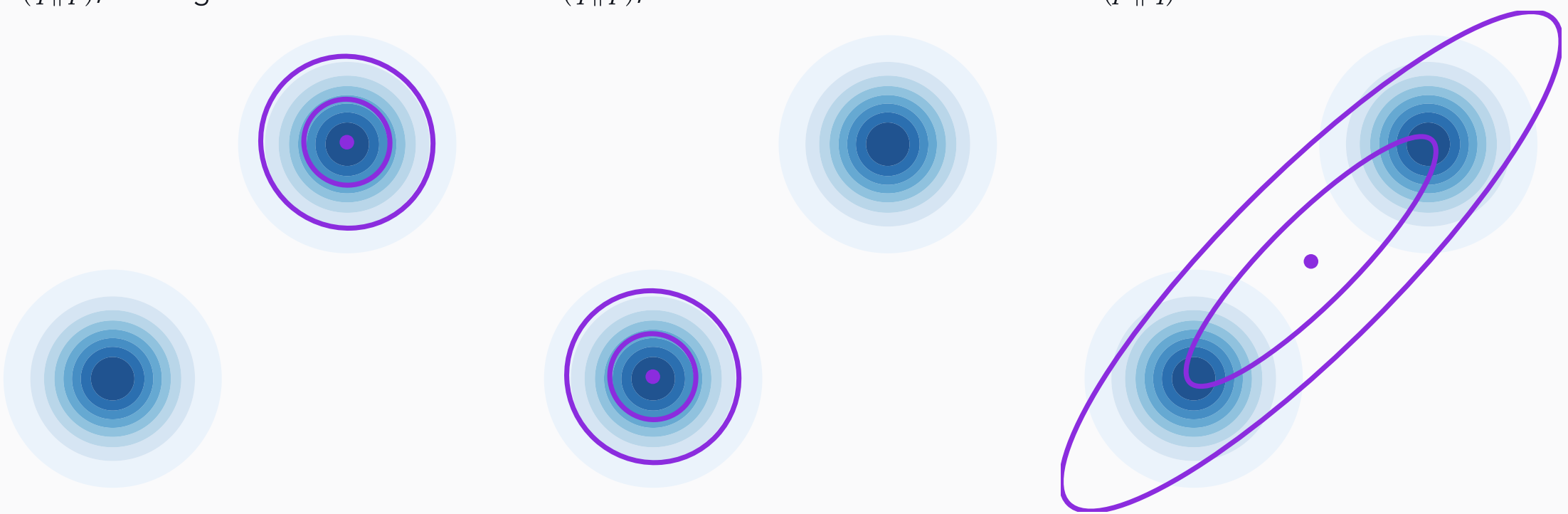
TWO SEPARATE BUMPS, IN A PLANE

Reverse KL picks one bump, and pays $\log 2$

$\mathrm{KL}(q \,\|\, p)$, start right of centre

$\mathrm{KL}(q \,\|\, p)$, start left of centre

$\mathrm{KL}(p \,\|\, q)$: cover both

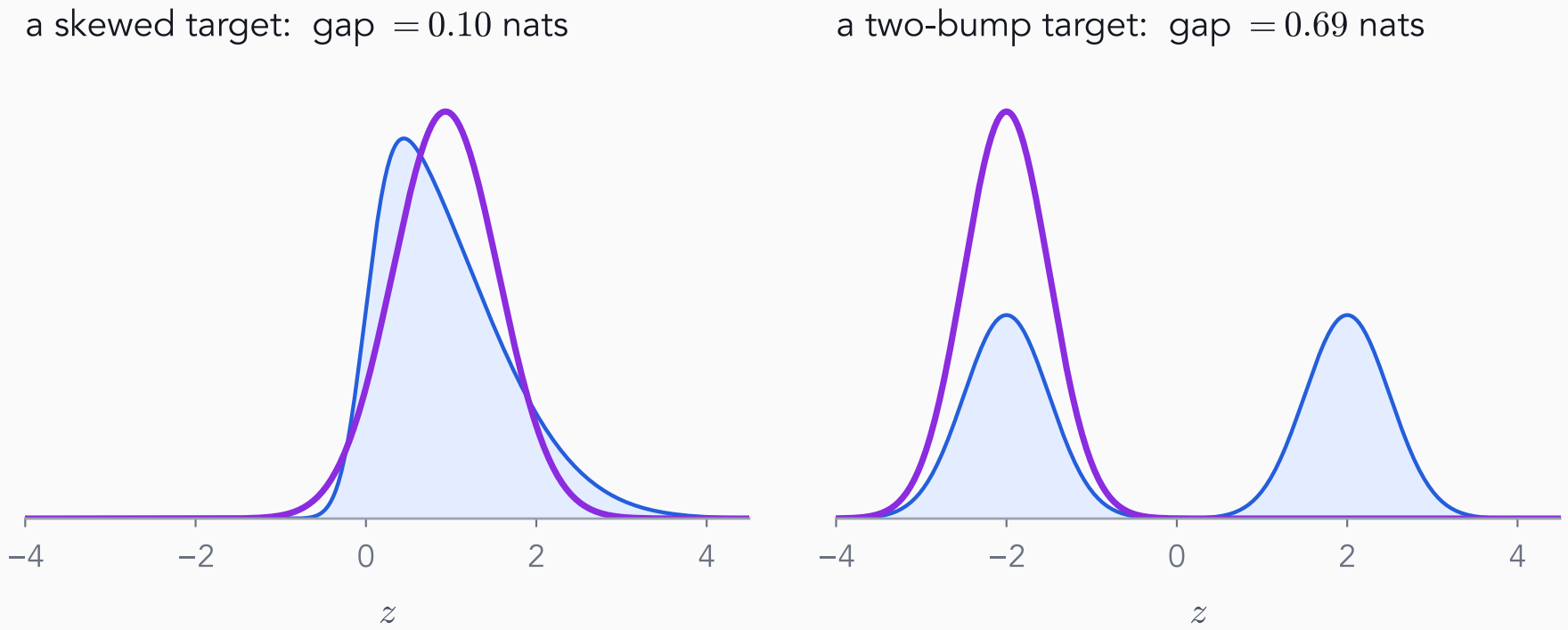


Start a little right of centre: the upper bump, KL 0.691. A little left: the lower bump, KL 0.691. Both $\approx \log 2 = 0.693$.

q fits half of p perfectly and ignores the other half: $\log 2$ is the price of missing half the mass. The forward KL covers both, through the empty middle.

BACK TO ONE NUMBER

The gap is the KL that no Gaussian can remove

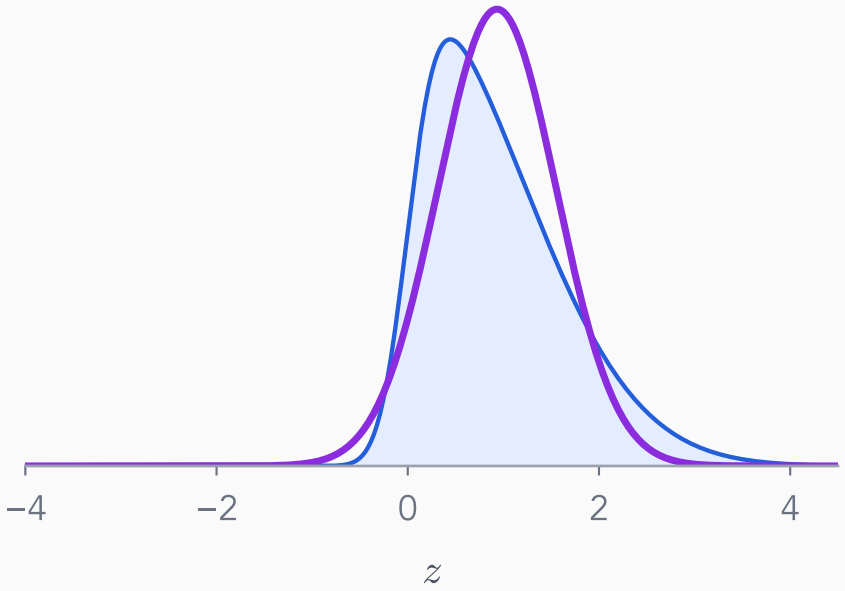


Skewed target: the best Gaussian, $\mathcal{N}(0.93,\ 0.62^2)$, leaves 0.10 nats.

BACK TO ONE NUMBER

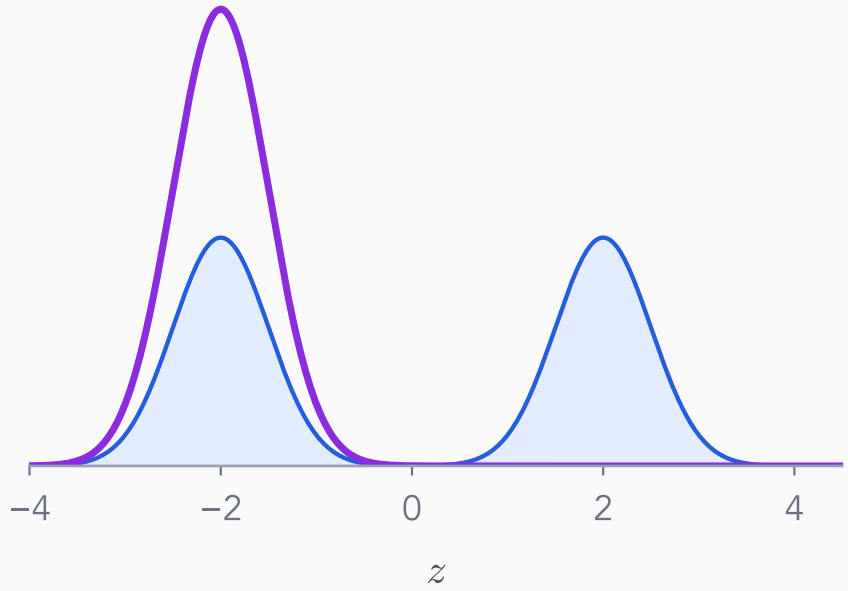
The gap is the KL that no Gaussian can remove

a skewed target: gap $= 0.10$ nats



Skewed target: the best Gaussian, $\mathcal{N}(0.93,\ 0.62^2)$, leaves 0.10 nats.

a two-bump target: gap $= 0.69$ nats

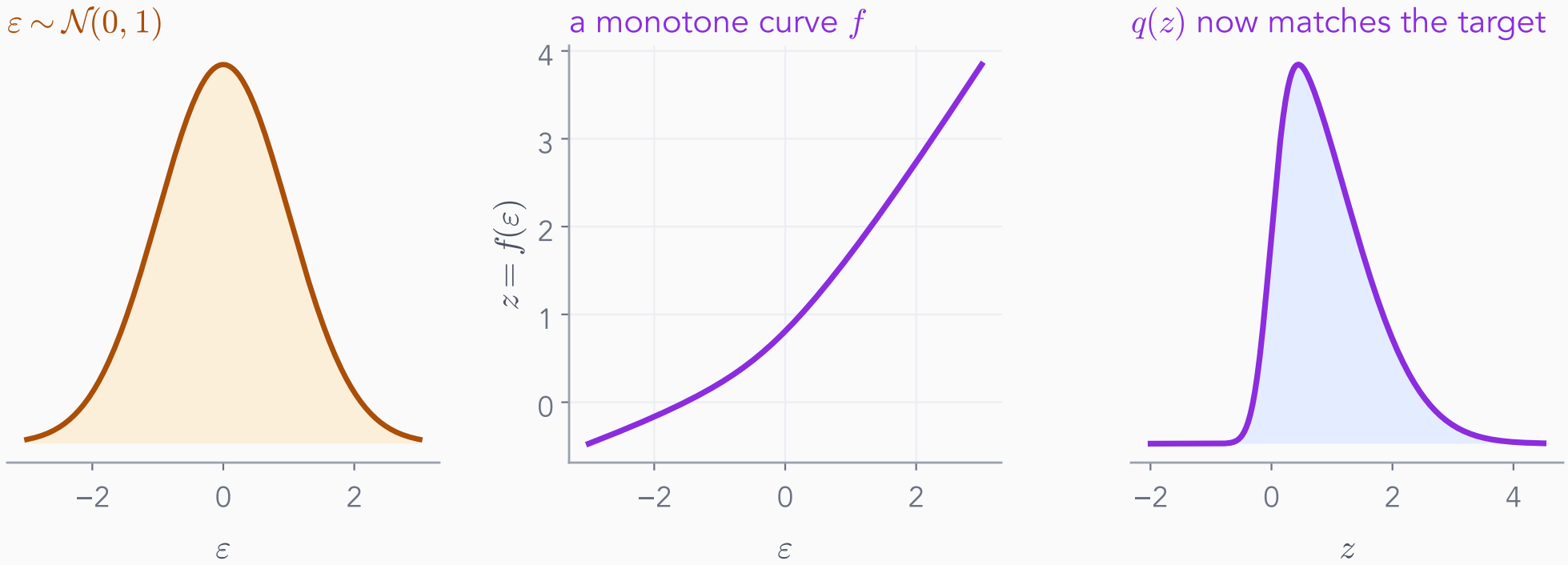


Two bumps: the best Gaussian is one bump, $\mathcal{N}(-2.0,\ 0.50^2)$, and leaves 0.693 nats $= \log 2$.

> **The ELBO can never reach $\log p(x)$ here: the gap stays at least this big, whatever the optimiser does.**

THE FIX: A RICHER q

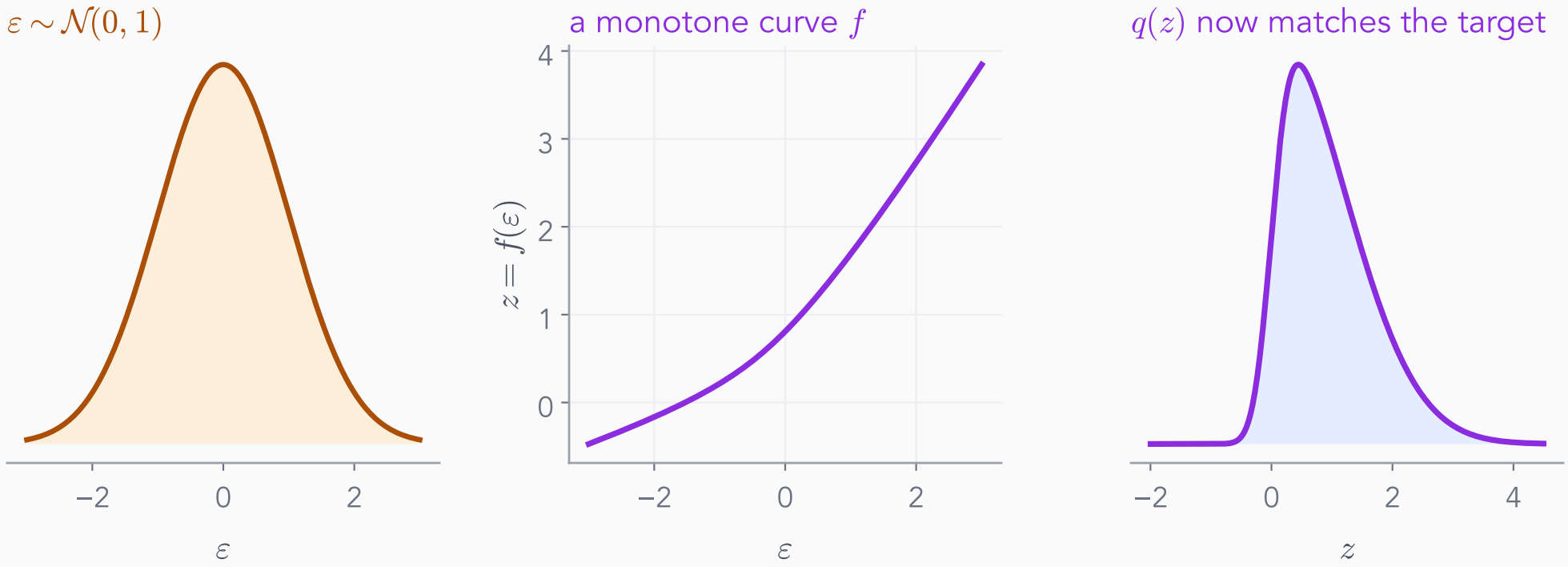
Push simple noise through a curve: a flow



$z = f(\varepsilon)$ with $\varepsilon \sim \mathcal{N}(0,1)$ and f monotone. The right curve makes q match the skewed target **exactly**.

THE FIX: A RICHER q

Push simple noise through a curve: a flow



$z = f(\varepsilon)$ with $\varepsilon \sim \mathcal{N}(0,1)$ and f monotone. The right curve makes q match the skewed target **exactly**.

Normalizing flows learn f as an invertible neural network, so $\log q(z)$ stays computable. Reparameterisation still works: z is a differentiable function of ε.

> **Richer families (flows, mixtures) shrink the gap; the ELBO and the loop do not change at all.**

PART 8 OF 8

Many data points: learn to predict q

So far: one coin, one q, 400 steps. A dataset has thousands of items, each with its **own** posterior. Running VI separately for each is wasteful. Learn a function from the data to q instead.

- ✓ Bayes: prior, likelihood, posterior
- ✓ Why the posterior is hard
- ✓ Averages from samples
- ✓ KL divergence
- ✓ Variational inference and the ELBO
- ✓ Climbing the ELBO: gradients through samples
- ✓ Beyond one number
- 8 **Many data points: towards the VAE**

A DATASET OF COINS

500 coins, each with its own hidden z

For each coin n: draw $z_n \sim \mathcal{N}(0, 1)$, toss it 10 times with $\theta_n = \mathrm{sigmoid}(z_n)$. The data: x_n, its ten tosses.

how many coins showed each number of heads



A DATASET OF COINS

500 coins, each with its own hidden z

For each coin n: draw $z_n \sim \mathcal{N}(0, 1)$, toss it 10 times with $\theta_n = \mathbf{sigmoid}(z_n)$. The data: x_n, its ten tosses.

Each coin has its own posterior $p(z_n \mid x_n)$, and so its own best $q_n = \mathcal{N}(m_n, s_n^2)$.

how many coins showed each number of heads



A DATASET OF COINS

500 coins, each with its own hidden z

For each coin n: draw $z_n \sim \mathcal{N}(0, 1)$, toss it 10 times with $\theta_n = \mathbf{sigmoid}(z_n)$. The data: x_n, its ten tosses.

Each coin has its own posterior $p(z_n \mid x_n)$, and so its own best $q_n = \mathcal{N}(m_n, s_n^2)$.

Per-coin VI: 500 separate loops of 400 steps. A new coin: another loop.

how many coins showed each number of heads



But each coin's best q depends only on its tosses. So learn the map from x to (m, s) once.

AMORTISED INFERENCE

An encoder: a small network from x to q's two numbers



AMORTISED INFERENCE

An encoder: a small network from x to q's two numbers

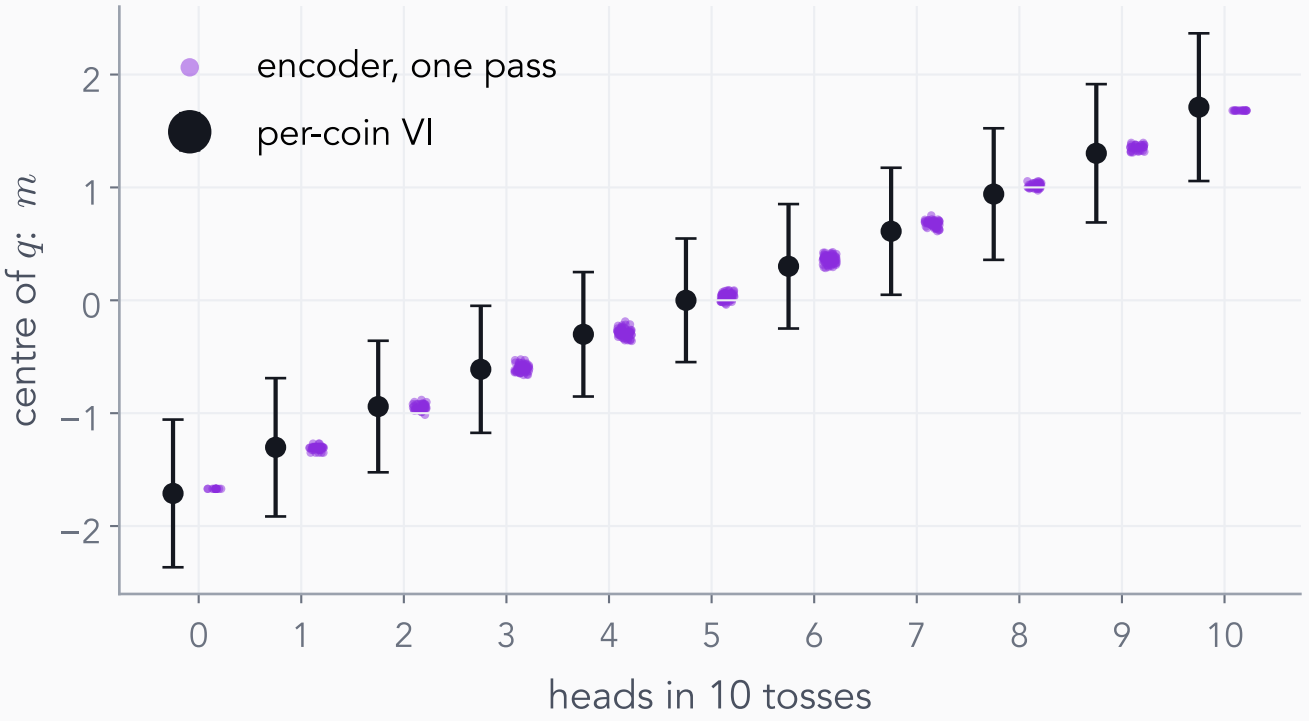



```
m, log_s = encoder(X).unbind(-1)                    # X: [500, 10] tosses, one row per coin
z   = m + log_s.exp() * torch.randn(500)          # one draw per coin
rec = h * F.logsigmoid(z) + (10 - h) * F.logsigmoid(-z)   # h: heads per coin
kl  = 0.5 * (m**2 + (2*log_s).exp() - 1 - 2*log_s)
loss = -(rec - kl).mean()                                 # minus the average ELBO
```

> Train the encoder's weights on the average ELBO over all coins: 3,000 Adam steps, one draw per coin. Then one forward pass gives any coin's q.

DOES IT WORK?

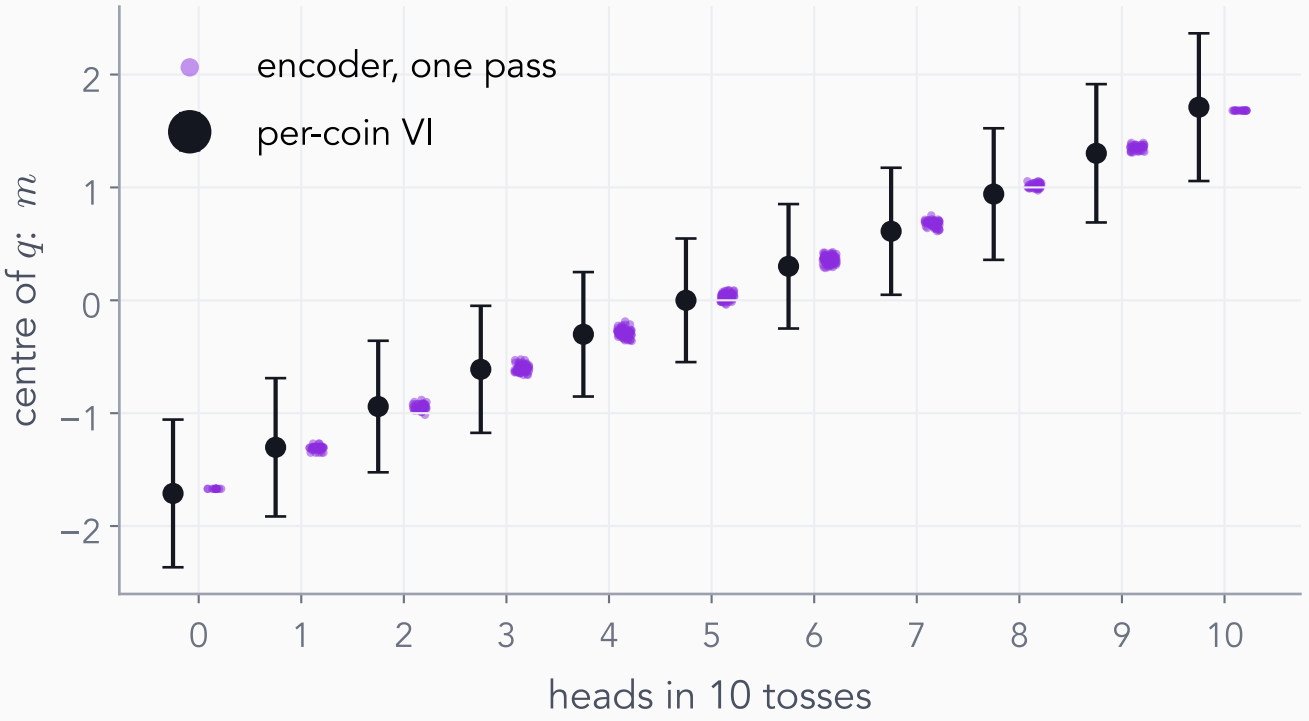
One pass of the encoder ≈ 400 steps of VI per coin



Black: per-coin VI, centre m with $\pm s$. Purple: the encoder's m for each of the 500 coins.

DOES IT WORK?

One pass of the encoder ≈ 400 steps of VI per coin

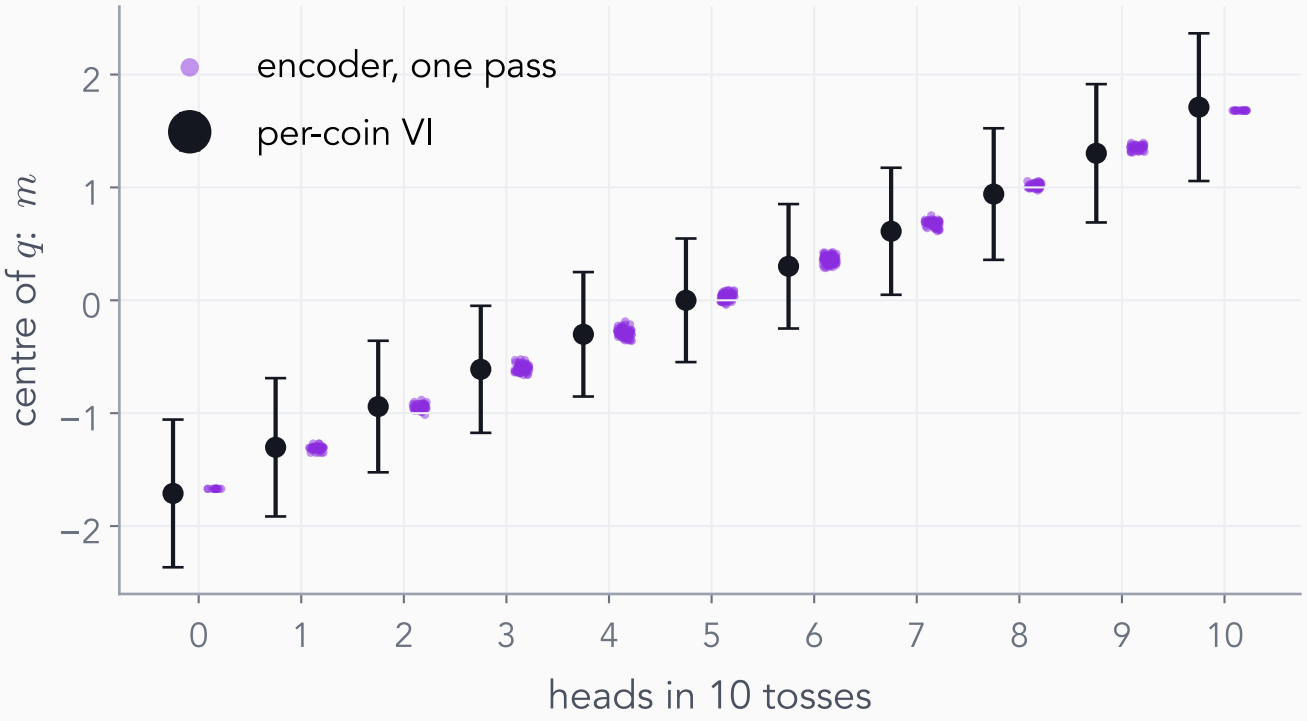


Black: per-coin VI, centre m with $\pm s$. Purple: the encoder's m for each of the 500 coins.

For the 33 coins with 8 heads: VI gives $m = 0.94$, the encoder 1.02; both give $s = 0.58$.

DOES IT WORK?

One pass of the encoder ≈ 400 steps of VI per coin



Black: per-coin VI, centre m with $\pm s$. Purple: the encoder's m for each of the 500 coins.

For the 33 coins with 8 heads: VI gives $m = 0.94$, the encoder 1.02; both give $s = 0.58$.

ELBO lost per coin, on average: 0.0045 nats (at most 0.031). That loss is the **amortisation gap**.

> **That network is the VAE's encoder. The VAE also learns the decoder (here sigmoid(z) was fixed), on the same ELBO.**

RECAP

What the VAE lecture will reuse

today, on toys	in the VAE
prior $p(z) = \mathcal{N}(0, 1)$ **on one number**	$p(z) = \mathcal{N}(0, I)$ on a code of a few numbers

RECAP

What the VAE lecture will reuse

today, on toys	in the VAE
prior $p(z) = \mathcal{N}(0, 1)$ **on one number**	$p(z) = \mathcal{N}(0, I)$ on a code of a few numbers
likelihood through $\mathrm{sigmoid}(z)$	the decoder network: 784 pixel probabilities
posterior with no formula; evidence intractable	the same, for every image

RECAP

What the VAE lecture will reuse

today, on toys	in the VAE
prior $p(z) = \mathcal{N}(0, 1)$ **on one number**	$p(z) = \mathcal{N}(0, I)$ on a code of a few numbers
likelihood through $\mathrm{sigmoid}(z)$	the decoder network: 784 pixel probabilities
posterior with no formula; evidence intractable	the same, for every image
$\log p(x) = \mathrm{ELBO} + \mathrm{KL}(q \,\|\, \mathrm{posterior})$	the loss is $-\mathrm{ELBO}$
$\mathbb{E}_q[\log p(x \mid z)]$ **from a few draws**	the reconstruction term, one draw per image
$\frac{1}{2}(m^2 + s^2 - 1 - \log s^2)$	the KL term, one per code coordinate

RECAP

What the VAE lecture will reuse

today, on toys	in the VAE
prior $p(z) = \mathcal{N}(0, 1)$ **on one number**	$p(z) = \mathcal{N}(0, I)$ on a code of a few numbers
likelihood through $\mathrm{sigmoid}(z)$	the decoder network: 784 pixel probabilities
posterior with no formula; evidence intractable	the same, for every image
$\log p(x) = \mathrm{ELBO} + \mathrm{KL}(q \,\|\, \text{posterior})$	the loss is $-\mathrm{ELBO}$
$\mathbb{E}_q[\log p(x \mid z)]$ **from a few draws**	the reconstruction term, one draw per image
$\frac{1}{2}(m^2 + s^2 - 1 - \log s^2)$	the KL term, one per code coordinate
$z = m + s\varepsilon$**,** `rsample`	$z = \mu + \sigma \odot \varepsilon$
diagonal q **is too narrow on tilted posteriors**	the VAE's q is diagonal
an encoder from x **to** (m, s)	the VAE's encoder, $q_\varphi(z \mid x)$

Next lecture: the same few lines of loss, with an image in place of ten tosses, and a decoder that is learned too.

BEFORE YOU LEAVE

Five questions

1. Why can't we compute $p(z \mid x)$ for the one-number decoder?

2. $\log p(x) = \mathrm{ELBO}(q) + \,?$

3. Why does VI minimise $\mathrm{KL}(q \| p)$, not $\mathrm{KL}(p \| q)$?

4. What does reparameterisation buy?

5. A diagonal Gaussian q fitted to a correlated target comes out too ...?

BEFORE YOU LEAVE

Five questions

1. Why can't we compute $p(z \mid x)$ for the one-number decoder?

   The evidence $\int p(z)\, p(x \mid z)\, dz$ has no formula. A grid works in 1-D but explodes with dimension.

2. $\log p(x) = \mathrm{ELBO}(q) + \,?$

   $\mathrm{KL}(q \,\|\, p(z \mid x)) \geq 0$. So the ELBO is a lower bound, and since $\log p(x)$ does not depend on q, raising the ELBO shrinks the KL.

3. Why does VI minimise $\mathrm{KL}(q \| p)$, not $\mathrm{KL}(p \| q)$?

   KL averages under its first argument: we can draw from q, not from the posterior. The price: q is mode-seeking and often too narrow.

4. What does reparameterisation buy?

   Gradients through random draws, with far less noise than the score function: 22× fewer draws on the toy, 13× on the coin.

5. A diagonal Gaussian q fitted to a correlated target comes out too ...?

   Narrow: width 0.436 instead of 1 for $\rho = 0.9$.